

UNIVERSIDADE ESTADUAL DO NORTE FLUMINENSE DARCY
RIBEIRO
LABORATÓRIO DE ENGENHARIA E EXPLORAÇÃO DE
PETRÓLEO
CENTRO DE CIÊNCIA E TECNOLOGIA

PROJETO DE ENGENHARIA
DESENVOLVIMENTO DE SOFTWARE
*SOFTWARE EDUCACIONAL COM INTERFACE INTERATIVA
PARA IMPLEMENTAÇÃO COMPUTACIONAL DO MÉTODO DAS
CARACTERÍSTICAS - SOLUÇÃO ANALÍTICA DE
BUCKLEY-LEVERETT*

Versão 2.0:
FABIANE DA SILVA BARROS
Prof. André Duarte Bueno

MACAÉ - RJ
Abril - 2026

Sumário

1	Introdução	14
1.1	Escopo do problema	15
1.2	Objetivos	16
1.3	Metodologia utilizada	17
2	Concepção	19
2.1	Nome do sistema/produto	20
2.2	Especificação	21
2.3	Requisitos	22
2.3.1	Requisitos funcionais	22
2.3.2	Requisitos não funcionais	23
2.4	Casos de uso	25
2.4.1	Diagrama de caso de uso geral	26
2.4.2	Diagrama de caso de uso específico	28
3	Elaboração	31
3.1	Análise de domínio	31
3.2	Formulação teórica	33
3.2.1	Padronização de Unidades	34
3.2.2	Fundamentação do Escoamento Bifásico e a Lei de Darcy Generalizada	35
3.2.3	Definições Auxiliares e Relações Constitutivas	35
3.2.4	Desenvolvimento Algébrico e Eliminação dos Gradientes de Pressão	36
3.2.5	Equação do Fluxo Fracionário Generalizada	37
3.2.6	Análise Dimensional e Simplificação do Modelo Matemático	38
3.2.7	Interpretação Física da Curva de Fluxo Fracionário: Uma Propriedade Constitutiva	41
3.2.8	A Lei de Conservação de Massa e o Acoplamento do Sistema	41
3.2.9	Análise Dimensional e a Equação de <i>Buckley-Leverett</i>	44
3.2.10	Solução Analítica pelo Método das Características (MOC)	45
3.2.11	Condição de Entropia e Soluções Descontínuas	46
3.2.12	Solução da Frente de Choque: A Construção de Welge	48
3.2.13	Cálculo dos Indicadores de Recuperação e Eficiência de Deslocamento	49
3.3	Identificação de pacotes – assuntos	50
3.4	Diagrama de pacotes – assuntos	51

4	AOO – Análise Orientada a Objeto	53
4.1	Diagramas de classes	53
4.1.1	Dicionário de classes	55
4.2	Diagrama de sequência – eventos e mensagens	56
4.2.1	Diagrama de sequência geral	56
4.2.2	Diagrama de sequência específico	58
4.3	Diagrama de comunicação – colaboração	60
4.4	Diagrama de estado	61
4.5	Diagrama de atividades	62
5	Projeto	63
5.1	Projeto do sistema	63
5.2	Projeto orientado a objeto – POO	68
5.3	Diagrama de componentes	72
5.4	Diagrama de implantação	73
5.4.1	Lista de características << <i>features</i> >>	74
5.4.2	Tabela classificação sistema	75
6	Ciclos de Planejamento/Detalhamento	77
6.1	Versão 1.0 - Prototipagem do Núcleo Analítico	77
6.2	Versão 2.1 - Migração para Interface Gráfica e Unificação	78
6.3	Versão 2.3 - Refinamento de Engenharia e Experiência do Usuário (UX)	79
7	Ciclos Construção - Implementação	82
7.1	Código fonte	82
8	Teste	163
8.1	Teste 1: Validação com Modelo Tabelado ((LAKE et al., 2014))	163
8.1.1	Arquivo de entrada de dados	164
8.1.2	Arquivo de saída de dados (resultados da tela)	164
8.1.3	Figuras	164
8.2	Teste 2: Gravidade e Teoria do Fluxo Fracionário (LAKE et al., 2014)	166
8.2.1	Arquivo de entrada de dados	167
8.2.2	Arquivo de saída de dados (resultados da tela)	167
8.2.3	Figuras	167
8.3	Teste 3: Validação do Modelo Racional LET (VALDEZ et al., 2020)	169
8.3.1	Arquivo de entrada de dados	171
8.3.2	Arquivo de saída de dados (resultados da tela)	171
8.3.3	Figuras	171
8.4	Teste 4: Validação do Modelo Exponencial de Chierici (VALDEZ et al., 2020)	171
8.4.1	Arquivo de entrada de dados	174
8.4.2	Arquivo de saída de dados (resultados da tela)	174
8.4.3	Figuras	174

9	Documentação para o Desenvolvedor	176
9.1	Dependências para compilar o software	176
9.2	Como gerar a documentação usando <i>doxygen</i>	176
A	Apêndice A: Formatação dos Arquivos de Entrada	183
A.1	Modelo de Tabela	183
A.2	Modelos Empíricos (Corey, LET e Chierici)	183
B	Apêndice B: Guia de Operação do Software	185
B.1	Configuração de Entrada de Dados	185
B.2	Parâmetros Fluidodinâmicos	185
B.3	Execução e Visualização	186
B.4	Geração de Relatórios e Temas	186

Lista de Figuras

1.1	Etapas para o desenvolvimento do software - <i>projeto de engenharia</i>	18
2.1	Diagrama de caso de uso – Caso de uso geral	27
2.2	Diagrama de caso de uso específico – Gerenciamento de Propriedades e Interatividade	29
2.3	Diagrama de caso de uso específico – Motor de Simulação de Deslocamento	30
3.1	Ilustração esquemática do efeito da razão de mobilidade na eficiência de deslocamento, no perfil de saturação e no fluxo fracionário. Fonte: Adaptado de (LAKE et al., 2014)	33
3.2	Volume Elementar Representativo (REV) para o balanço de massa unidimensional na fase aquosa	42
3.3	Diagrama de Pacotes - Arquitetura modular do simulador evidenciando o padrão de camadas e o desacoplamento pelo Controlador	52
4.1	Diagrama de Classes do Simulador Educacional	54
4.2	Diagrama de sequência geral	57
4.3	Diagrama de Sequência: Execução de um passo de tempo	58
4.4	Diagrama de Sequência: Cálculo da Tangente de Welge	59
4.5	Diagrama de Comunicação: Fluxo hierárquico de execução	60
4.6	Diagrama de Máquina de Estados da classe <code>CSimulador</code>	61
4.7	Diagrama de Atividades: Algoritmo de avanço temporal com correção de Welge	62
5.1	Diagrama de componentes	73
5.2	Diagrama de implantação	74
6.1	Versão 1.0, imagem do programa rodando	78
6.2	Versão 2.1, imagem do programa rodando	79
6.3	Versão 2.3, imagem do programa rodando	81
6.4	Ícone para tema claro	81
6.5	Ícone para tema escuro	81
8.1	Exercício 5.1	165
8.2	Extração dos diagnósticos, teste 1	165
8.3	Tela do programa mostrando	166
8.4	Exercício 5.2	168
8.5	Cenário horizontal	168
8.6	Cenário com ângulo de 30°	168

8.7	Cenário com ângulo de -30°	170
8.8	Tela do programa exibindo o perfil de saturação acelerado para injeção descendente em declive ($\alpha = -30^\circ$)	172
8.9	Tabela de coeficientes extraída de (VALDEZ et al., 2020) utilizada para os modelos LET e Chierici	172
8.10	Diagnóstico numérico do instante de ruptura para o modelo racional LET .	173
8.11	Painel principal do simulador processando o escoamento governado pelas curvas de permeabilidade do modelo LET	173
8.12	Resultados analíticos do choque de <i>Buckley-Leverett</i> empregando o modelo de decaimento exponencial de Chierici	175
8.13	Painel de resultados da interface gráfica demonstrando a estabilidade computacional na interpolação do modelo de Chierici	175
9.1	Página Class Hierarchy da documentação técnica do sistema gerada via Doxygen	177
9.2	Detalhe da documentação de classe CSimulador, funções membro públicas	178
9.3	Detalhe da documentação de classe CSimulador, note que doxygen gera diagramas UML	178
9.4	Detalhe da documentação de classe ICurvasPermeabilidade, note a hierarquia de classes	179

Lista de Tabelas

2.1	Nome do sistema/produto	20
2.2	Requisitos funcionais	22
2.3	Requisitos não funcionais	24
2.4	Caso de uso geral	25
3.1	Sistema de unidades prático adotado na modelagem matemática	34
5.1	Tabela classificação sistema	76

Listagens

7.1	Arquivo de Cabeçalho (CCelula.h)	82
7.2	Arquivo de Implementação CCellula.cpp	84
7.3	Arquivo de Cabeçalho CMalha.h	85
7.4	Arquivo de Implementação CMalha.cpp	87
7.5	Arquivo de Cabeçalho ICurvasPermeabilidade.h	88
7.6	Arquivo de Cabeçalho CCurvasPermeabilidadeCorey.h	90
7.7	Arquivo de Implementação CCurvasPermeabilidadeCorey.cpp	93
7.8	Arquivo de Cabeçalho CCurvasPermeabilidadeTabelada.h	94
7.9	Arquivo de Implementação CCurvasPermeabilidadeTabelada.cpp	97
7.10	Arquivo de Cabeçalho (CCurvasPermeabilidadeLET.h)	99
7.11	Arquivo de Implementação CCurvasPermeabilidadeLET.cpp	102
7.12	Arquivo de Cabeçalho CCurvasPermeabilidadeChierici.h	104
7.13	Arquivo de Implementação CCurvasPermeabilidadeChierici.cpp	107
7.14	Arquivo de Cabeçalho CCalculadoraFluxoFracionario.h	109
7.15	Arquivo de Implementação CCalculadoraFluxoFracionario.cpp	112
7.16	Arquivo de Cabeçalho CSolver.h	116
7.17	Arquivo de Implementação CSolver.cpp	118
7.18	Arquivo de Cabeçalho CWelge.h	121
7.19	Arquivo de Implementação CWelge.cpp	124
7.20	Arquivo de Cabeçalho CSimulador.h	126
7.21	Arquivo de Implementação CSimulador.cpp	129
7.22	Arquivo de Cabeçalho CRelatorio.h	132
7.23	Arquivo de Implementação CRelatorio.cpp	136
7.24	Arquivo de Cabeçalho MainWindow.h	141
7.25	Arquivo de Implementação MainWindow.cpp	145
7.26	Documentação da Main.cpp	161
8.1	Arquivo de texto, teste 1	164
8.2	Arquivo de texto, teste 2.	167
8.3	Arquivo de texto, teste 3	171
8.4	Arquivo de texto, teste 4	174
A.1	Arquivo de texto, teste 1	183

Nomenclatura

AOO Análise Orientada a Objetos

API Interface de Programação de Aplicações

ASCII \em{}American Standard Code for Information Interchange

BSW Razão de água no fluxo

C++ \em{}\nospellcheck onLinguagem de programa\c{c}\ {a}o de alto n\'\{i}vel

CLI \em{}Command Line Interface

CMG \em{}\nospellcheck onComputer Modelling Group

CPU Unidade central de processamento

DLLs \em{}Dynamic Link Library

EDOs Equações Diferenciais Ordinárias

GDI \em{}Graphics Device Interface

GUI \em{}Graphical User Interface

IDE \em{}Integrated Development Environment

MOC Método das Características

MVC \em{}Model-View-Controller

QSS Qt \em{}Style Sheets

Qt \em{}Qt Framework

RAM \em{}Random-access memory

SAN \em{}Storage Area Network

SI Sistema Internacional de Unidades

SRP \em{}Single Responsibility Principle

STL \em{}Standard Template Library\em{}

UML \em{}Unified Modeling Language

UX \em{}User Experience

UX \em{}User Experience

$\bar{S}_w$ Saturação Média (adimensional)

E_D Eficiência de Deslocamento (adimensional)

N_{g0} Número de gravidade no ponto final (adimensional)

α Ângulo de inclinação do reservatório (radianos)

$\Delta\rho$ Variação das massas específicas ($\rmfamily{} \series medium \shape up \size normal \emph{} \nospellcheck off \bar no \strikeout off \xout off \uuline off \uwave off \noun off \color none \texttt{kg/m} \textasciicircum{}^3 \normalfont{} \mdseries{} \upshape{} \size default \em{} \nospellcheck default \nomeqref 2.0$)

μ Viscosidade Dinâmica ($\text{Pa} \cdot \text{s}$)

μ_o Viscosidade do óleo (Pa.s)

μ_w Viscosidade da água (Pa.s)

ϕ Porosidade (porcentagem)

ρ Massa Específica ($\text{kg/m} \textasciicircum{}^3$)

σ Tensão Interfacial (N/m)

θ Ângulo de contato de molhabilidade rocha-fluido (radianos)

A Área da Seção Transversal (m^2)

f Fluxo Fracionário (adimensional)

g Gravidade (m^2/s)

K Permeabilidade absoluta (m^2)

M Razão de mobilidade (adimensional)

n_o Expoente de Corey para a fase oleica (adimensional)

N_p Recuperação de óleo acumulada (adimensional)

N_{RL} Número de Rapoport-Leas (adimensional)

n_w Expoente de Corey para a fase aquosa (adimensional)

p_c Pressão capilar ($\rmfamily{} \series medium \shape up \size normal \emph{} \nospellcheck off \bar no \strikeout off \xout off \uuline off \uwave off \noun off \color none \text{Pa}(\text{N/m} \textasciicircum{}^2)$)

Q	Vazão Volumétrica ($\frac{m^3}{s}$)
S	Saturação (adimensional)
S_w	Saturação de água (adimensional, fração)
t	Tempo (s)
t_{bt}	Tempo de Ruptura (Breakthrough) (adimensional)
t_D	Tempo Adimensional (adimensional)
u	Velocidade de Darcy (Superficial) ($\frac{m}{s}$)
v_{σ}	Velocidade da frente de choque (adimensional)
v_L	Velocidade da saturação imediatamente atrás do choque (adimensional)
v_R	Velocidade da saturação imediatamente à frente do choque (adimensional)
x	Comprimento, distância (metros)
k_{rel}	Permeabilidade Relativa (adimensional)
f_w	Fluxo fracionário para água (adimensional)
S_{wf}	Saturação de água da frente de choque (adimensional, fração)

Software Educacional Interativo para Simulação do Método das Características: Solução Analítica de Buckley-Leverett

Resumo

O desenvolvimento de simuladores computacionais robustos é fundamental para a otimização da recuperação de petróleo em reservatórios. O presente trabalho apresenta o desenvolvimento de um software educacional interativo, baseado no Método das Características (MOC), para a resolução analítica da equação de Buckley-Leverett aplicada ao escoamento imiscível linear em meios porosos. A ferramenta foi implementada em linguagem C++ com o framework Qt, integrando os modelos de permeabilidade relativa de Corey, LET, Chierici e o modelo experimental tabelado. O simulador permite a configuração de parâmetros geomorfológicos, viscosidades, densidades e efeitos gravitacionais, gerando diagnósticos automatizados sobre o instante de ruptura (breakthrough) e a eficiência de varrido. A validação do motor matemático foi realizada através da comparação com cenários clássicos da literatura, demonstrando alta precisão na ancoragem da tangente de Welge e na aplicação da condição de entropia para frentes de choque. Os resultados indicam que a ferramenta é eficaz tanto para o ensino de engenharia de reservatórios quanto como suporte à tomada de decisão em projetos de injeção de água. O software é de código aberto, com repositório disponível para a comunidade técnica e acadêmica.

Palavras chave: Buckley-Leverett. Permeabilidade Relativa. Fluxo Fracionário. Engenharia de Reservatórios. Software Educacional. Simulador Computacional.

Interactive Educational Software for Method of Characteristics Simulation: Buckley-Leverett Analytical Solution

Abstract

The development of robust computational simulators is fundamental for optimizing oil recovery in reservoirs. This work presents the development of interactive educational software, based on the Method of Characteristics (MOC), for the analytical resolution of the Buckley-Leverett equation applied to linear immiscible flow in porous media. The tool was implemented using the C++ language and the Qt framework, integrating several relative permeability models, including Corey, LET, Chierici, and the experimental tabulated model. The simulator allows for the configuration of geomorphological parameters, viscosities, densities, and gravitational effects, generating automated diagnostics on breakthrough time and sweep efficiency. Mathematical engine validation was performed through comparison with classical scenarios from technical literature, demonstrating high precision in anchoring the Welge tangent and correctly applying the entropy condition for shock fronts. The results indicate that the tool is effective both for reservoir engineering education and as a decision-support aid in waterflooding projects. The software is open-source, with its repository available to the technical and academic community.

Keywords: Buckley-Leverett. Relative Permeability. Fractional Flow. Reservoir Engineering. Educational Software. Computational Simulator.

Capítulo 1

Introdução

O presente trabalho de conclusão de curso propõe o desenvolvimento do Software Educacional com Interface Interativa para Implementação Computacional do Método das Características – Solução Analítica de Buckley-Leverett. A concepção desta ferramenta tecnológica alinha-se às necessidades contemporâneas do ensino da Engenharia de Reservatórios, fornecendo um suporte computacional robusto para o estudo de métodos de recuperação secundária.

Conforme destacam (ROSA; CARVALHO; XAVIER, 2006), a injeção de água (*waterflooding*) permanece como a técnica mais difundida na indústria petrolífera mundial para a manutenção de pressão e otimização do varrido de óleo. Nesse cenário, a aplicação desenvolvida em C++/Qt utiliza o paradigma da Orientação a Objetos para simular e visualizar, de forma dinâmica, os fenômenos físicos que regem a competição entre fluidos no meio poroso, preenchendo uma lacuna importante entre a teoria abstrata e a prática de engenharia.

A base científica para a modelagem desse fenômeno fundamenta-se na teoria clássica de deslocamento imiscível linear estabelecida por (BUCKLEY; LEVERETT, 1941)¹. Embora fundamental, a aplicação analítica dessa teoria exige a resolução de equações diferenciais parciais através do Método das Características (MOC) e a aplicação de condições de entropia para a determinação da frente de choque, processos que apresentam elevada complexidade matemática para cálculos manuais. O software proposto supera essa barreira ao automatizar o cálculo das curvas de fluxo fracionário e dos perfis de saturação, permitindo que o usuário manipule parâmetros críticos — como a razão de mobilidade e a viscosidade dos fluidos — e visualize instantaneamente o impacto dessas variáveis na eficiência de recuperação, consolidando o aprendizado através da interatividade.

¹O acesso ao conceito ocorreu através de autores modernos como Lake, que referenciam Buckley, S.E. and Leverett, M.C.

1.1 Escopo do problema

A competência fundamental que distingue a formação do engenheiro é a capacidade de conceber e desenvolver soluções tecnológicas para problemas complexos, indo além da operação de sistemas preexistentes. Segundo (BAZZO; PEREIRA, 2013), a essência da engenharia reside na síntese de conhecimentos científicos e empíricos para a criação de projetos que satisfaçam necessidades humanas e econômicas. Em consonância com as Diretrizes Curriculares Nacionais do Curso de Graduação em Engenharia (BRASIL, 2002), o perfil do egresso deve contemplar a aptidão para projetar e modelar sistemas, habilidade esta que diferencia a atuação plena do engenheiro das atribuições técnicas operacionais. A [Proposta Curricular para os Cursos de Bacharelado em Engenharia de Petróleo](#) estabelece explicitamente o desenvolvimento de conhecimentos e habilidades relacionados ao desenvolvimento de simuladores aplicados à Engenharia de Petróleo.

Nesse contexto, este trabalho delimita-se ao desenvolvimento de um projeto de engenharia de software aplicado à Engenharia de Petróleo, especificamente na subárea de Engenharia de Reservatórios. O problema central abordado é a complexidade na análise preliminar de projetos de injeção de água (*waterflooding*), o método de recuperação secundária mais utilizado mundialmente (ROSA; CARVALHO; XAVIER, 2006).

Embora a teoria do deslocamento imiscível, fundamentada na clássica solução de (BUCKLEY; LEVERETT, 1941), seja a base para a compreensão desses mecanismos, sua aplicação prática manual é laboriosa e propensa a erros de cálculo, especialmente quando se consideram variações nos modelos de permeabilidade relativa ou a necessidade de determinar a frente de choque através da construção da tangente de Welge.

Por outro lado, as ferramentas comerciais de simulação numérica disponíveis na indústria (como Eclipse ou CMG - *Computer Modelling Group*) são projetadas para modelos complexos e tridimensionais (ERTEKIN; ABOU-KASSEM; KING, 2001). A utilização desses softwares robustos para análises unidimensionais simples ou para fins didáticos apresenta uma curva de aprendizado íngreme e um custo computacional desproporcional à simplicidade do problema físico 1D. Existe, portanto, uma lacuna tecnológica entre o cálculo analítico manual e a simulação numérica de grande porte.

O escopo deste projeto consiste em preencher essa lacuna através do desenvolvimento integral de uma ferramenta computacional dedicada. O trabalho abrange todo o ciclo de vida do desenvolvimento de software preconizado por (PRESSMAN; MAXIM, 2016), incluindo a especificação de requisitos, a modelagem matemática e algorítmica do Método das Características (MOC), a implementação do código-fonte utilizando o Qt *framework*/C++, a validação dos resultados através da comparação com a teoria analítica, a documentação do software desenvolvido e seu armazenamento e disponibilização no [github/ldsc](#).

A escolha das tecnologias de implementação — C++ e Qt *framework* — fundamenta-se na necessidade de alta performance computacional e na portabilidade da interface gráfica. Diferente de linguagens interpretadas, o C++ oferece o controle de memória e a velocidade de execução necessárias para cálculos matemáticos complexos, enquanto o Qt *framework* assegura a integridade da Interface Gráfica do Usuário (GUI - *Graphical User Interface*) em diferentes sistemas operacionais. Garantindo assim alto desempenho, interface amigável e uso em múltiplas plataformas.

1.2 Objetivos

Os objetivos definidos para este projeto de engenharia são apresentados a seguir.

- Objetivo geral:
 - Desenvolver uma ferramenta computacional interativa para a análise do fluxo fracionário e simulação do avanço da frente de saturação em reservatórios de petróleo, visando facilitar a interpretação dos fenômenos físicos envolvidos no deslocamento imiscível água-óleo.
- Objetivos específicos:
 - Modelar física e matematicamente o problema do deslocamento linear de fluidos imiscíveis, fundamentando-se na equação de fluxo fracionário e no modelo de permeabilidade relativa de Corey.
 - Realizar a modelagem estática do sistema, elaborando diagramas de casos de uso, pacotes e classes, com a aplicação de padrões de projeto (*Design Patterns*), como o *Strategy*, para assegurar a flexibilidade e escalabilidade do código.
 - Desenvolver a modelagem dinâmica do software, estruturando os algoritmos numéricos para o cálculo da curva de fluxo fracionário (f_w) e de sua derivada (df_w/dS_w) em relação à saturação.
 - Implementar o método da construção da tangente de Welge para determinar a saturação da frente de choque (S_{wf}) e a saturação média do reservatório.
 - Construir a Interface Gráfica do Usuário (GUI) utilizando o Qt *framework*/C++, proporcionando uma experiência de uso intuitiva para a entrada de parâmetros (viscosidades, expoentes de saturação) e visualização de resultados.
 - Simular cenários de injeção e validar a robustez da ferramenta através da análise de sensibilidade de parâmetros e comparação com a teoria clássica de Buckley-Leverett.
 - Desenvolver o manual do usuário (instalação e uso).

1.3 Metodologia utilizada

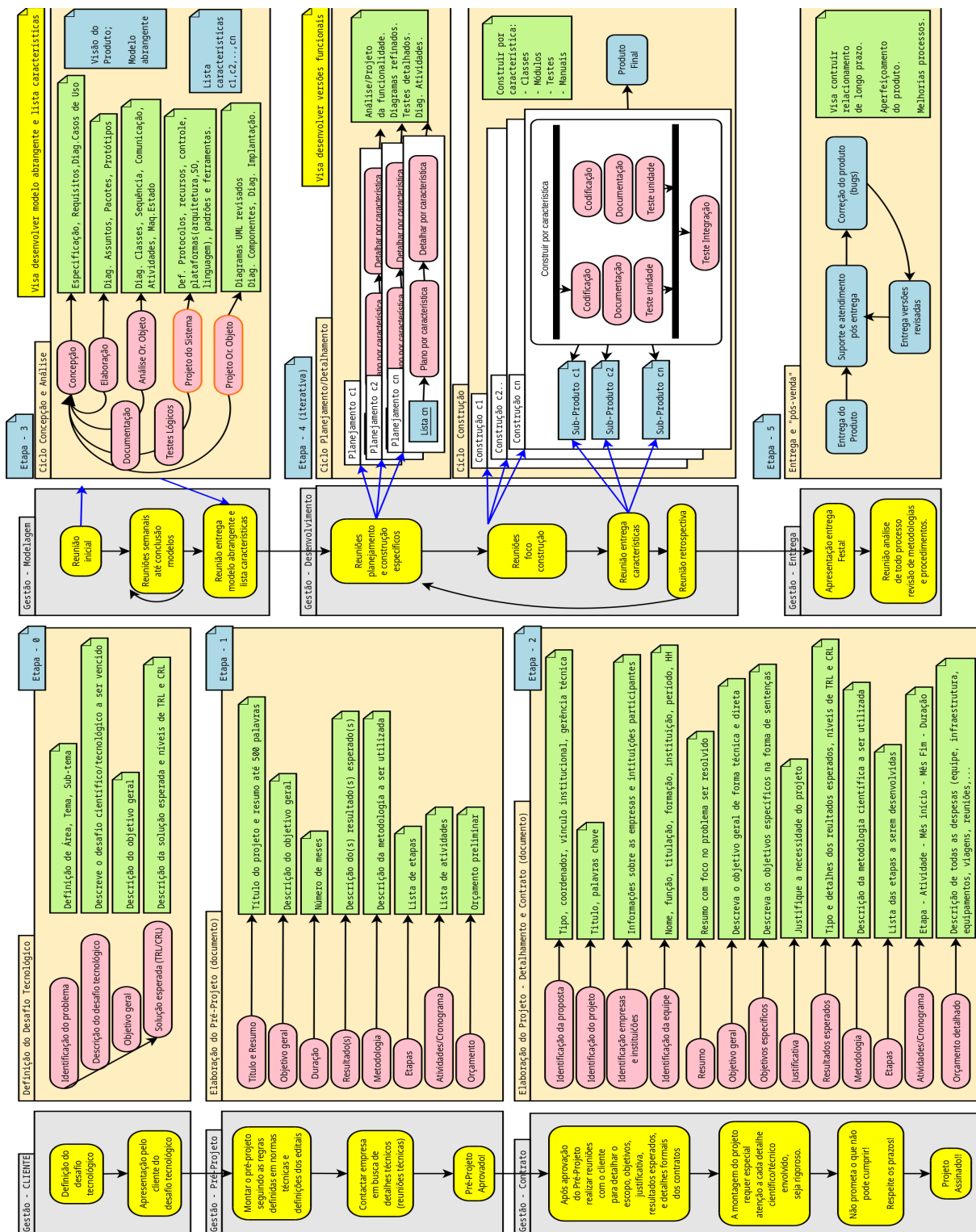
O desenvolvimento deste projeto fundamenta-se nos princípios da Engenharia de Software estabelecidos por (PRESSMAN; MAXIM, 2016), que preconizam uma abordagem sistemática, disciplinada e quantificável para a construção de sistemas computacionais. A adoção de um processo de software estruturado é essencial para garantir a qualidade, a manutenibilidade e a conformidade do produto final com os requisitos de engenharia de reservatórios.

Para a operacionalização desses princípios, adotou-se a metodologia de projeto aplicada pelo Prof. André Bueno nas disciplinas de Introdução ao Projeto de Engenharia (LEP01348) e Projeto de Software Aplicado à Engenharia (LEP01449) da UENF Universidade Estadual do Norte Fluminense Darcy Ribeiro. Esta abordagem adapta o modelo clássico de ciclo de vida em etapas sequenciais e iterativas, adicionando conceitos de metodologias ágeis, conforme ilustrado na Figura 1.1. Isto nos permitiu um refinamento progressivo da solução.

O fluxo de trabalho está estruturado nas seguintes fases, que correspondem à organização dos capítulos deste documento:

- Ciclo de Concepção e Análise: Foca no entendimento do problema físico (fluxo fracionário), levantamento de requisitos e modelagem do domínio. Esta etapa utiliza a abstração da Orientação a Objetos para mapear as equações de Buckley-Leverett em classes computacionais, conforme as práticas de modelagem descritas por (BUENO, 2003);
- Ciclo de Planejamento e Detalhamento: Compreende a arquitetura do software, a definição das estruturas de dados e o design da interface gráfica (GUI), detalhados em capítulo específico;
- Ciclo de Construção (Implementação): Consiste na codificação propriamente dita, utilizando a linguagem C++ e o Qt *framework*, seguida das etapas de testes e validação dos resultados numéricos.

Dessa forma, a metodologia empregada assegura que o Software Educacional com Interface Interativa seja desenvolvido como um produto de engenharia documentado e robusto.

Figura 1.1: Etapas para o desenvolvimento do software - *projeto de engenharia*

Capítulo 2

Concepção

Apresenta-se neste capítulo a concepção e a especificação do sistema modelado e desenvolvido. Esta fase inicial, classificada por ([PRESSMAN; MAXIM, 2016](#)) como o alicerce da engenharia de requisitos, é o momento determinante onde o escopo do projeto é delimitado e as necessidades abstratas do problema são traduzidas em definições técnicas concretas. A concepção não se resume apenas a um planejamento preliminar, mas envolve um esforço analítico para identificar os atores envolvidos, compreender as restrições do domínio e estabelecer as funcionalidades fundamentais que guiarão todo o ciclo de construção do software.

O resultado desta etapa consolida-se em um conjunto formal de especificações que definem a identidade do Software Educacional com Interface Interativa. Neste contexto, detalha-se como a ferramenta deve operar para cumprir sua missão pedagógica de auxiliar na análise de Engenharia de Reservatórios, estabelecendo os requisitos necessários para a modelagem computacional da solução analítica de Buckley-Leverett. O objetivo é assegurar que o produto final ofereça não apenas precisão matemática, mas também a interatividade necessária para a visualização dinâmica do avanço da frente de saturação.

2.1 Nome do sistema/produto

A definição formal da identidade do software é essencial para comunicar seu propósito e delimitar seu escopo de atuação dentro do contexto educacional e tecnológico. A Tabela 2.1 apresenta a nomenclatura oficial adotada para o sistema, seus componentes estruturantes e a missão que orienta seu desenvolvimento.

Tabela 2.1: Nome do sistema/produto

Nome	Software Educacional com Interface Interativa para Implementação Computacional do Método das Características – Solução Analítica de Buckley-Leverett
Componentes principais	1. Interface Gráfica do Usuário (GUI): Desenvolvida no <i>framework</i> Qt, responsável pela interação homem-máquina e entrada de dados. 2. Núcleo de Processamento Analítico: Módulo em C++ responsável pela implementação dos algoritmos do Método das Características (MOC) e da construção da tangente de Welge. 3. Módulo de Visualização de Dados: Sistema de renderização gráfica em tempo real (baseado na biblioteca <code>QCustomPlot</code>) para exibição das curvas de fluxo e perfis de saturação.
Missão	Proporcionar uma plataforma computacional interativa para a modelagem e visualização do deslocamento imiscível linear, facilitando a assimilação dos fundamentos teóricos da Engenharia de Reservatórios e permitindo a análise rápida de sensibilidade em projetos de injeção de água.

2.2 Especificação

A especificação do sistema delimita as fronteiras da solução tecnológica, descrevendo o comportamento esperado do software e suas interações com o usuário. Segundo (SOMMERVILLE, 2011), esta etapa é fundamental para documentar os serviços que o sistema deve fornecer e as restrições sob as quais deve operar. Neste trabalho, a especificação define uma ferramenta computacional analítica com Interface Gráfica (GUI), projetada para abstrair a complexidade matemática inerente ao modelo de deslocamento imiscível linear.

O sistema proposto destaca-se pela flexibilidade na entrada de dados. Diferentemente de ferramentas rígidas, a solução permite que o usuário opte entre dois modos de operação: a importação direta de arquivos externos (tabelas em `.txt`) para usuários que já possuem dados consolidados, ou a inserção manual de parâmetros através de formulários intuitivos na interface gráfica.

No que tange ao núcleo de processamento, o software adota uma arquitetura extensível capaz de suportar múltiplos modelos de permeabilidade relativa. A implementação apresenta seleção dinâmica de modelos de permeabilidade relativa — com ênfase primária no modelo de (COREY, 1954), amplamente utilizado devido à sua simplicidade e base física, mas também incorpora correlações paramétricas avançadas para maior precisão em cenários complexos, como o modelo LETtextbackslashstrikeout default\color inheritLombey, Escovedo e Trindade (LOMBEZ; ESCOVEDO; TRINDADE, 2008) e o modelo de Chierici (CHIERICI, 1984).

Independentemente do modelo escolhido, o cálculo da curva de fluxo fracionário (f_w) e a determinação do Perfil de saturação através do Método de Características (MOC) são executados automaticamente. Por fim, diferentemente de abordagens que dependem de softwares externos para a plotagem de dados (como Excel ou Gnuplot), a especificação deste projeto exige que a visualização dos resultados seja imediata e integrada. O sistema deve renderizar gráficos interativos de $f_w \times S_w$ e perfis de saturação $S_w \times x$ na própria janela da aplicação, proporcionando um ambiente de simulação autocontido (*standalone*) que favorece a análise comparativa entre diferentes correlações empíricas.

2.3 Requisitos

A etapa de engenharia de requisitos é fundamental para traduzir a necessidade física de simular o deslocamento de fluidos em especificações técnicas de software. Segundo (PRESSMAN; MAXIM, 2016), o entendimento claro dos requisitos estabelece a base para todas as etapas subsequentes de design e implementação.

Para o desenvolvimento desta ferramenta, os requisitos foram divididos em duas categorias: funcionais, que descrevem as capacidades matemáticas e interativas do sistema (cálculo do MOC, plotagem de curvas); e não funcionais, que definem as restrições de desempenho, usabilidade e arquitetura (uso de C++, tempo de resposta).

A correta definição destes requisitos assegura que o software final não apenas resolva as equações de Buckley-Leverett, mas o faça de maneira didática e responsiva para o usuário.

2.3.1 Requisitos funcionais

Os requisitos funcionais (RFtextbackslashstrikeout default\ xout default\ uuline default\ uwave default\ upshape{}\color inheritRequisitos Funcionais) detalham as operações que o sistema deve ser capaz de realizar para cumprir sua missão educacional. Conforme definido por (SOMMERVILLE, 2011), tais requisitos descrevem o que o sistema deve fazer, como deve reagir a entradas específicas e como deve se comportar em situações particulares. A Tabela 2.2 lista as funcionalidades mapeadas para este projeto, abrangendo desde a entrada flexível de dados petrofísicos até a lógica complexa de verificação de estabilidade da frente de choque e a renderização gráfica dos resultados.

Tabela 2.2: Requisitos funcionais

RF-01	O sistema deve permitir a entrada manual de dados petrofísicos (porosidade, saturações irreduzíveis) e propriedades dos fluidos (viscosidades) através de formulários na interface gráfica.
RF-02	O sistema deve permitir o carregamento de dados de permeabilidade relativa de duas formas: (a) Importação de arquivos (.txt) contendo tabelas ou parâmetros; (b) Inserção manual de parâmetros na interface.
RF-03	O sistema deve implementar algoritmos para calcular permeabilidades relativas baseados nos modelos: Corey, LET, Chierici e Interpolação de Tabelas.
RF-04	O sistema deve validar os dados de entrada, impedindo cálculos com valores fisicamente inconsistentes e alertando o usuário.

RF-05	O sistema deve recalcular instantaneamente a Razão de Mobilidade (M) e atualizar todos os gráficos sempre que o usuário alterar os valores de viscosidade (μ_o, μ_w) na interface.
RF-06	O sistema deve permitir a alteração dinâmica dos parâmetros dos modelos analíticos (Corey, LET, Chierici) na interface, atualizando os gráficos em tempo real para análise de sensibilidade (molhabilidade). Nota: Para o modelo de Tabela, os dados permanecem estáticos.
RF-07	O sistema deve calcular a curva de Fluxo Fracionário Generalizada (f_w), incorporando (opcionalmente) os termos de gradiente de segregação gravitacional ($\Delta\rho g \sin \alpha$).
RF-08	O sistema deve calcular as velocidades de avanço e o perfil preliminar de saturação utilizando a solução geral do Método das Características (MOC).
RF-09	O sistema deve analisar a estabilidade física da solução MOC. Caso identifique multivaloração, deve aplicar o método da Tangente de Welge para definir a frente de choque (S_{wf}); caso contrário, deve manter a solução direta.
RF-10	O sistema deve simular e renderizar graficamente o cenário de injeção 1D entre um poço injetor e um poço produtor, aplicando o perfil de saturação resultante.
RF-11	O sistema deve gerar o gráfico da Curva de Fluxo Fracionário ($f_w \times S_w$) indicando visualmente o ponto de tangência quando este for calculado.

2.3.2 Requisitos não funcionais

Enquanto os requisitos funcionais definem 'o que' o sistema faz, os requisitos não funcionais (RNF) estabelecem 'como' ele deve se comportar, impondo restrições aos serviços oferecidos pelo sistema (SOMMERVILLE, 2011). A Tabela 2.3.2 apresenta as restrições arquiteturais adotadas. A escolha pela linguagem C++ e pelo

framework Qt (RNF-01 e RNF-02) justifica-se pela necessidade de alto desempenho computacional para garantir a interatividade em tempo real proposta no RF-06, enquanto o uso do padrão de projeto *Strategy* (RNF-03) visa garantir a extensibilidade do código para futuros modelos de permeabilidade, seguindo as boas práticas de design de software recomendadas por (PRESSMAN; MAXIM, 2016).

Tabela 2.3: Requisitos não funcionais

RNF-01	O software deve ser desenvolvido utilizando a linguagem C++ (padrão C++17 ou superior) e o paradigma de Orientação a Objetos para garantir desempenho nos cálculos iterativos.
RNF-02	A interface gráfica deve ser construída utilizando o <i>framework</i> Qt (versão 6. ou superior), assegurando a responsividade dos controles (<i>sliders</i> e formulários) e integração nativa com o sistema operacional.
RNF-03	O sistema deve implementar o padrão de projeto <i>Strategy</i> para o cálculo de permeabilidade relativa. Isso é mandatório para permitir a alternância limpa entre os algoritmos de Corey, LET, Chierici e Tabelas sem modificar o núcleo do simulador, também assegura que novos modelos matemáticos possam ser adicionados sem modificação do código existente (princípio Aberto-Fechado).
RNF-04	A plotagem dos gráficos deve utilizar a biblioteca <code>QCustomPlot</code> , devido à sua capacidade de renderização de alto desempenho, necessária para a atualização instantânea das curvas durante a análise de sensibilidade.
RNF-05	O sistema deve garantir um tempo de resposta inferior a $200ms$ para o recálculo e re-renderização dos gráficos ao alterar parâmetros via interface, proporcionando uma sensação de continuidade (tempo real).
RNF-06	O sistema deve possuir tratamento de exceções para a leitura de arquivos externos (<code>.txt</code>), garantindo que formatações incorretas ou dados corrompidos não encerrem a aplicação abruptamente, mas exibam mensagens de erro claras.

RNF-07	O software deve ser compilável e executável tanto em ambientes Windows quanto Linux, sem necessidade de refatoração do código fonte.
---------------	--

2.4 Casos de uso

A modelagem de casos de uso é utilizada para descrever a interação entre o usuário (ator) e o sistema, abstraindo a complexidade do código fonte para focar no fluxo de eventos e nos objetivos do usuário ([PRESSMAN; MAXIM, 2016](#)).

A Tabela 2.4 detalha o cenário principal de utilização do software: 'Simulação de Injeção 1D e Análise de Sensibilidade'. Este caso de uso captura o fluxo completo de trabalho de um engenheiro de reservatórios, desde a configuração das propriedades da rocha até a análise do tempo de ruptura (*breakthrough*).

Tabela 2.4: Caso de uso geral

Nome do caso de uso:	Simulação de Injeção 1D e Análise de Sensibilidade
Ator Principal:	Usuário (Estudante de Engenharia ou Engenheiro de Reservatórios)
Pré-condições:	O software deve estar inicializado e pronto para receber dados.

Etapas:	1. Entrada de Dados de Fluidos e Rocha: O usuário insere manualmente na interface as propriedades escalares dos fluidos (viscosidades μ_o, μ_w, densidades) e do reservatório (porosidade ϕ, permeabilidade absoluta K, saturações irreduzíveis e área/distância). 2. Definição da Permeabilidade Relativa: O usuário define a origem das curvas de permeabilidade: (a) Opção A (Modelos Analíticos): Seleciona Corey, LET ou Chierici e ajusta os parâmetros (ex: expoentes n_o, n_w) diretamente na tela. (b) Opção B (Dados Externos): Seleciona a opção de importação e carrega um arquivo (.txt ou .xlsx) contendo a tabela de permeabilidade relativa. 3. Análise de Sensibilidade (Interatividade): O usuário altera parâmetros críticos (ex: viscosidade do óleo) na interface. O sistema recalcula instantaneamente a Razão de Mobilidade (M) e atualiza o gráfico de Fluxo Fracionário (f_w), permitindo a visualização imediata do impacto na eficiência de varrido. 4. Execução do MOC: O sistema calcula as velocidades de avanço para cada saturação utilizando a solução geral do Método das Características (MOC). 5. Verificação de Estabilidade: O sistema analisa o perfil gerado pelo MOC para detectar multivaloração (curva S). 6. Aplicação de Condição de Choque: (a) Se Instável: O sistema aplica a Tangente de Welge para determinar a saturação da frente de choque (S_{wf}) e corrige o perfil. (b) Se Estável (Fluxo Pistão/Dispersivo): O sistema mantém a solução direta do MOC. 7. Renderização do Cenário: O sistema projeta o perfil de saturação final na distância definida entre o poço injetor e o produtor, exibindo o avanço da frente em função do tempo ou volume injetado.
Cenários alternativos:	1. Inclusão de Gravidade: O usuário ativa os termos da Equação Generalizada na interface. O sistema recalcula a curva f_w considerando o ângulo de inclinação (Gravidade), alterando o critério de formação do choque. 2. Erro na Leitura de Arquivo: Caso o usuário importe um arquivo mal formatado na Etapa 2, o sistema exibe um alerta de erro e reverte para o modelo padrão (Corey), impedindo o travamento da aplicação.

2.4.1 Diagrama de caso de uso geral

O Diagrama de Caso de Uso Geral, apresentado na Figura 2.1, oferece uma visão macroscópica das funcionalidades do sistema. A estrutura visual foi organizada de forma hierárquica para refletir o fluxo lógico de uma simulação de engenharia, integrando as

definições de múltiplos modelos de permeabilidade como Corey (COREY, 1954), LET (LOMBEZ; ESCOVEDO; TRINDADE, 2008) e Chierici (CHIERICI, 1984) diretamente na etapa de configuração. Esta organização demonstra que o software não é apenas uma ferramenta de cálculo, mas um ambiente integrado de análise.

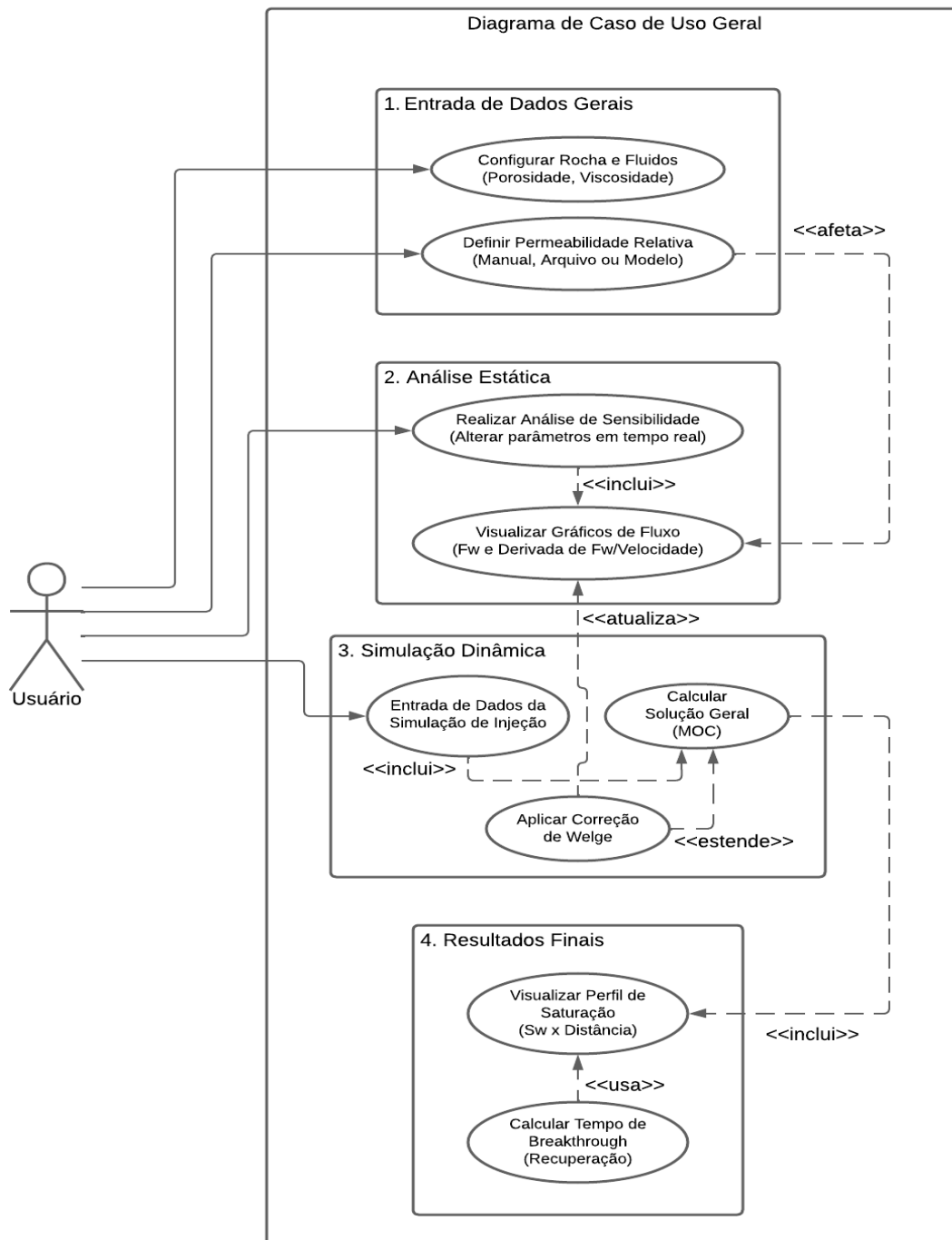


Figura 2.1: Diagrama de caso de uso – Caso de uso geral

2.4.2 Diagrama de caso de uso específico

Para detalhar a complexidade do sistema, a modelagem foi dividida em dois diagramas específicos que cobrem, respectivamente, a entrada de dados (gerenciamento de propriedades) e o processamento matemático (motor de simulação).

Diagrama Específico 1: Gerenciamento de Propriedades e Interatividade

A Figura 2.2 detalha o subsistema de 'Gerenciamento de Propriedades'. Este diagrama ilustra o cumprimento dos requisitos de flexibilidade (RF-02) e extensibilidade (RNF-03). A estrutura evidencia a aplicação do padrão de projeto *Strategy*, onde o usuário seleciona a fonte dos dados de permeabilidade relativa: seja através de modelos analíticos parametrizáveis (como Corey, LET ou Chierici) ou pela importação direta de dados experimentais via arquivos externos.

Adicionalmente, o diagrama destaca o ciclo de 'Análise de Sensibilidade' (RF-06). A modelagem demonstra que a alteração de parâmetros na interface permite a exploração rápida de cenários, onde o motor de cálculo é acionado sob demanda pelo usuário. Esta arquitetura de execução sob demanda garante que o processamento matemático intensivo (cálculo de f_w e MOC) ocorra apenas quando os parâmetros estiverem validados, otimizando o uso de recursos computacionais e evitando instabilidades gráficas durante a edição das variáveis.

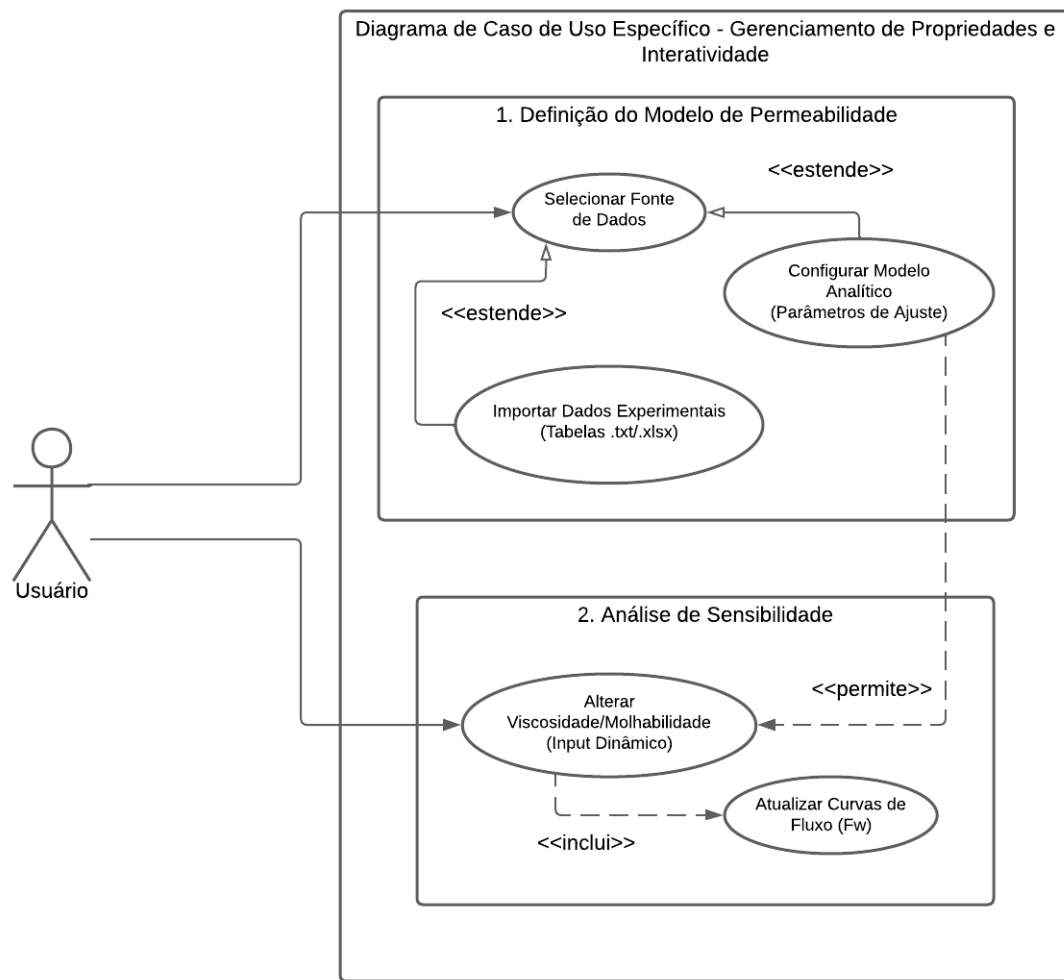


Figura 2.2: Diagrama de caso de uso específico – Gerenciamento de Propriedades e Interatividade

Diagrama Específico 2: Motor de Simulação de Deslocamento

Complementarmente, a Figura 2.3 apresenta o 'Motor de Simulação', focando no rigor físico do algoritmo de avanço frontal. O fluxo inicia com o cálculo da solução analítica direta via Método das Características (MOC).

O ponto crítico deste sistema é o caso de uso 'Verificar Condição de Entropia'. O software não aplica a construção de Welge indiscriminadamente; ela é modelada como uma extensão (<<estende>>) que é ativada apenas se o perfil de saturação gerado pelo MOC apresentar instabilidade física (multivaloração). Esta abordagem assegura que o simulador trate corretamente tanto cenários de fluxo dispersivo quanto a formação de frentes de choque, culminando no cálculo preciso do tempo de ruptura (*breakthrough*) e do fator de recuperação final.

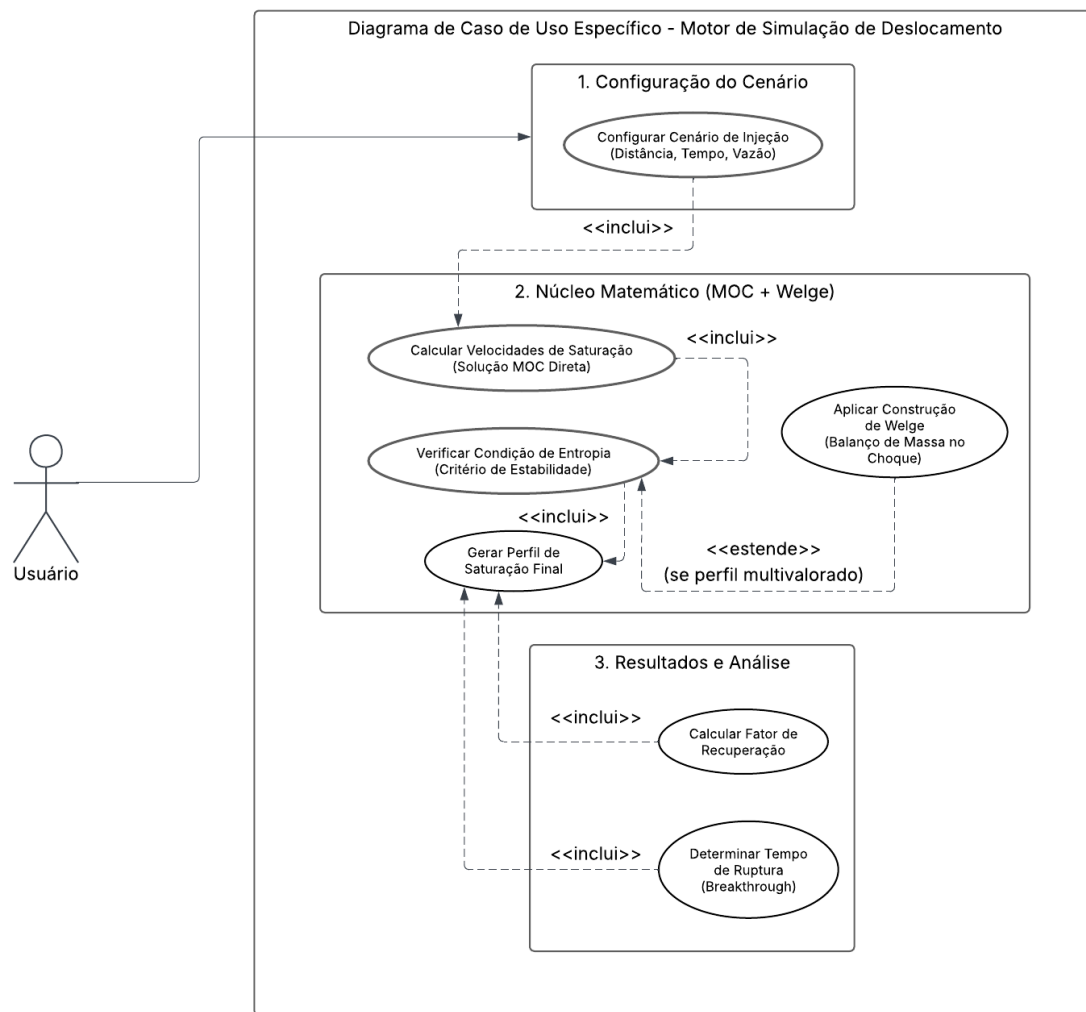


Figura 2.3: Diagrama de caso de uso específico – Motor de Simulação de Deslocamento

Capítulo 3

Elaboração

Após a etapa de concepção, onde foram definidos o escopo e os requisitos fundamentais do sistema, o projeto avança para a fase de elaboração. Segundo ([PRESSMAN; MAXIM, 2016](#)), esta fase é caracterizada pela expansão e refinamento das representações preliminares do software, transformando os requisitos abstratos em um modelo técnico mais detalhado. É o momento em que a ênfase transita do entendimento do problema para a estruturação da solução, focando na arquitetura do sistema e na consistência dos dados.

Neste contexto, o processo de elaboração busca estabelecer uma base arquitetural sólida antes que a construção do código se inicie massivamente. Conforme destaca ([SOMMERVILLE, 2011](#)), é essencial que, durante esta análise, as fronteiras do sistema sejam perfeitamente compreendidas e que os componentes lógicos sejam organizados de forma a maximizar a coesão e minimizar o acoplamento.

Desta forma, este capítulo detalha as atividades de análise de domínio e identificação de pacotes. O objetivo é ajustar os requisitos iniciais às restrições técnicas identificadas, eliminando funcionalidades inviáveis e consolidando a estrutura dos subsistemas (como o núcleo matemático e a interface gráfica) para garantir que o simulador final seja robusto e extensível.

3.1 Análise de domínio

A análise de domínio é uma atividade contínua da engenharia de software que não foca apenas em uma aplicação específica, mas sim na identificação, análise e especificação de requisitos comuns a um campo de aplicação inteiro. Segundo ([PRESSMAN; MAXIM, 2016](#)), o objetivo é encontrar padrões, conceitos e regras que se aplicam a todos os sistemas dentro desse domínio, permitindo a reutilização de conhecimento e componentes. No contexto deste trabalho, a análise de domínio delimita as fronteiras da Engenharia de Reservatórios que serão computacionalmente modeladas, garantindo que o software respeite as leis físicas e as práticas padrão da indústria petrolífera.

Definição e caracterização do domínio a ser investigado

O domínio de investigação deste trabalho situa-se na interseção entre a Engenharia de Reservatórios e a Computação Científica. O objeto de estudo é a modelagem do

escoamento bifásico imiscível (água e óleo) em meios porosos, regido pela teoria clássica do Fluxo Fracionário. Diferentemente de simuladores comerciais focados em modelos tridimensionais complexos de campo (*full-field*), o domínio deste software é delimitado à simulação analítica unidimensional (1D). O foco principal é a caracterização precisa do avanço da frente de saturação e a determinação da eficiência de recuperação secundária, servindo como uma ferramenta de apoio à decisão rápida e ao ensino de engenharia.

Descrição das áreas/disciplinas relacionadas

O desenvolvimento deste simulador exige uma abordagem multidisciplinar, integrando conceitos fundamentais de três áreas distintas:

- **Engenharia de Petróleo:** Fornece a base física do problema, incluindo a Lei de Darcy, as correlações de permeabilidade relativa (modelos de Corey, LET e Chierici) e a própria teoria de deslocamento de (BUCKLEY; LEVERETT, 1941) .
- **Matemática Aplicada:** Envolve a resolução de Equações Diferenciais Parciais (EDP) do tipo hiperbólico através do Método das Características (MOC) e a aplicação de condições de contorno e entropia, como a construção da tangente de (WELGE, 1952).
- **Engenharia de Software e Programação:** Aplica-se na estruturação da arquitetura do sistema utilizando o paradigma de Orientação a Objetos (BUENO, 2003), na implementação de algoritmos de alta performance em C++ e na construção de interfaces gráficas interativas com o framework Qt.

Descrição de questões associadas a espaço/tempo (espaço físico, local, instalação)

- **Espaço (Físico/Simulado):** O sistema opera sobre um domínio espacial que representa um meio poroso. Neste projeto, o espaço é simplificado para uma dimensão linear (1D), correspondendo à distância entre um poço injetor e um produtor.
- **Tempo (Dinâmica):** O domínio envolve processos dinâmicos que evoluem com o tempo. O software deve lidar com a variável temporal não apenas em segundos (tempo de processamento), mas principalmente em "Volumes de Poros Injetados" (PVI) ou dias/anos, que representam a escala de tempo geológica/operacional do processo de recuperação de óleo.

Representação Visual de Sistemas Equivalentes

Para ilustrar a interdependência dos fenômenos físicos modelados, a Figura 3.1 apresenta um esquema clássico da influência da razão de mobilidade (M) no deslocamento. A imagem correlaciona visualmente as três saídas principais do simulador proposto: a curva de fluxo fracionário (linha inferior), o perfil de saturação com a formação da frente de choque (linha intermediária) e a curva de eficiência de recuperação (linha superior). Este

sistema equivalente demonstra como a física do escoamento (f_w) dita a forma da frente de avanço e, conseqüentemente, o sucesso da injeção de água.

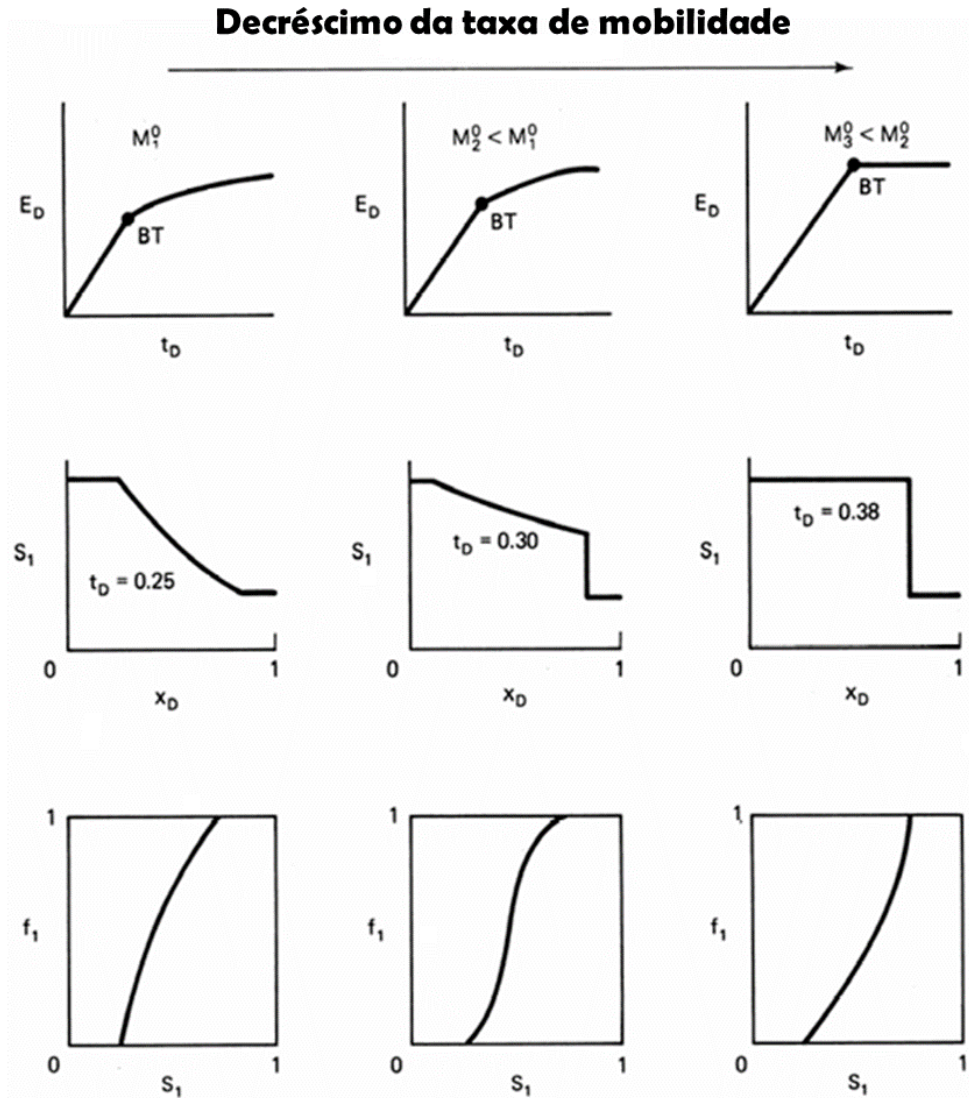


Figura 3.1: Ilustração esquemática do efeito da razão de mobilidade na eficiência de deslocamento, no perfil de saturação e no fluxo fracionário. Fonte: Adaptado de (LAKE et al., 2014)

3.2 Formulação teórica

A construção do núcleo de processamento matemático do software exige a tradução rigorosa dos fenômenos físicos de transporte em algoritmos computáveis. Para que a ferramenta desenvolvida seja capaz de prever o comportamento do reservatório com a precisão necessária para aplicações de engenharia, a modelagem deve fundamentar-se em

leis de conservação e equações constitutivas consagradas.

Nesta seção, detalha-se o arcabouço teórico que foi implementado nas classes do simulador. A formulação parte da descrição do movimento de fluidos em meios porosos (Lei de Darcy) e do princípio da conservação de massa, cuja combinação resulta na equação governante do fluxo fracionário. Em seguida, apresenta-se a solução analítica para o avanço da frente de saturação (BUCKLEY; LEVERETT, 1941) e o tratamento matemático para descontinuidades físicas (frentes de choque) através da técnica de (WELGE, 1952), algoritmos essenciais para a geração dos perfis de saturação exibidos pela interface gráfica.

3.2.1 Padronização de Unidades

O desenvolvimento matemático e a implementação computacional do núcleo deste trabalho adotam estritamente o Sistema Internacional de Unidades (SI). A escolha por um sistema de unidades consistentes simplifica o formalismo algébrico das equações de transporte e balanço de massa, garantindo a integridade dimensional direta sem a necessidade de fatores de conversão empíricos ou constantes multiplicativas adicionais.

A Tabela 3.1 especifica as grandezas físicas modeladas no simulador, seus respectivos símbolos analíticos e as unidades padronizadas empregadas em todo o arcabouço teórico.

Grandeza Física	
Comprimento / Distância	
Área da Seção Transversal	
Tempo	
Tempo Adimensional	
Vazão Volumétrica	
Velocidade de Darcy (Superficial)	
Permeabilidade Absoluta	
Permeabilidade Relativa	
Viscosidade Dinâmica	
Massa Específica	
Pressão	P
Tensão Interfacial	
Porosidade	
Saturação	
Fluxo Fracionário	
Ângulo de inclinação do reservatório	
Ângulo de contato de molhabilidade rocha-fluido	

Tabela 3.1: Sistema de unidades prático adotado na modelagem matemática

3.2.2 Fundamentação do Escoamento Bifásico e a Lei de Darcy Generalizada

A descrição matemática do escoamento de fluidos em meios porosos fundamenta-se, primariamente, na relação empírica proposta por (DARCY, 1856)¹. No entanto, para a análise de reservatórios de petróleo, onde comumente ocorre a coexistência de fluidos imiscíveis (como água e óleo), faz-se necessária a extensão da Lei de Darcy original para o escoamento multifásico.

Conforme estabelecido por (LAKE et al., 2014), a descrição do fluxo simultâneo de fluidos imiscíveis — neste caso, água e óleo — pressupõe que cada fase escoar através de redes de poros distintas e interconectadas. Assume-se, para esta formulação, um escoamento unidimensional, incompressível e isotérmico, onde as forças inerciais são desprezíveis em comparação às forças viscosas (regime laminar). A interferência mútua entre as fases é quantificada pelo conceito de permeabilidade relativa, que reduz a condutividade efetiva do meio para cada fluido em função de sua saturação local.

Sob estas condições, as velocidades superficiais (velocidades de Darcy) para as fases aquosa (w) e oleosa (o) em um sistema unidimensional inclinado são expressas pelas seguintes equações constitutivas, que consideram os efeitos viscosos, os efeitos gravitacionais e os gradientes de pressão atuantes em cada fase (ROSA; CARVALHO; XAVIER, 2006):

- Para fase aquosa:

$$u_w = -\frac{kk_{rw}}{\mu_w} \left(\frac{\partial p_w}{\partial x} + \rho_w g \sin \alpha \right) \quad (3.1)$$

- Para fase oleosa:

$$u_o = -\frac{kk_{ro}}{\mu_o} \left(\frac{\partial p_o}{\partial x} + \rho_o g \sin \alpha \right) \quad (3.2)$$

Onde u_i representa a velocidade de Darcy da fase i (sendo $i = w, o$), definida como a vazão volumétrica por unidade de área transversal (m/s). O termo k denota a permeabilidade absoluta do meio poroso (m^2), uma propriedade intrínseca da matriz rochosa, enquanto k_{ri} refere-se à permeabilidade relativa da fase i (adimensional).

As propriedades físicas dos fluidos são representadas pela viscosidade dinâmica μ_i ($Pa \cdot s$) e pela massa específica ρ_i (Kg/m^3). O termo $\frac{\partial p_i}{\partial x}$ corresponde ao gradiente de pressão na direção do escoamento. Por fim, o termo gravitacional é dado por $\rho_i g \sin \alpha$, onde g é a aceleração da gravidade e α é o ângulo de inclinação do reservatório, convencionado como positivo para o fluxo no sentido ascendente (*updip*).

3.2.3 Definições Auxiliares e Relações Constitutivas

Para a resolução do sistema de equações de fluxo, é necessário relacionar as pressões das fases oleosa e aquosa. Define-se a pressão capilar (p_c) como a diferença de pressão existente na interface entre dois fluidos imiscíveis em um meio poroso.

Embora a magnitude e o sinal da pressão capilar dependam da tensão interfacial e do ângulo de contato (molhabilidade) da rocha, para fins de desenvolvimento algébrico desta formulação, adota-se a convenção padrão apresentada por (LAKE et al., 2014):

¹O acesso ao conceito ocorreu através de autores modernos como Lake, que referenciam Henry Darcy.

$$p_c = p_o - p_w \implies p_o = p_w + p_c \quad (3.3)$$

Desta forma, os gradientes de pressão relacionam-se através da derivada espacial:

$$\frac{\partial p_o}{\partial x} = \frac{\partial p_w}{\partial x} + \frac{\partial p_c}{\partial x} \quad (3.4)$$

Adicionalmente, define-se a velocidade total de escoamento (u_t) como a soma algébrica das velocidades superficiais de cada fase. Considerando a premissa de incompressibilidade dos fluidos e da rocha, a velocidade total permanece constante ao longo da direção do fluxo unidimensional:

$$u_t = u_w + u_o \quad (3.5)$$

Por fim, introduz-se o conceito de fluxo fracionário da água (f_w), uma variável adimensional que representa a razão entre a vazão da fase aquosa e a vazão total do sistema:

$$f_w = \frac{u_w}{u_t} \quad (3.6)$$

Estas definições permitem a manipulação das equações de transporte para a eliminação dos gradientes de pressão individuais, consolidando o problema na determinação da saturação.

3.2.4 Desenvolvimento Algébrico e Eliminação dos Gradientes de Pressão

O objetivo desta etapa é combinar as leis de Darcy para as fases fluido com a definição de pressão capilar, eliminando-se os gradientes de pressão individuais $\frac{\partial p_w}{\partial x}$ e $\frac{\partial p_o}{\partial x}$.

Inicialmente, rearranjam-se as equações de Darcy (Eqs. 3.1 e 3.2) para explicitar os gradientes de pressão em função das velocidades e das forças gravitacionais:

$$\begin{aligned} \frac{\partial p_w}{\partial x} &= -\frac{u_w \mu_w}{k k_{rw}} - \rho_w g \sin \alpha \\ \frac{\partial p_o}{\partial x} &= -\frac{u_o \mu_o}{k k_{ro}} - \rho_o g \sin \alpha \end{aligned}$$

Substituindo-se estas expressões na equação diferencial da pressão capilar (3.4), obtém-se:

$$\frac{\partial p_c}{\partial x} = \left(-\frac{u_o \mu_o}{k k_{ro}} - \rho_o g \sin \alpha \right) - \left(-\frac{u_w \mu_w}{k k_{rw}} - \rho_w g \sin \alpha \right)$$

Agrupando-se os termos viscosos e gravitacionais, a expressão torna-se:

$$\frac{\partial p_c}{\partial x} = \left(\frac{u_w \mu_w}{k k_{rw}} - \frac{u_o \mu_o}{k k_{ro}} \right) + (\rho_w - \rho_o) g \sin \alpha$$

Neste ponto, é conveniente definir a diferença de densidade entre as fases, $\Delta\rho$, como a diferença entre a densidade da fase molhante (água) e a fase não-molhante (óleo):

$$\Delta\rho = \rho_w - \rho_o \quad (3.7)$$

Reescrevendo a equação com 3.7 e isolando os termos de velocidade no lado direito:

$$\frac{\partial p_c}{\partial x} - \Delta\rho g \sin \alpha = \frac{u_w \mu_w}{k k_{rw}} - \frac{u_o \mu_o}{k k_{ro}}$$

Para expressar a equação unicamente em termos da velocidade total (u_t) e da fração de fluxo (f_w), utiliza-se a relação de continuidade $u_o = u_t - u_w$:

$$\frac{\partial p_c}{\partial x} - \Delta\rho g \sin \alpha = \frac{u_w \mu_w}{k k_{rw}} - \frac{(u_t - u_w) \mu_o}{k k_{ro}}$$

A fim de simplificar a manipulação algébrica e isolar a velocidade da água (u_w), multiplica-se toda a equação pela mobilidade da fase oleosa, definida pelo termo $\frac{k k_{ro}}{\mu_o}$:

$$\frac{k k_{ro}}{\mu_o} \left(\frac{\partial p_c}{\partial x} - \Delta\rho g \sin \alpha \right) = u_w \frac{\mu_w k_{ro}}{\mu_o k_{rw}} - (u_t - u_w)$$

Expandindo o termo à direita e fatorando u_w :

$$\frac{k k_{ro}}{\mu_o} \left(\frac{\partial p_c}{\partial x} - \Delta\rho g \sin \alpha \right) = u_w \left(1 + \frac{\mu_w k_{ro}}{\mu_o k_{rw}} \right) - u_t$$

Rearranjando para isolar o termo que contém u_w :

$$u_w \left(1 + \frac{k_{ro} \mu_w}{k_{rw} \mu_o} \right) = u_t + \frac{k k_{ro}}{\mu_o} \left(\frac{\partial p_c}{\partial x} - \Delta\rho g \sin \alpha \right)$$

Finalmente, introduzindo a definição de fluxo fracionário ($f_w = u_w/u_t$) e dividindo toda a expressão por u_t , chega-se à forma preliminar da equação generalizada:

$$f_w \left(1 + \frac{k_{ro} \mu_w}{k_{rw} \mu_o} \right) = 1 + \frac{k k_{ro}}{u_t \mu_o} \left(\frac{\partial p_c}{\partial x} - \Delta\rho g \sin \alpha \right)$$

3.2.5 Equação do Fluxo Fracionário Generalizada

A partir da relação obtida na etapa anterior, isola-se explicitamente a fração de fluxo da água (f_w), resultando na formulação matemática generalizada que descreve a competição entre as forças viscosas, capilares e gravitacionais durante o escoamento bifásico em meios porosos:

$$f_w = \frac{1 + \frac{k k_{ro}}{u_t \mu_o} \left(\frac{\partial p_c}{\partial x} - \Delta\rho g \sin \alpha \right)}{1 + \frac{k_{ro} \mu_w}{k_{rw} \mu_o}} \quad (3.8)$$

Esta equação é a base fundamental para o desenvolvimento de modelos de simulação de reservatórios e para a aplicação de métodos analíticos de solução, como a teoria de *Buckley-Leverett*. A análise de seus termos permite distinguir três regimes de fluxo ou forças motrizes principais (LAKE et al., 2014):

1. **Forças Viscosas:** Representadas principalmente pelos termos de mobilidade no denominador ($M = \frac{k_{rw}/\mu_w}{k_{ro}/\mu_o}$). Em altas velocidades de fluxo (u_t), o termo aditivo no numerador tende a zero, e o fluxo é dominado pela razão de viscosidades.
2. **Forças Gravitacionais:** Representadas pelo termo $\frac{k k_{ro} \Delta \rho g \sin \alpha}{u_t \mu_o}$. Este componente quantifica a segregação de fluidos devido à diferença de densidade ($\Delta \rho$). Em escoamentos com baixa velocidade ou alta permeabilidade, a gravidade pode causar a contra-corrente (*countercurrent flow*) ou a segregação vertical, alterando significativamente a eficiência de varrido.
3. **Forças Capilares:** Representadas pelo gradiente de pressão capilar $\frac{\partial p_c}{\partial x}$. Embora frequentemente negligenciado em escalas de campo (hipótese de *Buckley-Leverett* clássica), este termo é crucial para descrever a difusão da frente de saturação e os efeitos de imbibição em escalas locais.

3.2.6 Análise Dimensional e Simplificação do Modelo Matemático

A equação generalizada obtida na etapa anterior descreve o escoamento bifásico considerando todas as forças atuantes. No entanto, para aplicações em escala de reservatório (macroscópica), é comum avaliar a magnitude relativa dessas forças para simplificar o modelo matemático sem perda significativa de precisão.

A influência das forças capilares na frente de avanço é regida pelo Número de Rapoport-Leas (N_{RL}). Conforme estabelecido no trabalho seminal de (RAPOPORT; LEAS, 1953) e discutido extensivamente por (LAKE et al., 2014), existe um valor crítico para o produto entre o comprimento do sistema (L), a velocidade de fluxo (u) e a viscosidade (μ), acima do qual o escoamento torna-se estabilizado e dominado pelas forças viscosas, tornando a dispersão capilar desprezível na escala de análise.

Considerando um sistema linear de comprimento L , este número adimensional é expresso pela razão entre o produto das forças viscosas macroscópicas e as forças capilares microscópicas:

$$N_{RL} = \sqrt{\left(\frac{1}{\phi k}\right) \frac{L \cdot u_t \cdot \mu_w}{k_{rw}^0 \sigma \cos \theta}} \quad (3.9)$$

Onde:

- $L[m]$ é o comprimento característico do sistema (ou a distância percorrida pela frente);
- u_t é a velocidade total de Darcy;
- μ_w é a viscosidade da fase deslocante (água);
- σ é a tensão interfacial entre os fluidos (água e óleo);
- k é a permeabilidade absoluta;
- ϕ é a porosidade da rocha;

- k_{rw}^0 é a permeabilidade relativa da água no ponto final (*endpoint*), isto é, avaliada na saturação residual de óleo (S_{or});
- $\cos\theta$ é o cosseno do ângulo de contato de molhabilidade rocha-fluido.

A análise dimensional demonstra que, para valores elevados de N_{RL} (tipicamente $N_{RL} > 3$ em unidades de campo consistentes), o termo convectivo (produto $Lu_t\mu_w$) domina o comportamento do fluxo. Nessas condições, a zona de transição capilar torna-se desprezível em relação à escala do reservatório, justificando a eliminação do termo $\frac{\partial p_c}{\partial x}$ na equação de fluxo fracionário.

Matematicamente, quando o critério de Rapoport-Leas é satisfeito ($N_{RL} \gtrsim 3$), o termo do gradiente de pressão capilar ($\frac{\partial p_c}{\partial x}$) pode ser desconsiderado em relação aos termos convectivos e gravitacionais. Sob esta hipótese de escoamento dominado por advecção, a formulação simplifica-se substancialmente. A equação geral do fluxo fracionário para reservatórios inclinados assume a seguinte forma paramétrica:

$$f_w = \frac{1 - \frac{k_{ro}}{u_t\mu_o}\Delta\rho g \sin\alpha}{1 + \frac{k_{ro}}{k_{rw}}\frac{\mu_w}{\mu_o}} \quad (3.10)$$

Nesta formulação geral, adequada para a aplicação do Método das Características (MOC), a fração de fluxo f_w torna-se uma propriedade constitutiva dependente exclusivamente da saturação de água (S_w), dado que as permeabilidades relativas (k_{rw} e k_{ro}) dependem apenas da saturação.

Para caracterizar o deslocamento de forma global e facilitar a análise adimensional, definem-se parâmetros constantes baseados nos pontos finais (*endpoints*) do sistema fluido-rocha, independentemente do modelo empírico escolhido para as curvas de permeabilidade (Corey, LET, Chierici, etc.):

- **Razão de mobilidade no ponto final (M^0):** Expressa a razão de mobilidade total calculada fixando as permeabilidades relativas máximas de cada fase:

$$M^0 = \frac{k_{rw}^0\mu_o}{\mu_w k_{ro}^0}$$

- **Número de gravidade no ponto final (N_g^0):** Define a magnitude máxima relativa entre as forças gravitacionais e as viscosas, calibrada pela permeabilidade de ponto final do óleo:

$$N_g^0 = \frac{k k_{ro}^0 \Delta\rho g}{\mu_o u_t}$$

Ao reescrever a equação geral de f_w utilizando essas constantes globais e agrupando os termos dependentes da saturação, obtém-se uma forma que evidencia a competição entre as forças motrizes, aplicável a qualquer modelo de permeabilidade:

$$f_w = \frac{1 - N_g^0 \left[\frac{k_{ro}(S_w)}{k_{ro}^0} \right] \sin\alpha}{1 + \frac{1}{M^0} \left[\frac{k_{ro}(S_w)}{k_{rw}(S_w)} \frac{k_{rw}^0}{k_{ro}^0} \right]}$$

A análise dos termos na equação geral revela a competição direta entre duas forças principais indexadas nos *endpoints*:

1. **Razão de Mobilidade no Ponto Final (M^0):** Governa o componente viscoso do escoamento, refletindo a facilidade relativa de transporte da fase injetada em comparação com a fase deslocada.
2. **Número de Gravidade no Ponto Final (N_g^0):** Modulado pela função local de permeabilidade do óleo ($\frac{k_{ro}(S_w)}{k_{ro}^0}$), quantifica o gradiente segregativo gerado pela diferença de massas específicas. A presença do termo gravitacional ($\sin \alpha$) é mantida pois atua como uma força aditiva de primeira ordem. Em reservatórios inclinados, ela altera de forma permanente a curvatura da função de fluxo fracionário, sendo capaz de gerar fenômenos de fluxo em contracorrente ou segregação gravitacional que são capturados com exatidão pela solução analítica hiperbólica do MOC.

Parametrização Constitutiva Padrão para a Análise Dimensional

Para a implementação computacional desta análise dimensional e o cálculo automático do Número de Rapoport-Leas (N_{RL}), a ferramenta analítica requer a definição de valores termodinâmicos para a tensão interfacial (σ) e o ângulo de contato (θ). Na ausência de dados laboratoriais específicos (análises PVTtextbackslashstrikeout default\textbackslashxout default\textbackslashuuline default\textbackslashuwave default\textbackslashupshape\textbackslash}\color inheritPressão, Volume e Tempo ou testes de testemunhos), o modelo matemático adota parâmetros padrão (*defaults*) estritamente fundamentados na literatura clássica para sistemas convencionais de recuperação secundária.

Assume-se, como premissa conservadora para a avaliação teórica de deslocamentos, um sistema rocha-fluido fortemente molhável à água (*strongly water-wet*). Sob esta condição, o ângulo de contato formado pela interface água-óleo na superfície porosa tende a zero ($\theta = 0^\circ$). Consequentemente, o cosseno do ângulo atinge seu valor máximo ($\cos 0^\circ = 1$), refletindo a máxima expressão das forças capilares de embebição no meio, uma hipótese amplamente validada em estudos de escalonamento linear ((CRAIG, 1971; RAPOPORT; LEAS, 1953)).

Para a tensão interfacial entre a água de injeção (salmoura) e um óleo cru típico, o simulador adota o valor constante de $\sigma = 30 \text{ mN/m}$ (numericamente equivalente a dinas/cm). Esta magnitude é representativa para escoamentos imiscíveis de água e óleo em condições de reservatório, na ausência de agentes tensoativos ou métodos químicos de recuperação ((CRAIG, 1971; LAKE et al., 2014)).

A adoção destas constantes é justificada pelas observações empíricas consolidadas na engenharia de petróleo. Conforme apontado na formulação original de (RAPOPORT; LEAS, 1953), o agrupamento no denominador da equação ($k_{rw}^0 \sigma \cos \theta$) frequentemente resulta em magnitudes da ordem de 1 mN/m para meios hidrofílicos, permitindo a correta avaliação do critério de estabilização do fluxo ($N_{RL} \gtrsim 3$). Desta forma, o desenvolvimento do software resguarda o rigor físico e a integridade da análise dimensional, evitando o uso de constantes arbitrárias ocultas no código-fonte.

Simultaneamente, a arquitetura da interface gráfica é projetada para exibir estes valores de forma transparente ao usuário, permitindo a flexibilização e inserção de dados experimentais reais caso o cenário de simulação assim o exija.

3.2.7 Interpretação Física da Curva de Fluxo Fracionário: Uma Propriedade Constitutiva

A equação generalizada do fluxo fracionário (f_w), deduzida na seção anterior, constitui a relação fundamental que descreve a hidrodinâmica local do escoamento bifásico. No entanto, é imperativo compreender a natureza física desta expressão antes de aplicá-la à modelagem do avanço da frente de saturação.

Conforme discutido por (LAKE et al., 2014), a função $f_w(S_w)$ não descreve, por si só, a evolução temporal do deslocamento. Ela atua, na verdade, como uma propriedade constitutiva estática do sistema trino formado pela:

- Matriz Rochosa (definida por k e curvas de k_r);
- Fase Deslocante (definida por μ_w, ρ_w);
- Fase Deslocada (definida por μ_o, ρ_o).

Analicamente, a equação afirma que a fração de fluxo é uma função de estado, dependente exclusivamente das propriedades instantâneas dos fluidos e da rocha em um ponto específico. (DAKE, 1978) reforça essa interpretação ao tratar o fluxo fracionário como uma medida da eficiência de deslocamento pontual.

Fisicamente, isso implica que, para um Volume Elementar Representativo (REV) do reservatório com uma saturação de água fixa S_w , as forças viscosas e gravitacionais entram em equilíbrio instantâneo para ditar que uma fração exata f_w da vazão total (u_t) será transportada pela água.

Esta característica "estática" impõe uma limitação: embora a equação f_w quantifique a competência de fluxo das fases, ela não possui a variável tempo (t) explícita. Portanto, ela não é capaz de prever a posição dos fluidos no reservatório ou quando a água atingirá o poço produtor. Para transformar essa "fotografia" das forças atuantes em um "filme" do deslocamento de fluidos, torna-se necessário acoplar esta relação constitutiva a uma lei de conservação fundamental.

3.2.8 A Lei de Conservação de Massa e o Acoplamento do Sistema

A transição de uma descrição estática das propriedades de fluxo (f_w) para um modelo dinâmico de previsão de reservatórios exige a aplicação de uma lei de conservação fundamental. Para determinar como a saturação de água evolui no tempo e no espaço, realiza-se um balanço de massa em um volume de controle fixo no meio poroso (DAKE, 1978).

O Princípio da Continuidade

Considere-se um elemento diferencial de volume no reservatório, com área de seção transversal A constante e espessura Δx , conforme ilustrado na Figura 3.2.

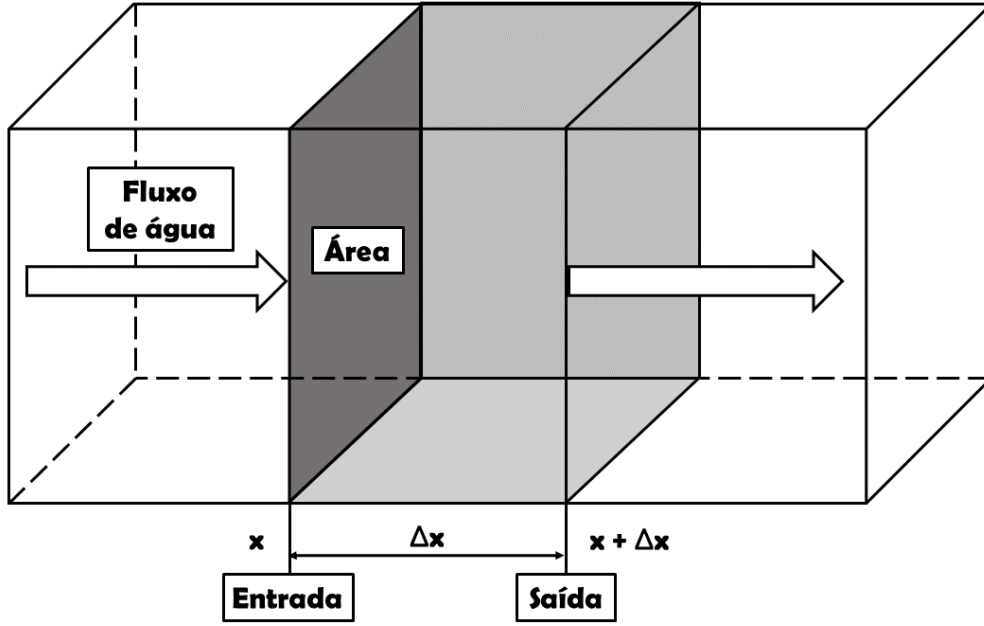


Figura 3.2: Volume Elementar Representativo (REV) para o balanço de massa unidimensional na fase aquosa

Assume-se a hipótese de imiscibilidade total, onde não há transferência de massa entre as fases (o componente água permanece restrito à fase aquosa). Desta forma, a lei de conservação de massa aplicada à fase água estabelece que o acúmulo de massa no volume é resultante do saldo líquido entre o fluxo de entrada e saída (LAKE et al., 2014). Da forma que:

$$[\text{Taxa de Acúmulo}] = [\text{Taxa de Entrada}] - [\text{Taxa de Saída}]$$

Matematicamente, para um intervalo de tempo Δt , o balanço pode ser escrito em termos de massa específica (ρ_w), velocidade superficial (u_w), porosidade (ϕ) e saturação (S_w):

1. Acúmulo de Massa: A variação da massa de água dentro do volume de controle ocorre devido à mudança na saturação, conforme descrito por (ROSA; CARVALHO; XAVIER, 2006):

$$\text{Acúmulo} = A\Delta x\phi [(\rho_w S_w)_{t+\Delta t} - (\rho_w S_w)_t]$$

2. Fluxo Líquido (Entrada - Saída): A massa que entra na face x e sai na face $x + \Delta x$ durante o intervalo Δt é dada por:

$$\text{Fluxo} = A\Delta t [(\rho_w u_w)_x - (\rho_w u_w)_{x+\Delta x}]$$

Igualando os termos e dividindo toda a expressão pelo volume do elemento e pelo intervalo de tempo ($A\Delta x\Delta t$), obtém-se a equação de diferenças finitas:

$$\frac{(\rho_w \phi S_w)_{t+\Delta t} - (\rho_w \phi S_w)_t}{\Delta t} + \frac{(\rho_w u_w)_{x+\Delta x} - (\rho_w u_w)_x}{\Delta x} = 0$$

Tomando-se o limite quando $\Delta t \rightarrow 0$ e $\Delta x \rightarrow 0$, a expressão converge para a equação diferencial parcial da continuidade (LAKE et al., 2014):

$$\frac{\partial(\rho_w \phi S_w)}{\partial t} + \frac{\partial(\rho_w u_w)}{\partial x} = 0 \quad (3.11)$$

Simplificações Físicas

Para o desenvolvimento da solução analítica clássica, aplicam-se as hipóteses de incompressibilidade dos fluidos e da rocha. Isto implica que:

1. A densidade da água (ρ_w) é constante no tempo e espaço.
2. A porosidade (ϕ) é constante e independente da pressão.

Sob estas condições, os termos constantes podem ser retirados das derivadas e cancelados, simplificando a equação para:

$$\phi \frac{\partial S_w}{\partial t} + \frac{\partial u_w}{\partial x} = 0$$

A Equação de Buckley-Leverett

O passo final consiste em acoplar a equação constitutiva do fluxo fracionário, deduzida na seção anterior, à equação da continuidade. Recordando a definição de fluxo fracionário:

$$u_w = u_t f_w$$

Substituindo u_w na equação de conservação:

$$\phi \frac{\partial S_w}{\partial t} + \frac{\partial(u_t f_w)}{\partial x} = 0$$

Em um escoamento incompressível e unidimensional sem fontes ou sumidouros ao longo do domínio, a velocidade total (u_t) é invariante com a posição ($\frac{\partial u_t}{\partial x} = 0$). Portanto, a equação assume sua forma final, conhecida como a equação fundamental do deslocamento de Buckley-Leverett:

$$\phi \frac{\partial S_w}{\partial t} + u_t \frac{\partial f_w}{\partial x} = 0 \quad (3.12)$$

Esta equação diferencial parcial hiperbólica descreve a dinâmica do transporte de saturação, indicando que a taxa de variação da saturação em um ponto é diretamente proporcional à velocidade de fluxo total e ao gradiente espacial do fluxo fracionário. Também conhecida como Equação de *Buckley-Leverett*, descreve o transporte convectivo da saturação através do meio poroso (BUCKLEY; LEVERETT, 1941).

3.2.9 Análise Dimensional e a Equação de *Buckley-Leverett*

Para que a solução matemática do problema de deslocamento tenha aplicabilidade universal — permitindo a extrapolação de resultados laboratoriais (escala de core) para reservatórios reais (escala de campo) —, é prática padrão na engenharia de reservatórios realizar a adimensionalização das variáveis independentes (LAKE et al., 2014).

O procedimento consiste em normalizar as coordenadas de tempo e espaço, eliminando a dependência explícita das dimensões físicas do sistema (L) e das propriedades volumétricas (ϕ), conforme detalhado a seguir.

Definição das Variáveis Adimensionais

Define-se a distância adimensional (x_D) como a fração do comprimento total do sistema linear (L) percorrida pelo fluido:

$$x_D = \frac{x}{L}$$

Desta relação, obtém-se o operador diferencial para o espaço: $\partial x = L \partial x_D$.

Analogamente, define-se o tempo adimensional (t_D) em termos de Volumes de Poros Injetados (PVI). Esta variável representa a razão entre o volume acumulado de fluido injetado e o volume poroso total do sistema (ROSA; CARVALHO; XAVIER, 2006). Para uma vazão de injeção constante, a expressão é dada por:

$$t_D = \frac{u_t t}{\phi L}$$

Diferenciando-se em relação ao tempo real, obtém-se o operador diferencial temporal: $\partial t = \frac{\phi L}{u_t} \partial t_D$.

A Equação Adimensional

Substituindo-se os operadores diferenciais na equação da conservação de massa (derivada na Subseção 3.2.8):

$$\phi \frac{\partial S_w}{\left(\frac{\phi L}{u_t} \partial t_D\right)} + u_t \frac{\partial f_w}{(L \partial x_D)} = 0$$

Simplificando os termos algébricos, observa-se o cancelamento das variáveis dimensionais ϕ , L e u_t :

$$\frac{u_t}{L} \frac{\partial S_w}{\partial t_D} + \frac{u_t}{L} \frac{\partial f_w}{\partial x_D} = 0 \implies \frac{\partial S_w}{\partial t_D} + \frac{\partial f_w}{\partial x_D} = 0$$

Como a fração de fluxo f_w é uma função exclusiva da saturação (S_w), conforme estabelecido pela equação constitutiva, aplica-se a regra da cadeia para expandir o termo espacial:

$$\frac{\partial f_w}{\partial x_D} = \frac{df_w}{dS_w} \frac{\partial S_w}{\partial x_D}$$

Substituindo este resultado na equação de transporte, chega-se à forma final da Equação de Buckley-Leverett:

$$\frac{\partial S_w}{\partial t_D} + \frac{df_w}{dS_w} \frac{\partial S_w}{\partial x_D} = 0 \quad (3.13)$$

Esta Equação Diferencial Parcial (EDP) quase-linear e hiperbólica relaciona a taxa de variação da saturação no tempo adimensional com o gradiente de saturação no espaço adimensional. O termo $\frac{df_w}{dS_w}$, correspondente à derivada da curva de fluxo fracionário em relação à saturação, emerge como o fator fundamental que governa a velocidade de propagação das saturações no meio poroso (BUCKLEY; LEVERETT, 1941).

3.2.10 Solução Analítica pelo Método das Características (MOC)

A equação de *Buckley-Leverett*, deduzida na forma adimensional na seção anterior, classifica-se matematicamente como uma Equação Diferencial Parcial (EDP) hiperbólica quase-linear de primeira ordem. Embora existam métodos numéricos (como Diferenças Finitas ou Volumes Finitos) capazes de resolver esta equação, a solução analítica via Método das Características (MOC) é frequentemente preferida para análises teóricas fundamentais. A resolução desta equação requer uma abordagem que preserve a integridade física das frentes de onda, evitando a dispersão numérica comum em métodos de diferenças finitas (LAKE et al., 2014).

Justificativa da Escolha do Método

A seleção do MOC fundamenta-se na sua capacidade de descrever o fenômeno de deslocamento com clareza física superior. Métodos numéricos tradicionais tendem a introduzir erros de truncamento que se manifestam como uma "dispersão numérica" artificial, suavizando artificialmente frentes de choque que deveriam ser abruptas (LAKE et al., 2014).

Em contraste, o Método das Características preserva a natureza hiperbólica do problema, tratando o escoamento como a propagação de ondas de saturação contínuas e descontínuas. Como ressalta (BEDRIKOVETSKY, 1993), esta abordagem permite visualizar explicitamente a competição entre as forças viscosas e gravitacionais através da velocidade das ondas, revelando a "física do escoamento" sem o mascaramento difusivo inerente às discretizações de malha fixa.

Desenvolvimento Matemático

O princípio central do MOC consiste em transformar a EDP original em um sistema de Equações Diferenciais Ordinárias (EDOs) válidas ao longo de trajetórias específicas no plano espaço-tempo (x_D, t_D) , denominadas curvas características (PRIIMENKO VIATCHESLAV E D. DE SIQUEIRA, 2017).

Considera-se que a saturação S_w é uma função contínua e diferenciável da posição e do tempo: $S_w = S_w(x_D, t_D)$. A diferencial total da saturação é dada por:

$$dS_w = \frac{\partial S_w}{\partial t_D} dt_D + \frac{\partial S_w}{\partial x_D} dx_D$$

O método busca identificar trajetórias no reservatório ao longo das quais a saturação de água permanece constante ($dS_w = 0$). Para uma saturação constante, a equação diferencial total rearranja-se para definir a inclinação destas trajetórias:

$$\left(\frac{dx_D}{dt_D} \right)_{S_w} = - \frac{\partial S_w / \partial t_D}{\partial S_w / \partial x_D}$$

Retomando a Equação de *Buckley-Leverett* (3.13):

$$\frac{\partial S_w}{\partial t_D} = - \frac{df_w}{dS_w} \frac{\partial S_w}{\partial x_D}$$

Substituindo esta relação na expressão da inclinação da trajetória, obtém-se a solução fundamental do MOC (BEDRIKOVETSKY, 1993):

$$\left(\frac{dx_D}{dt_D} \right)_{S_w} = \frac{df_w}{dS_w}$$

Interpretação Física

A equação acima demonstra que cada valor de saturação S_w propaga-se com uma velocidade constante e própria, igual à derivada da função de fluxo fracionário $f'_w(S_w)$. Integrando para uma injeção contínua ($t_D = 0, x_D = 0$), a posição da frente é:

$$x_D = t_D \cdot f'_w(S_w)$$

Este resultado analítico expõe a natureza não-linear do problema: como a função $f_w(S_w)$ não é linear (devido à gravidade e viscosidade), saturações diferentes viajam a velocidades diferentes. Isso leva inevitavelmente à sobreposição matemática de ondas ("quebra da onda"), exigindo a imposição de uma descontinuidade física (choque) para garantir a unicidade da solução, tema que será tratado na próxima seção sob a ótica da condição de entropia.

3.2.11 Condição de Entropia e Soluções Descontínuas

A aplicação direta do Método das Características à equação de *Buckley-Leverett* revela uma limitação fundamental quando a função de fluxo fracionário (f_w) não é convexa, como é o caso típico no deslocamento de óleo por água (formato em "S").

Conforme a solução analítica derivada ($x_D = t_D f'_w$), a velocidade de propagação de uma saturação é proporcional à derivada da curva de fluxo. Em regiões onde a derivada f'_w é crescente com a saturação, saturações mais altas viajam mais rapidamente do que saturações mais baixas situadas à sua frente. Matematicamente, isso leva à interseção das curvas características no plano espaço-tempo, resultando em um perfil de saturação "multivalorado" (BEDRIKOVETSKY, 1993)(PRIIMENKO VIATCHESLAV E D. DE SIQUEIRA, 2017).

O Colapso da Solução Clássica

Fisicamente, a existência de múltiplos valores de saturação no mesmo ponto do espaço (x_D) é impossível. Este fenômeno, análogo à quebra de uma onda mecânica, sinaliza o colapso da solução clássica (contínua) da Equação Diferencial Parcial.

Para resolver este impasse físico, recorre-se à teoria das leis de conservação hiperbólicas, conforme estudado em Métodos da Física-Matemática. É necessário abandonar a exigência de continuidade e buscar uma solução fraca (integral) que admita descontinuidades. No contexto do deslocamento de fluidos, essa descontinuidade manifesta-se como uma Frente de Choque.

A Condição de Entropia

A introdução de descontinuidades matemáticas permite infinitas soluções fracas possíveis para a EDP. Para selecionar a única solução fisicamente admissível, aplica-se a Condição de Entropia (ou Condição de Oleinik).

Esta condição estabelece que uma descontinuidade (choque) só é estável se as características "entrarem" no choque e não "saírem" dele. Em termos de engenharia de reservatórios, isso significa que a velocidade da frente de choque (v_σ) deve ser intermediária entre a velocidade da saturação imediatamente atrás do choque (v_L) e a velocidade da saturação imediatamente à frente dele (v_R):

$$v_L \geq v_\sigma \geq v_R$$

Ou:

$$f'_w(S_L) \geq \frac{f_w(S_L) - f_w(S_R)}{S_L - S_R} \geq f'_w(S_R)$$

Onda de Choque e Onda de Rarefação

A aplicação deste critério resulta na decomposição do perfil de saturação em duas estruturas distintas, fundamentais na análise física do problema:

1. **Onda de Choque:** Uma frente abrupta onde a saturação salta instantaneamente de um valor inicial (geralmente S_{wi}) para um valor de frente (S_{wf}). Ela ocorre na região onde as velocidades das características diminuem na direção do fluxo, violando a unicidade da solução contínua.
2. **Onda de Rarefação:** Uma zona de variação suave da saturação, que ocorre atrás da frente de choque (entre S_{wf} e $1 - S_{or}$). Nesta região, as características divergem ("se espalham") sem se cruzar, permitindo que a solução contínua do MOC permaneça válida.

A determinação exata da saturação onde ocorre a transição entre a rarefação e o choque (S_{wf}) é obtida através da relação de balanço de massa através da descontinuidade (Condição de Rankine-Hugoniot), geometricamente resolvida pela Construção de Welge.

3.2.12 Solução da Frente de Choque: A Construção de Welge

A imposição da condição de entropia exige que a solução física para o deslocamento contenha uma descontinuidade abrupta de saturação (choque) na região onde a velocidade das ondas é multivalorada. Para determinar a magnitude e a posição deste choque, utiliza-se a técnica analítica desenvolvida por (WELGE, 1952).

A Condição de Rankine-Hugoniot

Em problemas de leis de conservação hiperbólicas, a velocidade de propagação de uma descontinuidade (v_σ) deve respeitar o balanço de massa integral através da interface do choque. Esta relação, conhecida na física-matemática como Condição de Rankine-Hugoniot, iguala a velocidade do choque à razão entre a variação de fluxo e a variação de saturação através da descontinuidade:

$$v_\sigma = \left(\frac{\Delta x_D}{\Delta t_D} \right)_{choque} = \frac{f_w(S_{wf}) - f_w(S_{wi})}{S_{wf} - S_{wi}}$$

Onde:

- S_{wf} : Saturação de água na frente de choque (o valor que buscamos determinar).
- S_{wi} : Saturação de água inicial do reservatório (estado à frente do choque).
- $f_w(S_{wf})$ e $f_w(S_{wi})$: Fluxos fracionários correspondentes.

O Princípio da Tangência

Para que a solução seja contínua e fisicamente consistente, a velocidade da frente de choque deve ser exatamente igual à velocidade da onda de saturação imediatamente atrás dela (o último ponto da zona de rarefação). Matematicamente, isso implica igualar a velocidade do choque (Rankine-Hugoniot) à velocidade característica (derivada da função f_w) no ponto S_{wf} :

$$\frac{f_w(S_{wf}) - f_w(S_{wi})}{S_{wf} - S_{wi}} = \left(\frac{df_w}{dS_w} \right)_{S_{wf}}$$

Geometricamente, esta igualdade descreve uma reta que parte do ponto inicial $(S_{wi}, f_w(S_{wi}))$ e tangencia a curva de fluxo fracionário exatamente no ponto $(S_{wf}, f_w(S_{wf}))$.

Esta construção gráfica, denominada Tangente de Welge, permite determinar univocamente a saturação da frente de choque (S_{wf}) mesmo em cenários complexos com efeitos gravitacionais, onde a curva f_w pode apresentar pontos de inflexão.

Determinação da Saturação Média ($\bar{S}_w$)

Além de localizar a frente de avanço, o método de Welge fornece uma maneira direta de calcular a saturação média de água em todo o reservatório atrás da frente, uma variável fundamental para o cálculo da recuperação de óleo acumulada (N_p).

Integrando o perfil de saturação desde a entrada ($x_D = 0$) até a frente de choque (x_{Df}), (WELGE, 1952) demonstrou que a saturação média ($\bar{S}_w$) na zona varrida é dada pela interceptação da reta tangente com a linha de topo do gráfico ($f_w = 1$):

$$\bar{S}_w = S_{wf} + \frac{1 - f_w(S_{wf})}{f'_w(S_{wf})}$$

Esta relação simplifica drasticamente o cálculo da recuperação de óleo, eliminando a necessidade de integração numérica do perfil de saturação a cada passo de tempo. Basta estender a reta tangente até o eixo $f_w = 1$ e ler o valor correspondente no eixo das abcissas.

3.2.13 Cálculo dos Indicadores de Recuperação e Eficiência de Deslocamento

A obtenção da solução analítica para o perfil de saturação e a determinação da frente de choque não constituem um fim em si mesmas, mas sim a base fundamental para a previsão de desempenho do reservatório. O objetivo final da modelagem matemática desenvolvida nas seções anteriores é quantificar volume de óleo recuperável e a evolução temporal da produção.

Uma vez determinado o perfil de saturação $S_w(x_D, t_D)$ e a saturação média $\bar{S}_w$ (via método de Welge), é possível calcular os principais indicadores de engenharia:

Eficiência de Deslocamento (E_D)

A área sob o perfil de saturação representa fisicamente o volume de água que entrou no sistema e, conseqüentemente, devido à incompressibilidade, o volume de óleo que foi deslocado. A eficiência de deslocamento microscópico é definida como a fração do volume de óleo originalmente existente que foi efetivamente recuperada num dado instante (ROSA; CARVALHO; XAVIER, 2006):

$$E_D = \frac{\bar{S}_w - S_{wi}}{1 - S_{wi}}$$

Onde $\bar{S}_w$ é a saturação média de água na zona varrida. Este indicador varia monotonicamente com o tempo, atingindo seu valor máximo teórico apenas quando todo o reservatório atinge a saturação residual de óleo (S_{or}).

Para o período pós-ruptura ($t_D > t_{bt}$), a frente de choque já abandonou o meio poroso, e a saturação de água na face de saída do reservatório ($S_{w,out}$) aumenta progressivamente. De acordo com o Método das Características aplicado a $x_D = 1$, a saturação que atinge o poço produtor num dado tempo t_D é governada pela relação $f'_w(S_{w,out}) = 1/t_D$.

A extensão do método de Welge demonstra que a saturação média de água no reservatório ($\bar{S}_w$) continua sendo determinada pela interceptação da tangente geométrica com a linha $f_w = 1$. Contudo, para $t_D > t_{bt}$, a reta não se origina na saturação inicial, mas tangencia localmente a curva de fluxo fracionário no ponto de saída ($S_{w,out}, f_{w,out}$). Matematicamente, a saturação média generalizada para a zona de escoamento bifásico é expressa por:

$$\bar{S}_w = S_{w,out} + (1 - f_w(S_{w,out})) \cdot t_D$$

É esta formulação dinâmica que permite prever o crescimento assintótico da recuperação de óleo e a queda da eficiência de deslocamento microscópico ao longo do tempo de injeção avançado.

Tempo de Ruptura (*Breakthrough*)

Um dos parâmetros mais críticos fornecidos pela solução analítica é o tempo de ruptura (t_{bt}), que marca o instante exato em que a frente de choque alcança o poço produtor ($x_D = 1$). Utilizando a equação da velocidade da frente derivada pelo MOC, o tempo adimensional de ruptura é calculado pelo inverso da derivada do fluxo fracionário na saturação de choque:

$$t_{bt} = \frac{1}{f'_w(S_{wf})}$$

Antes deste instante ($t_D < t_{bt}$), apenas óleo é produzido no poço de saída, e a recuperação acumulada é linearmente igual ao volume injetado ($N_p = t_D$). Após a ruptura ($t_D > t_{bt}$), a água começa a ser coproduzida, e a razão de água no fluxo (BSW) aumenta progressivamente, reduzindo a eficiência econômica do projeto (LAKE et al., 2014).

Recuperação de Óleo Acumulada (N_p)

Por fim, a integração desses conceitos permite prever a curva de produção acumulada de óleo (N_p) em função do tempo ou do volume injetado. Em unidades de volumes porosos (PVI), a recuperação é dada diretamente pela variação da saturação média no reservatório:

$$N_{p(PVI)} = \bar{S}_w - S_{wi}$$

Portanto, todo o esforço matemático de deduzir f_w , aplicar a conservação de massa e resolver via Welge culmina nesta capacidade preditiva: estimar quanto óleo será produzido e em que momento a água invadirá os poços produtores, permitindo a análise técnica e econômica de diferentes cenários de injeção e molhabilidade.

3.3 Identificação de pacotes – assuntos

A identificação de pacotes constitui a etapa fundamental do projeto arquitetural, onde o sistema é decomposto em subsistemas coesos e fracamente acoplados. Segundo (PRESMAN; MAXIM, 2016), a arquitetura de software não trata apenas da estrutura operacional dos componentes, mas estabelece a organização hierárquica que permite o desenvolvimento paralelo, a manutenibilidade e o reuso de código.

Neste trabalho, a arquitetura foi estruturada seguindo o padrão arquitetural em camadas, isolando a lógica de apresentação (interface) das regras de negócio (núcleo matemático). Conforme recomenda (SOMMERVILLE, 2011), essa modularização é essencial em softwares de engenharia para garantir que alterações nos modelos físicos (como a adição de novas correlações de permeabilidade) não impactem a estabilidade da interface gráfica ou dos módulos de persistência de dados.

- **Pacote de Interface (View):** Responsável exclusivamente pela camada de apresentação. Contém as classes que definem as janelas (`MainWindow`), diálogos e formulários construídos com o *framework* Qt. Sua função é capturar os eventos do usuário (cliques, entradas de dados) e repassá-los ao Controlador, sem realizar processamento matemático direto, garantindo uma interface leve e responsiva.

- **Pacote de Controle (*Controller*)**: Atua como o intermediário entre a Interface e o Domínio, implementando a lógica de aplicação. Este pacote recebe as requisições da interface (ex: "Calcular Fluxo Fracionário"), orquestra a validação dos dados, invoca os métodos numéricos do núcleo e devolve os resultados formatados para a visualização. Ele assegura o desacoplamento do sistema.
- **Pacote de Domínio (*Model*)**: Constitui o núcleo científico do simulador. Encapsula as classes que representam as entidades físicas da Engenharia de Reservatórios (Fluido, Rocha, Reservatório) e os algoritmos de solução (**BuckleyLeverettSolver**). É aqui que residem as leis de conservação e as equações constitutivas, mantendo-se totalmente independente de bibliotecas gráficas.
- **Pacote de Modelos de Permeabilidade (*Strategies*)**: Implementa o padrão de projeto *Strategy* para flexibilizar o cálculo das propriedades da rocha. Contém a interface comum e as classes concretas para os modelos analíticos (Corey, LET, Chierici) e para a interpolação de dados tabulares. Este isolamento permite que novos modelos de permeabilidade sejam adicionados futuramente sem alterar o código do Domínio.
- **Pacote de Gerenciamento de Dados (*IO*)**: Responsável pela persistência e recuperação de informações. Agrupa as classes especializadas na leitura e interpretação (*parsing*) de arquivos externos (.txt), tratando exceções de formatação e convertendo dados brutos em objetos do domínio, protegendo o núcleo de erros de entrada/saída.
- **Pacote de Visualização (*Plotting*)**: Especializado na renderização científica dos resultados. Integra a biblioteca **QCustomPlot** para gerar os gráficos cartesianos ($f_w \times S_w$, Perfis de Saturação). Este pacote encapsula as configurações estéticas (eixos, legendas, *grids*) e de interatividade gráfica, fornecendo "*widgets*" prontos para serem exibidos pela Interface.

3.4 Diagrama de pacotes – assuntos

A representação visual da arquitetura do sistema é consolidada no Diagrama de Pacotes, apresentado na Figura 3.3. Este artefato oferece uma visão de alto nível sobre a organização modular do software, ilustrando como as fronteiras arquiteturais foram estabelecidas para promover a separação de interesses.

Segundo (PRESSMAN; MAXIM, 2016), a modelagem de pacotes é essencial para comunicar a estrutura estática do sistema, evidenciando as dependências entre os subsistemas. No diagrama proposto, observa-se a centralidade do pacote de Controle, que atua como o orquestrador do sistema, isolando a Interface (camada de apresentação) das complexidades do Domínio (camada de negócio) e dos detalhes técnicos de Visualização e Gerenciamento de Dados.

Esta estrutura hierárquica reflete o princípio da alta coesão e baixo acoplamento preconizado por (SOMMERVILLE, 2011), assegurando que mudanças nos algoritmos matemáticos (Domínio) ou nos modelos de permeabilidade (Estratégias) não propaguem

efeitos colaterais para a interface gráfica, garantindo a manutenibilidade e a robustez do simulador a longo prazo.

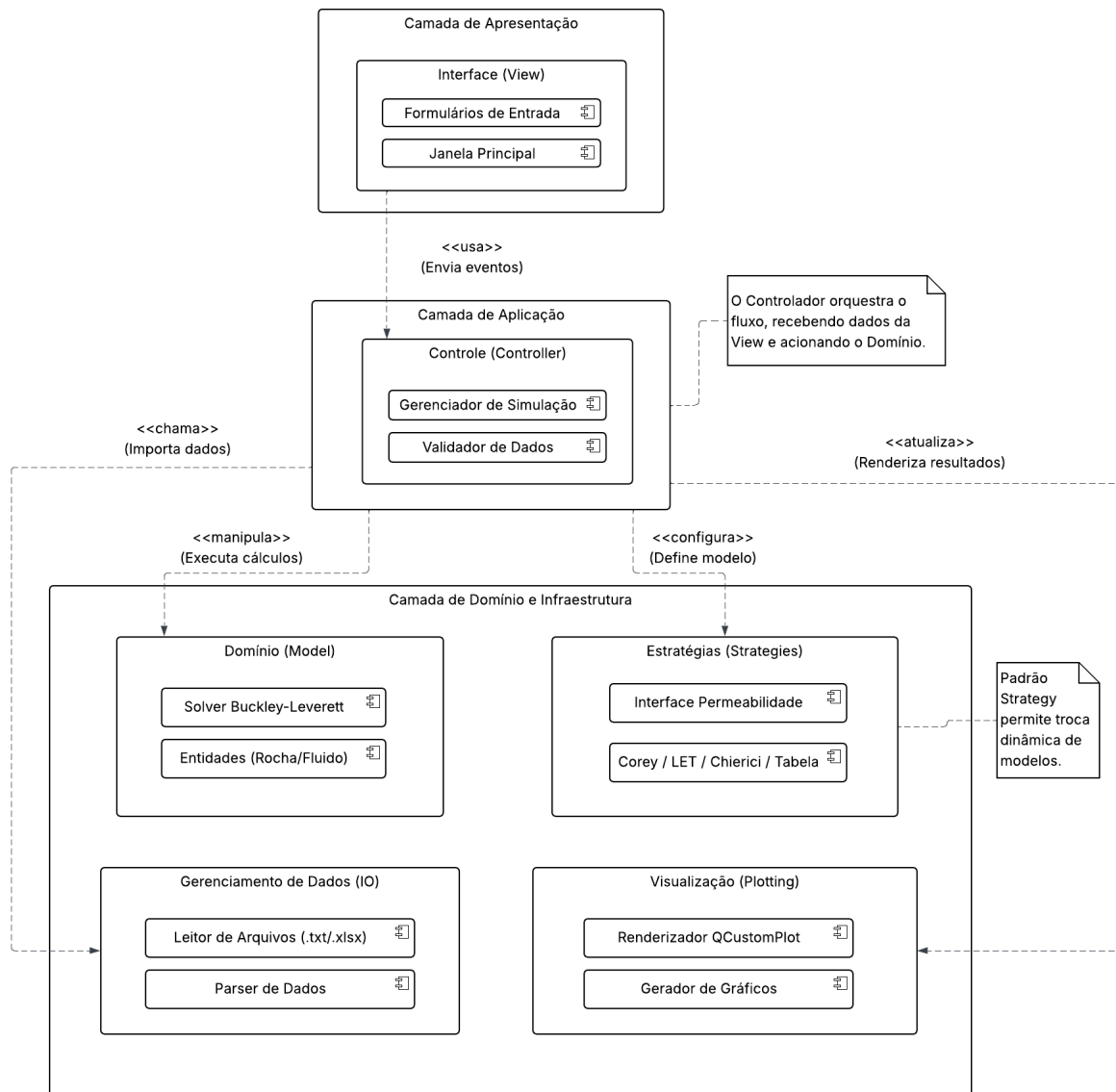


Figura 3.3: Diagrama de Pacotes - Arquitetura modular do simulador evidenciando o padrão de camadas e o desacoplamento pelo Controlador

Capítulo 4

AOO – Análise Orientada a Objeto

Neste capítulo, apresenta-se a Análise Orientada a Objetos (AOO) do simulador educacional, detalhando como as equações de Buckley-Leverett e as correlações de permeabilidade foram encapsuladas em classes e como esses objetos interagem para resolver o problema de fluxo fracionário.

A AOO constitui uma etapa fundamental no desenvolvimento de software científico, pois permite transpor os modelos matemáticos e físicos definidos na fundamentação teórica para uma estrutura computacional lógica e coesa. Segundo (BUENO, 2003), a abordagem orientada a objetos facilita a modelagem de sistemas complexos ao decompor o problema em entidades autônomas (objetos) que possuem atributos (dados) e métodos (comportamentos), espelhando as entidades do mundo real, como reservatórios, fluidos e malhas de simulação.

O foco desta etapa é identificar o que o sistema deve fazer e quais elementos o compõem, independentemente de como eles serão implementados em código.

A modelagem resultante busca garantir um sistema modular, reutilizável e de fácil manutenção, alinhado às boas práticas da Engenharia de Software (SOMMERVILLE, 2011).

Nota: Detalhes da UML podem ser vistos aqui: <https://www.omg.org/uml/>.

4.1 Diagramas de classes

Para a representação da estrutura estática do sistema, utilizou-se a notação UML (*Unified Modeling Language*), resultando no Diagrama de Classes apresentado na Figura 4.1. A arquitetura segue uma separação de responsabilidades onde a interface gráfica (**View**) é desacoplada da lógica matemática.

Um aspecto central da implementação é a aplicação do Padrão de Projeto *Strategy* (GAMMA et al., 1994) para o cálculo das permeabilidades relativas. Essa escolha arquitetural permite que o modelo matemático de permeabilidade (Corey, LET ou Chierici) seja alterado em tempo de execução através de polimorfismo, sem a necessidade de modificar o código fonte do simulador ou do solver numérico.

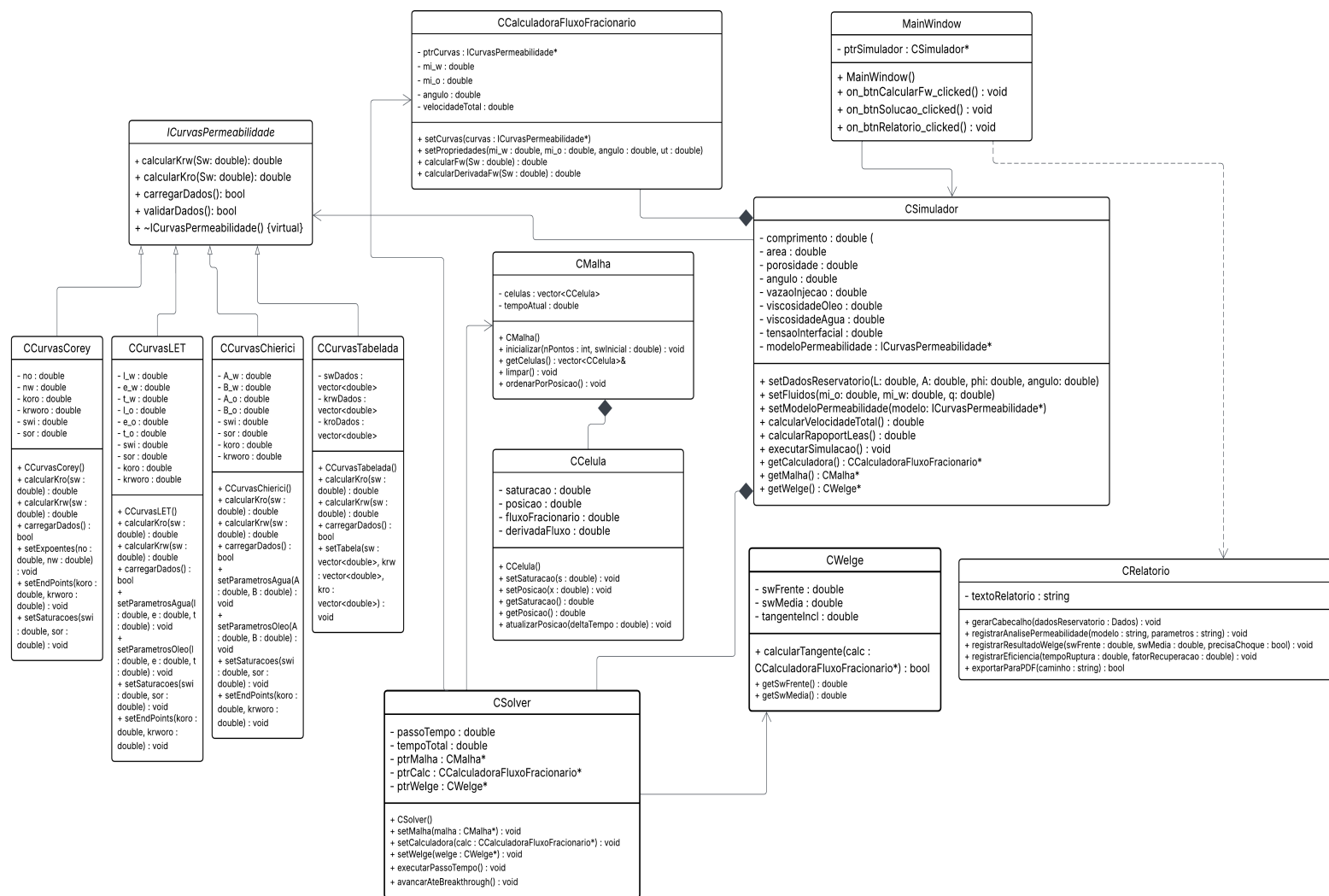


Figura 4.1: Diagrama de Classes do Simulador Educacional

4.1.1 Dicionário de classes

Esta seção detalha as responsabilidades e os principais atributos das classes que compõem o sistema (BOOCH et al., 2007):

A. Interface e Controle (MainWindow e CSimulador)

MainWindow: Classe responsável pela camada de apresentação (GUI). Gerencia a captura de eventos gerados pelo usuário (como `on_btnSolucao_clicked`) e a visualização dos gráficos. Possui um ponteiro para o controlador principal (`ptrSimulador`), para o qual delega as requisições de cálculo.

CSimulador: Atua como o gerenciador central do sistema. Suas responsabilidades incluem:

- Armazenar os dados de entrada do reservatório (comprimento, área, porosidade) e propriedades dos fluidos (viscosidades, tensão interfacial).
- Gerenciar a estratégia de permeabilidade ativa através do ponteiro polimórfico `modeloPermeabilidade`.
- Instanciar e configurar os objetos matemáticos antes da execução da simulação.

B. Núcleo Numérico (CSolver, CCalculadora e CWelge)

CSolver: Implementa a lógica do Método das Características (MOC). É responsável por controlar o passo de tempo (`passoTempo`) e orquestrar a evolução da saturação ao longo da malha.

CCalculadoraFluxoFracionario: Classe auxiliar focada em cálculos pontuais. Mantém um ponteiro `ptrCurvas` para acessar as permeabilidades relativas e utiliza as viscosidades (`mi_w`, `mi_o`) para calcular o fluxo fracionário (f_w) e sua derivada (velocidade da onda) para qualquer saturação fornecida.

CWelge: Responsável pela aplicação do método da tangente de Welge. É acionada pelo solver quando há necessidade de determinar a saturação da frente de choque (`swFrente`) e a saturação média (`swMedia`) para garantir o balanço de massa na frente de avanço.

C. Estrutura de Dados Lagrangiana (CMalha e CCelula)

CMalha: Representa o domínio do reservatório de forma dinâmica. Armazena um vetor de células (`celulas`) e controla o tempo atual da simulação. Diferente de malhas eulerianas fixas, esta classe gerencia pontos que se movem no espaço.

CCelula: Unidade fundamental da simulação lagrangiana. Cada objeto representa um ponto de saturação constante que se desloca pelo reservatório. Seus principais atributos são:

- `saturacao`: O valor de saturação de água (S_w) transportado pela célula.
- `posicao`: A localização espacial atual (x) da célula.
- `derivadaFluxo`: Armazena a velocidade de avanço da célula, calculada com base na derivada do fluxo fracionário.

D. Modelagem de Permeabilidade (Estratégias)

ICurvasPermeabilidade: Interface abstrata que define o contrato para os modelos de permeabilidade. Declara métodos virtuais puros como `calcularKrw` e `calcularKro`, além do destrutor virtual, obrigando as classes derivadas a implementarem a física específica de cada correlação.

CCurvasCorey: Implementação do modelo de Corey, utilizando os expoentes `no` e `nw` para definir a curvatura das permeabilidades.

CCurvasLET: Implementa a correlação LET, caracterizada por três parâmetros empíricos para cada fase: L (forma), E (elevação) e T (translação). No código, estes são representados pelos atributos `l_w`, `e_w`, `t_w` (para água) e seus correspondentes para óleo.

CCurvasChierici: Implementa o modelo exponencial de Chierici, utilizando os parâmetros `A_w` e `B_w` para a fase aquosa e `A_o`, `B_o` para a fase óleo.

CCurvasTabelada: Permite o uso de dados experimentais arbitrários, armazenados em vetores (`swDados`, `krwDados`, `kroDados`), utilizando interpolação para os cálculos.

E. Saída de Dados (CRelatorio)

CRelatorio: Classe responsável por compilar os resultados da simulação e exportá-los em formato PDF. Ela formata os dados de eficiência de varrido, fator de recuperação e análise de Welge em um texto técnico para fins didáticos.

4.2 Diagrama de sequência – eventos e mensagens

A modelagem estática, apresentada anteriormente no Diagrama de Classes, define a estrutura e a hierarquia dos componentes do sistema, mas é insuficiente para descrever o comportamento do software durante sua execução. Para capturar a dinâmica do sistema e a lógica temporal das operações, utilizam-se os Diagramas de Sequência.

Conforme (BOOCH et al., 2007), esses diagramas enfatizam a ordem temporal das mensagens trocadas entre os objetos, detalhando como o fluxo de controle passa de um componente para outro para realizar uma tarefa específica.

No contexto deste simulador, os diagramas de sequência são essenciais para demonstrar como os eventos da interface gráfica (cliques do usuário) são convertidos em chamadas de métodos no núcleo numérico, evidenciando a orquestração entre o controlador (`CSimulador`), o solver (`CSolver`) e as entidades matemáticas durante o avanço do tempo e a correção da frente de choque.

4.2.1 Diagrama de sequência geral

Enquanto o Diagrama de Classes descreve a estrutura estática do sistema, os Diagramas de Sequência são utilizados para modelar o comportamento dinâmico e a troca de mensagens entre os objetos ao longo do tempo (BOOCH et al., 2007).

A Figura 4.2 apresenta o Diagrama de Sequência Geral, que ilustra o fluxo macroscópico de execução quando o usuário solicita a solução do problema de Buckley-Leverett.

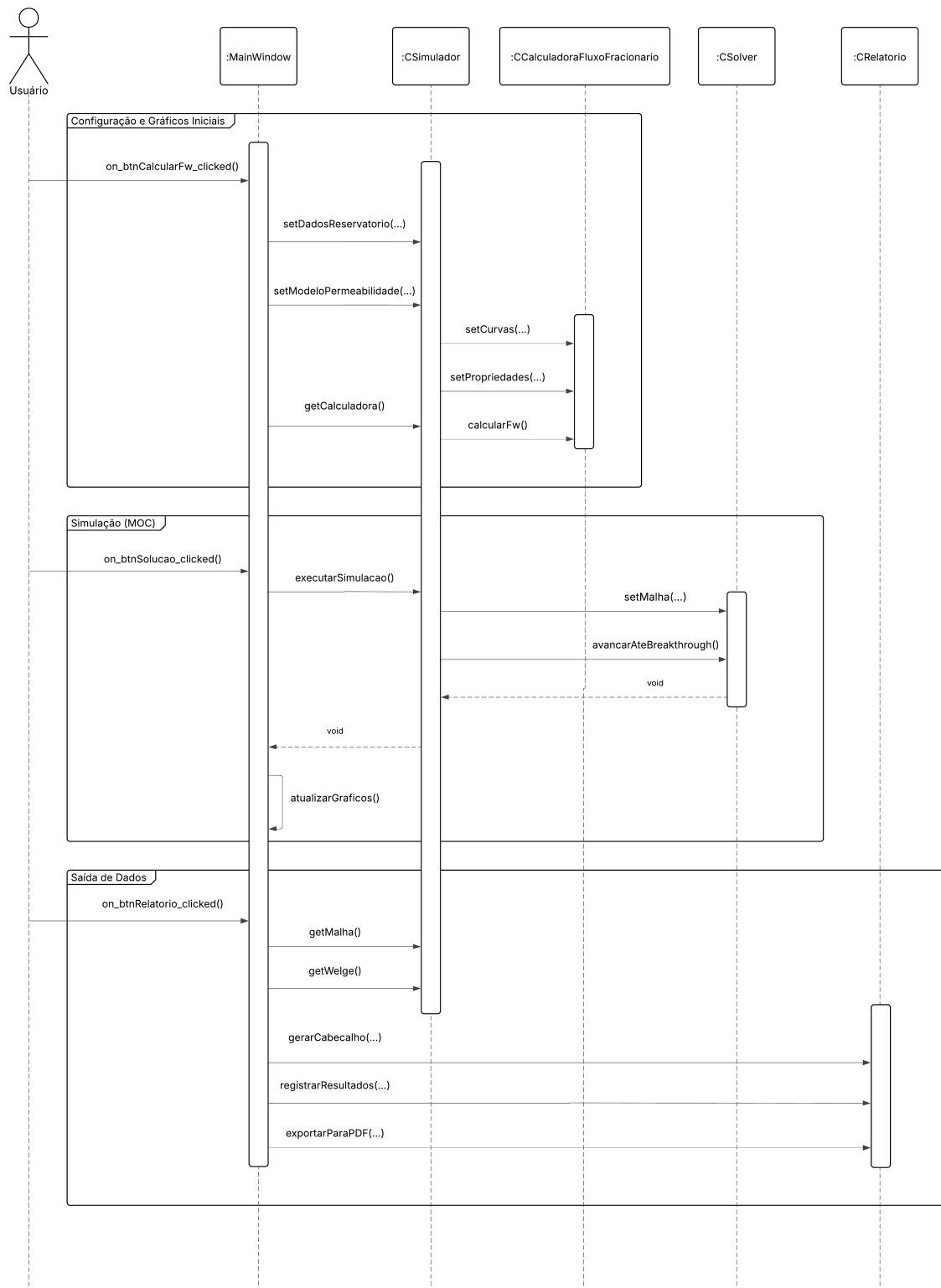


Figura 4.2: Diagrama de sequência geral

O processo inicia-se na camada de interface (`MainWindow`), que captura os parâmetros inseridos pelo usuário e configura o objeto controlador (`CSimulador`). Nota-se que a interface não se comunica diretamente com o núcleo numérico; ela delega a tarefa ao

`CSimulador`, respeitando o princípio de desacoplamento.

O `CSimulador`, por sua vez, atua como um orquestrador: ele prepara a malha e aciona o `CSolver`. O método `avancarAteBreakthrough()` encapsula todo o processo iterativo do Método das Características. Somente após o retorno deste método — indicando que a frente de saturação atingiu o poço produtor ou o tempo final — o controle retorna à interface para a renderização dos gráficos.

4.2.2 Diagrama de sequência específico

Cenário 1: O Passo de Tempo do MOC

O Diagrama de Sequência detalhado na Figura 4.3 ilustra a implementação computacional de um passo de tempo discreto (Δt) utilizando a abordagem Lagrangiana do Método das Características.

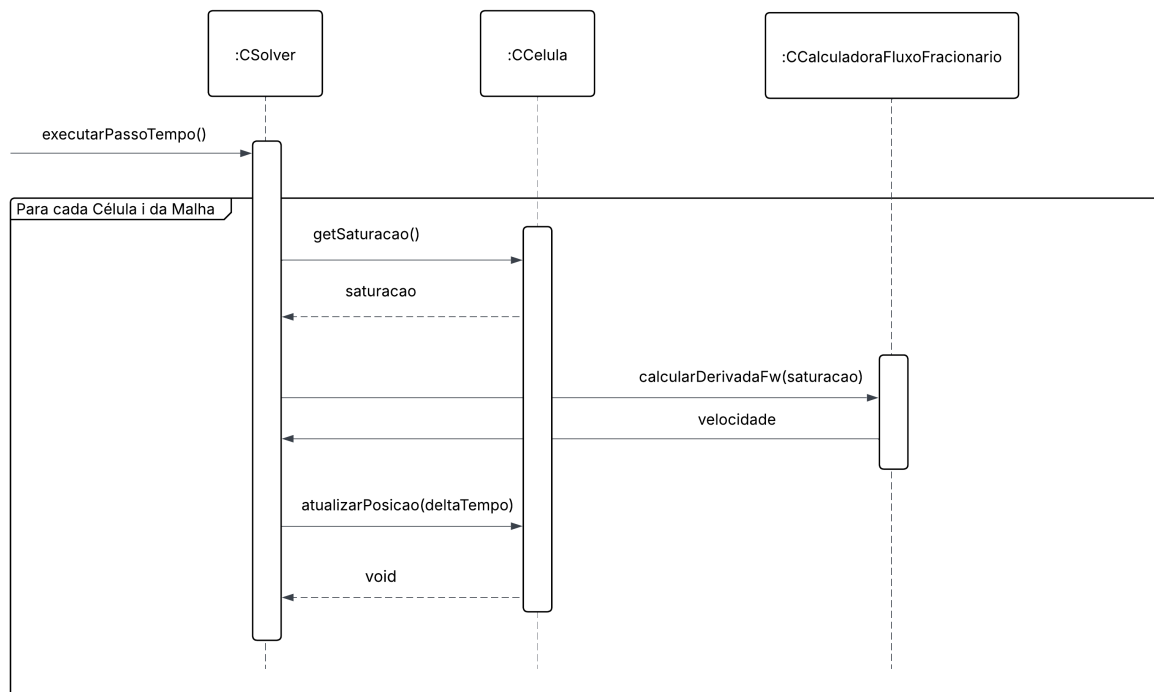


Figura 4.3: Diagrama de Sequência: Execução de um passo de tempo

O processo é iniciado pelo método `executarPassoTempo()` da classe `CSolver`. Para cada célula ativa na malha (`:CCelula`), o algoritmo realiza as seguintes operações sequenciais:

1. Consulta de Estado: O solver recupera a saturação atual da célula através do método `getSaturacao()`.
2. Cálculo da Velocidade de Onda: Com o valor da saturação, o solver aciona a `:CCalculadoraFluxoFracionario` para computar a derivada da curva de fluxo fracionário (f'_w) no ponto específico, através do método `calcularDerivadaFw()`.

3. Atualização Posicional: De posse da velocidade da onda característica, o solver ordena que a célula atualize sua posição espacial ($x_{new} = x_{old} + v \cdot \Delta t$) invocando o método `atualizarPosicao()`.
4. Esta estrutura garante que a lógica física (cálculo de derivada) permaneça isolada na calculadora, enquanto a célula mantém a responsabilidade sobre seus próprios dados de posição, respeitando o princípio de responsabilidade única.

Cenário 2: A Verificação de Welge

Em problemas de injeção de água, a formação de uma frente de choque abrupta exige um tratamento matemático especial para garantir o balanço de massa. A Figura 4.4 detalha como o sistema delega essa responsabilidade à classe `CWelge`.

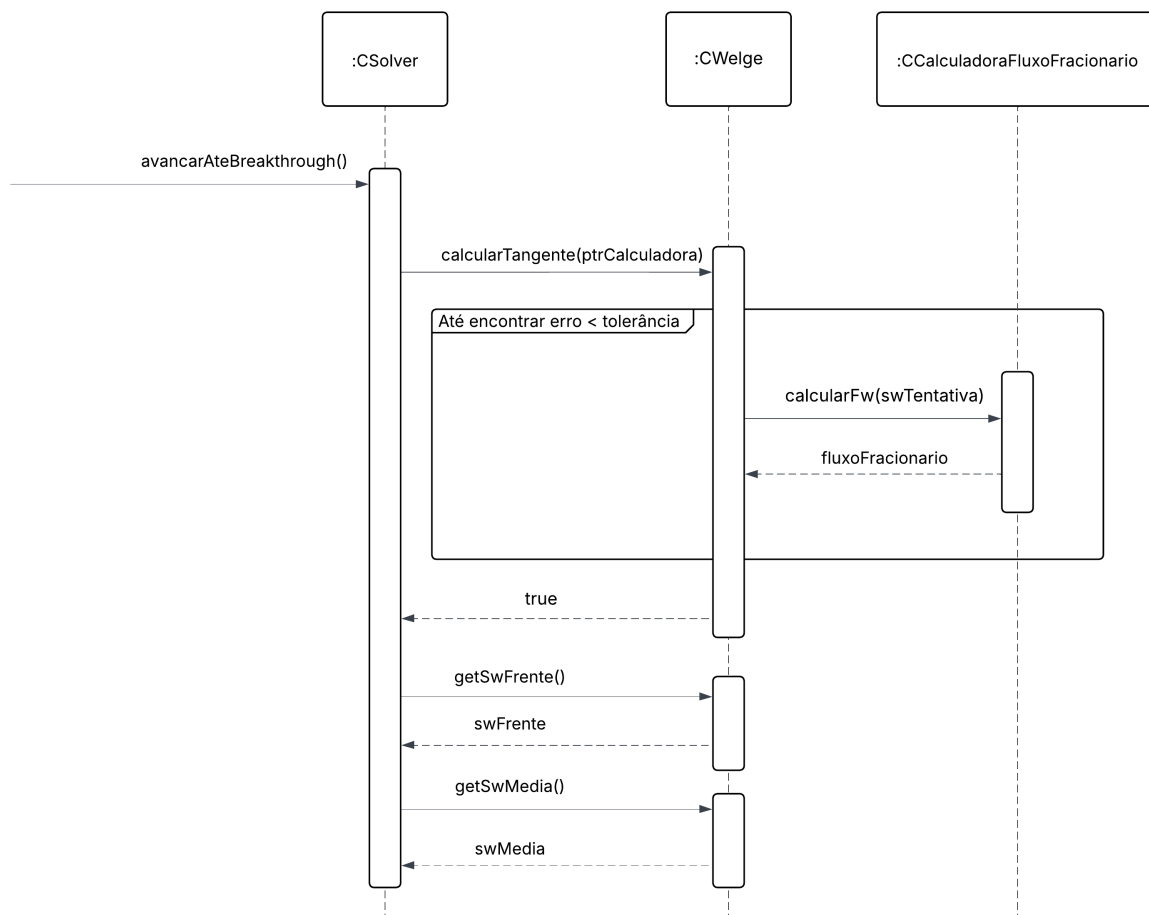


Figura 4.4: Diagrama de Sequência: Cálculo da Tangente de Welge

O processo de verificação é iniciado pelo `CSolver` durante a execução da simulação. Ao invocar o método `calcularTangente()`, o solver fornece à classe `CWelge` um ponteiro para a `CCalculadoraFluxoFracionario`. Isso exemplifica uma associação temporária, onde o objeto especialista recebe as ferramentas necessárias apenas para realizar sua tarefa.

Dentro do método `calcularTangente()`, ocorre um processo iterativo (representado pelo *loop* no diagrama) onde a classe `CWelge` realiza múltiplas consultas à calculadora (`calcularFw()`) para testar diferentes valores de saturação até satisfazer a condição de tangência geométrica.

Uma vez confirmada a convergência (retorno `true`), o `CSolver` solicita os resultados processados — a saturação da frente (`getSwFrente`) e a saturação média (`getSwMedia`) — para ajustar as posições das células na malha e corrigir o perfil de saturação.

4.3 Diagrama de comunicação – colaboração

Enquanto os diagramas de sequência enfatizam a ordem temporal das mensagens, o Diagrama de Comunicação foca na organização estrutural dos objetos que trocam mensagens (BOOCH et al., 2007). Ele evidencia as conexões entre os elementos, oferecendo uma visão do acoplamento do sistema como um todo e dos acoplamentos entre pares de objetos.

A Figura 4.3 apresenta a visão geral de comunicação, abrangendo desde a configuração das curvas até a geração do relatório.

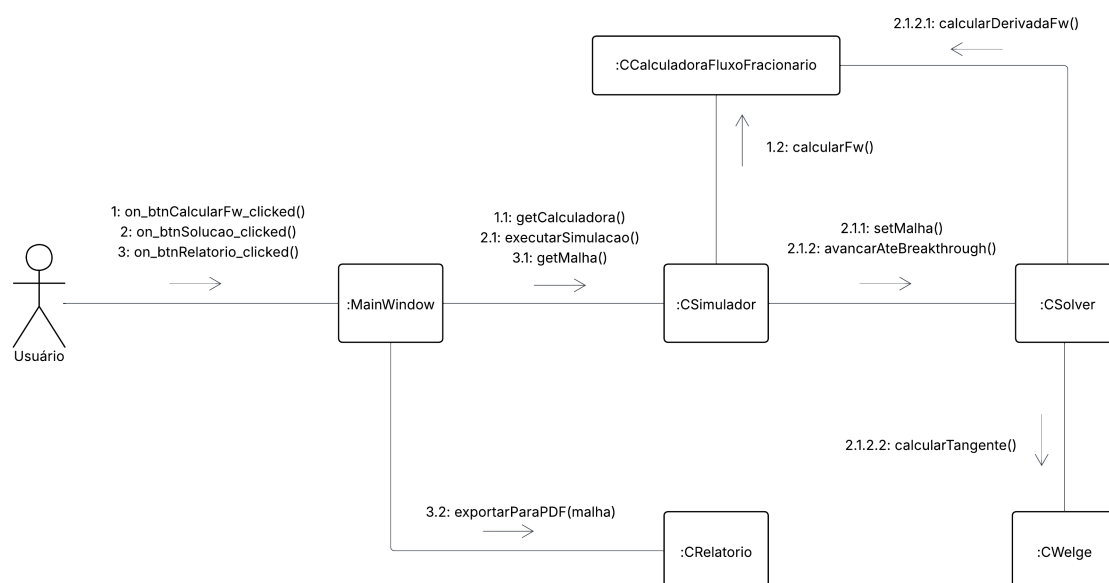


Figura 4.5: Diagrama de Comunicação: Fluxo hierárquico de execução

O diagrama utiliza uma numeração hierárquica para representar três fases distintas de interação iniciadas pelo ator **Usuário**:

1. Fase de Configuração (Mensagens 1.x): Inicia-se com a solicitação de cálculo das curvas. A `:MainWindow` obtém acesso à `:CCalculadoraFluxoFracionario` através do simulador (1.1) para exibir os gráficos iniciais.
2. Fase de Simulação (Mensagens 2.x): Ao solicitar a solução, o comando é delegado ao `:CSimulador`, que orquestra o `:CSolver`. Nota-se a profundidade da execução nas

mensagens 2.1.2.1 e 2.1.2.2, onde o solver interage intensamente com a calculadora e com o especialista `:CWelge` para resolver o avanço da frente de saturação.

3. Fase de Saída (Mensagens 3.x): Por fim, a interface solicita os dados processados e aciona o `:CRelatorio` para a exportação dos resultados.

Essa estrutura visual confirma que o princípio de Baixo Acoplamento foi respeitado: o `:CSolver` e o `:CWelge` (o núcleo matemático) estão isolados da interface gráfica, sendo acessados apenas indiretamente através do controlador, o que facilita a manutenção e testes dos componentes numéricos.

4.4 Diagrama de estado

Para garantir a robustez da aplicação e evitar operações inválidas (como tentar gerar um relatório antes de executar a simulação), o comportamento da classe controladora `:CSimulador` foi modelado através de uma Máquina de Estados Finita (SOMMERVILLE, 2011). A Figura 4.6 ilustra as transições de estado permitidas durante o ciclo de vida da execução.

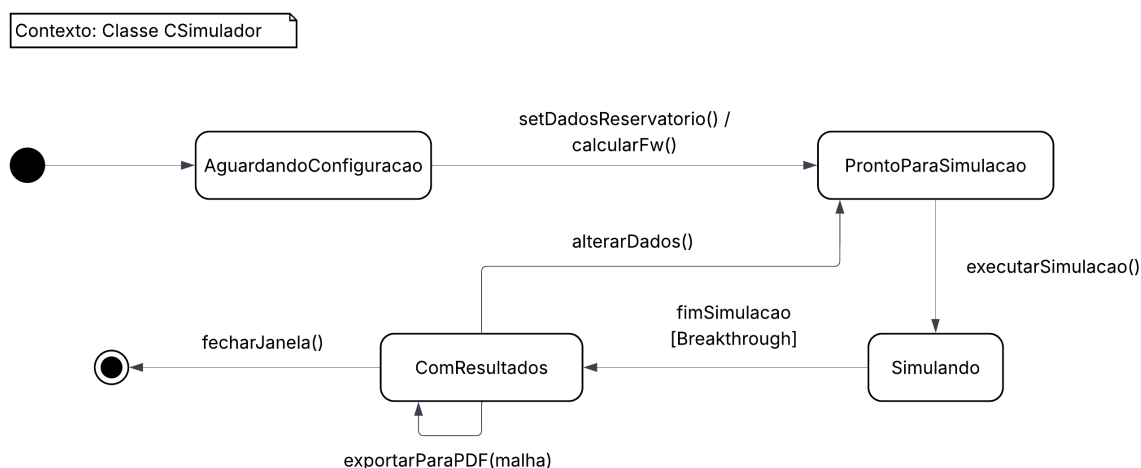


Figura 4.6: Diagrama de Máquina de Estados da classe `CSimulador`

O diagrama reflete a sequência lógica de operação imposta pela interface gráfica:

1. Estado `AguardandoConfiguracao`: O sistema inicia neste estado. Nenhuma simulação é permitida até que os parâmetros físicos sejam inseridos.
2. Transição de Configuração: Ao acionar o cálculo das curvas de fluxo fracionário (`calcularFw`), o objeto transita para o estado `ProntoParaSimulacao`. Isso valida os dados de entrada.
3. Estado `Simulando`: Ocorre durante a execução do método `executarSimulacao()`. Neste estado, a interface é bloqueada para evitar condições de corrida.

4. Estado **ComResultados**: Após o *breakthrough* (chegada da frente de saturação ao final do reservatório), o sistema alcança este estado final. Somente aqui a operação `exportarParaPDF()` é habilitada, garantindo que o relatório contenha dados consistentes e completos.

4.5 Diagrama de atividades

O Diagrama de Atividades permite modelar o fluxo sequencial e condicional de um processo algorítmico (BOOCH et al., 2007). Neste projeto, ele foi utilizado para detalhar a lógica crítica implementada no método `executarPassoTempo()` da classe `CSolver`, onde ocorre a integração entre o transporte lagrangiano e a correção física da frente de choque.

A Figura 4.7 apresenta o fluxograma de decisão para um passo de tempo.

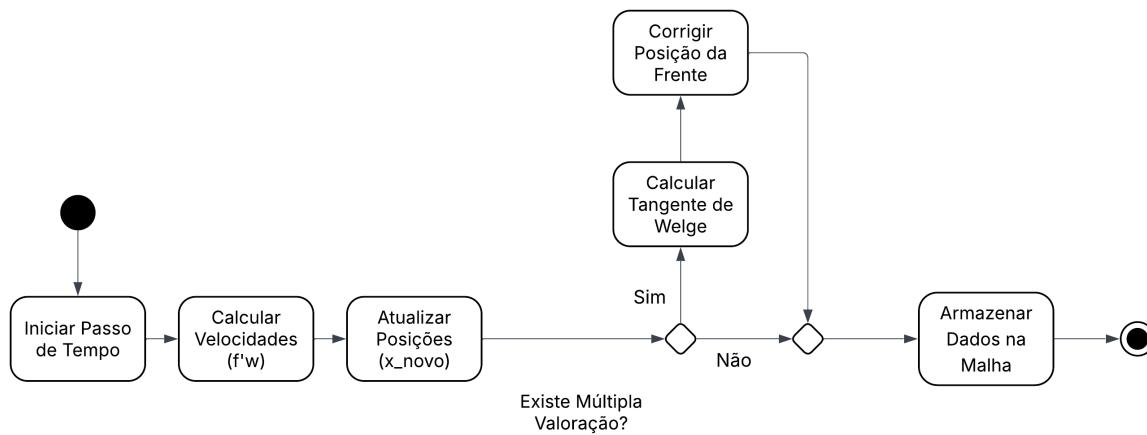


Figura 4.7: Diagrama de Atividades: Algoritmo de avanço temporal com correção de Welge

O algoritmo inicia-se com o cálculo das velocidades de onda para todas as células ativas. Após a atualização provisória das posições ($x_{new} = x_{old} + v\Delta t$), o sistema verifica a ocorrência de inconsistências físicas, especificamente o fenômeno de "múltipla valoração", onde uma saturação maior ultrapassaria uma menor devido à não-linearidade da curva de fluxo fracionário.

O losango de decisão ilustra o tratamento desta condição:

1. Fluxo Normal: Se o perfil de saturação permanece monotônico, os dados são armazenados diretamente.
2. Fluxo de Correção: Caso seja detectada a inversão física, o sistema desvia o fluxo para acionar a rotina de Welge (classe `CWelge`), que calcula a saturação da frente de choque (S_{wf}) e ajusta a posição das células para garantir a conservação de massa (Áreas Iguais), conforme fundamentado teoricamente no Capítulo 3.

Capítulo 5

Projeto

Neste capítulo associado ao projeto de engenharia de software, são apresentadas as decisões de alto nível e as especificações estruturais que transformam os requisitos conceituais em uma solução computacional executável. Inicialmente, estabelece-se o projeto do sistema por meio da definição de protocolos de comunicação, alocação de recursos computacionais, mecanismos de controle de fluxo e seleção detalhada das plataformas e bibliotecas tecnológicas. Posteriormente, detalha-se o projeto orientado a objeto, quantificando os impactos dessas decisões de engenharia sobre os modelos estruturais, dinâmicos, atributos, métodos, heranças e associações desenvolvidos na fase de análise. Por fim, a infraestrutura física e lógica do software é consolidada através dos diagramas de componentes e de implantação, culminando no planejamento de lançamentos (*features*) e na tabela de classificação do sistema.

5.1 Projeto do sistema

Depois da análise orientada a objeto desenvolve-se o projeto do sistema, o qual envolve etapas como a definição dos protocolos, da interface API (Interface de Programação de Aplicações), o uso de recursos, a subdivisão do sistema em subsistemas, a alocação dos subsistemas ao hardware e a seleção das estruturas de controle, a seleção das plataformas do sistema, das bibliotecas externas, dos padrões de projeto, além da tomada de decisões conceituais e políticas que formam a infraestrutura do projeto.

Deve-se definir padrões de documentação, padrões para o nome das classes, padrões de retorno e de parâmetros em métodos, características da interface do usuário e características de desempenho. O projeto do sistema constitui a estratégia de alto nível para resolver o problema de modelagem do fluxo fracionário e deslocamento bifásico imiscível.

1. Protocolos

- Definição dos protocolos de comunicação entre os diversos elementos externos (como dispositivos):
 - Neste projeto, por se tratar de um software científico do tipo *desktop* voltado para a execução local em estações de trabalho de engenharia, não há integração direta com sensores, nós de *clusters*, atuadores ou dispositivos de hardware externos via rede. Portanto, protocolos de comunicação de

rede externa não são aplicáveis a esta arquitetura, operando o sistema de forma isolada e autônoma.

- Definição dos protocolos de comunicação entre os diversos elementos internos (como objetos):
 - Neste projeto, a comunicação interobjeto ocorre de maneira síncrona através da invocação direta de métodos públicos e da passagem de referências e ponteiros na memória primária. Adicionalmente, a comunicação entre o motor matemático de cálculo e a interface gráfica é regida pelo protocolo de *Signals and Slots* (Sinais e *Slots*), nativo do *framework* adotado. Esse mecanismo garante o desacoplamento entre as classes de visualização e as classes de modelo físico, permitindo que eventos gerados na interface (como a alteração de um parâmetro) disparem atualizações automáticas nos subcomponentes de cálculo.
- Definição da interface API de suas bibliotecas e sistemas:
 - Neste projeto, a interface de programação de aplicação (API) interna é estruturada por meio de contratos rígidos de desenvolvimento baseados em classes abstratas e interfaces puramente virtuais. A API do subsistema rocha-fluido é exposta pela interface `ICurvasPermeabilidade`, que define a assinatura obrigatória para o cálculo das permeabilidades relativas. O motor matemático encapsula suas operações através da classe `CCalculadoraFluxoFracionario`, cuja API expõe métodos públicos de cálculo como `calcularFw(double sw)` e `calcularDerivadaFw(double sw)`. A execução macroscópica é concentrada na API do resolvidor (`CSolver`), através do método `calcularPerfilSaturacao(double tempoInjetado, double swi, double sw_max)`.
- Definição do formato dos arquivos gerados pelo software:
 - Neste projeto, o software utiliza dois formatos distintos para a entrada e saída de dados. Para a importação de dados externos associados ao modelo de permeabilidade relativa tabelada, o sistema faz a leitura de arquivos de texto plano com codificação ASCII (`.txt`), estruturados em colunas tabuladas para garantir legibilidade e portabilidade. Para a saída de dados, o sistema exporta relatórios executivos fechados no formato *Portable Document Format* (`.pdf`), gerados de forma nativa para documentar de maneira inalterável os parâmetros estáticos, os diagnósticos adimensionais e os gráficos gerados.

2. Recursos

- Identificação e alocação dos recursos globais, como os recursos do sistema serão alocados, utilizados, compartilhados e liberados:
 - Neste projeto, os recursos globais referem-se à memória RAM de curta duração e ao ciclo de processamento da CPU. A alocação ocorre dinamicamente durante a inicialização da janela principal, onde o simulador central e as calculadoras são instanciados. O compartilhamento de dados

brutos entre as rotinas de cálculo e os componentes de plotagem é otimizado através do uso de estruturas de dados eficientes do ecossistema de desenvolvimento (como `QVector`), evitando a cópia desnecessária de vetores em memória. A liberação dos recursos ocorre de forma controlada no destruidor da classe principal (`~MainWindow`), garantindo a desalocação completa de toda a memória dinâmica e prevenindo vazamentos (*memory leaks*).

- Identificação da necessidade do uso de banco de dados:
 - Neste projeto, devido à natureza estritamente analítica e matemática do problema de *Buckley-Leverett*, as simulações operam em tempo real com dados inteiramente residentes em memória. Os volumes de dados manipulados restringem-se a coeficientes empíricos e matrizes discretizadas de saturação de tamanho reduzido (geralmente matrizes de 100 a 1000 pontos). Sendo assim, identificou-se a total desnecessidade do acoplamento de sistemas de bancos de dados relacionais (como SQLite ou PostgreSQL) ou não-relacionais, simplificando a arquitetura e eliminando gargalos de escrita e leitura em disco. Versões futuras do simulador poderão considerar o uso de SQLite para o armazenamento de simulações padrões usadas por exemplo em sala de aula.
- Identificação da necessidade de sistemas de armazenamento de massa:
 - Neste projeto, não há necessidade de infraestruturas corporativas de armazenamento de massa, tais como servidores de arquivos dedicados, redes de armazenamento (SAN), sistemas de backup em nuvem em tempo real ou unidades de *storage* em rede. O armazenamento dos arquivos de texto de entrada e dos relatórios executivos em formato PDF gerados é delegado por completo ao sistema de arquivos local do computador do usuário, utilizando caminhos definidos dinamicamente por caixas de diálogo padrão do sistema operacional.

3. Controle

- Identificação e seleção da implementação de controle, sequencial ou concorrente, baseado em procedimentos ou eventos:
 - Neste projeto, adota-se um modelo de controle híbrido perfeitamente segmentado por responsabilidade de camadas. O fluxo de controle da interface gráfica é totalmente orientado a eventos, permanecendo em estado de espera (*event loop*) até que ações do usuário disparem os gatilhos lógicos. Em contrapartida, o fluxo de controle interno do motor físico de simulação e cálculo da tangente de Welge é estritamente sequencial e baseado em procedimentos, executando de forma determinística as equações matemáticas do balanço de massa sem ramificações assíncronas concorrentes durante o processamento de um mesmo cenário.
- Identificação das condições extremas e de prioridades:

- Neste projeto, as condições extremas foram mapeadas para evitar falhas de execução e colapsos numéricos. Elas englobam: razões de viscosidade excessivamente desfavoráveis (onde a cauda da derivada do fluxo fracionário aproxima-se assintoticamente de zero), divisões por zero em tempos avançados de injeção pós-ruptura ($1/t_D$ com valores infinitésimos) e valores nulos de porosidade ou área. A prioridade máxima do sistema foi dada ao tratamento preventivo dessas exceções diretamente no motor de cálculo, validando os dados na função `sincronizarDadosComSimulador` antes do início do pipeline matemático.
- Identificação da necessidade de otimização:
 - Neste projeto, a necessidade de otimização concentrou-se na estabilidade numérica da curva assintótica *pós-breakthrough*. Devido ao comportamento plano da curva de fluxo fracionário próximo à saturação máxima ($1 - S_{or}$), pequenas imprecisões no cálculo da derivada geravam ruídos que travavam a evolução gráfica. A otimização foi realizada substituindo as derivadas analíticas simples por uma formulação paramétrica de diferenças finitas centrais de alta precisão com passo refinado ($h = 10^{-5}$), eliminando a necessidade de algoritmos iterativos de busca e reduzindo o tempo de resposta computacional para a escala de microssegundos.
- Identificação e definição de *loops* de controle e das escalas de tempo:
 - Neste projeto, os laços de controle executam *loops* definidos de varredura paramétrica de saturação (incrementos fixos de $\Delta S_w = 0.001$), partindo da saturação inicial até a saturação máxima de água. A escala de tempo física do fenômeno simulado é adimensionalizada em Volumes de Poros Injetados (PVI), cobrindo uma escala operacional típica de 0.0 a 3.0 PVI. Na escala de tempo computacional, o processamento analítico ocorre de forma instantânea em tempo de CPU inferior a 10 milissegundos, garantindo respostas em tempo real para o usuário.
- Identificação de concorrências – quais algoritmos podem ser implementados usando processamento paralelo:
 - Neste projeto, por se tratar de uma solução analítica pelo Método das Características (MOC) e não de um simulador numérico tridimensional baseado em malhas de blocos por diferenças finitas, o esforço computacional é extremamente baixo. Os *loops* de discretização gráfica operam em vetores pequenos (101 pontos para fluxo fracionário e cerca de 1000 pontos para os perfis). Dessa forma, determinou-se que não há justificativa técnica para a implementação de concorrência ou processamento paralelo (como OpenMP ou *threads*), mantendo a execução em *Thread* única (*single-thread*) de alto desempenho e livre de condições de corrida.

4. Plataformas

- Identificação das estruturas arquitetônicas comuns:

- Neste projeto, a estrutura arquitetônica comum adotada fundamenta-se no padrão arquitetural MVC (*Model-View-Controller*) simplificado, onde a camada de visualização (`MainWindow` e formulários UI) interage com o controlador central (`CSimulador`), que gerencia o fluxo de dados em direção ao modelo de domínio matemático (`CSolver`, `CWelge`, `CCalculadoraFluxo-Fracionario`).
- Identificação de subsistemas relacionados à plataforma selecionada:
 - Neste projeto, os subsistemas estão intimamente ligados à API de classes do *framework* multiplataforma adotado. Destacam-se o subsistema de janelas e componentes gráficos de entrada (`QtWidgets`), o subsistema de renderização de relatórios baseado no motor de renderização de texto enriquecido (`QTextDocument`) e o subsistema de paginação e impressão vetorial para arquivos digitais (`QPdfWriter`).
- Identificação e definição das plataformas a serem suportadas: hardware, sistema operacional e linguagem de software:
 - Neste projeto, a plataforma de software suportada baseia-se na linguagem C++ nativa (padrão ISO/IEC C++11 ou superior), garantindo compilação eficiente. O sistema operacional alvo prioritário é o Microsoft Windows (7, 10 e 11), mas a escolha tecnológica confere portabilidade total para os sistemas Linux e macOS. Em termos de hardware, o software é extremamente leve, exigindo apenas uma estação de trabalho padrão de engenharia com processador x86 ou x64 e um mínimo de 2 GB de memória RAM livre.
- Seleção das bibliotecas externas a serem utilizadas:
 - Neste projeto, a principal biblioteca externa selecionada foi a `QCustomPlot`, um componente de plotagem científica de alto desempenho desenvolvido especificamente para o ecossistema Qt. Ela foi escolhida por sua capacidade de renderizar gráficos bidimensionais complexos de forma nativa, permitindo interações em tempo real (como zoom e redimensionamento automático) e a exportação direta de suas telas em formato vetorial de imagem (`Pixmap`), recurso essencial para a alimentação automática do nosso módulo de relatórios. Maiores detalhes da biblioteca acesse <https://www.qcustomplot.com/>,
- Seleção da biblioteca utilizada para montar a interface gráfica do software – GDI:
 - Neste projeto, a biblioteca de interface gráfica (*Graphics Device Interface* / GUI) selecionada foi o módulo Qt Widgets corporativo. Afastou-se o uso de APIs nativas de baixo nível do sistema operacional (como a GDI clássica do Windows) para garantir que toda a renderização visual, folhas de estilo customizadas (QSS para cantos arredondados, fontes personalizadas Segoe UI e modos claro/escuro) e gerenciadores de layout (layouts responsivos) operem de forma agnóstica e integrada à plataforma. A biblioteca Qt é multiplataforma, funciona nos três principais sistemas operacionais, a saber, Windows, GNU/Linux e Mac OS X.

- Seleção do ambiente de desenvolvimento para montar a interface de desenvolvimento – IDE:
 - Neste projeto, o ambiente integrado de desenvolvimento adotado foi a IDE oficial Qt Creator em sua versão estável. A ferramenta foi selecionada por centralizar de forma unificada o editor de código C++, o compilador GCC/MinGW, o gerenciador de automação de compilação `qmake`, o depurador (*debugger*) e a ferramenta integrada de design de telas Qt Designer (responsável pela edição visual do arquivo `mainwindow.ui`). Maiores detalhes acesse <https://www.qt.io/development/tools/qt-creator-ide>

5. Padrões de projeto

- Identificação dos padrões de projeto incorporados:
 - Neste projeto, o padrão de projeto comportamental *Strategy* (Estratégia) foi implementado de forma rigorosa como pilar de flexibilidade do sistema. A interface pura `ICurvasPermeabilidade` atua como a estratégia abstrata. As classes `CCurvasPermeabilidadeCorey`, `CCurvasPermeabilidade-LET`, `CCurvasPermeabilidadeChierici` e `CCurvasPermeabilidadeTabelada` atuam como as estratégias concretas que encapsulam os diferentes modelos empíricos rocha-fluido da engenharia de reservatórios. O simulador atua como o contexto, alterando dinamicamente o algoritmo de permeabilidade em tempo de execução com base na escolha do usuário na interface, sem que o motor de cálculo precise conhecer os detalhes de implementação de cada modelo matemático.
 - O padrão arquitetural MVC definiu a estrutura inicial do projeto. Uma descrição do MVC é disponibilizada no site [padrão MVC](#).

5.2 Projeto orientado a objeto – POO

O projeto orientado a objeto é a etapa posterior ao projeto do sistema. Baseia-se na análise, mas considera as decisões do projeto do sistema. Acrescenta a análise desenvolvida e as características da plataforma escolhida (hardware, sistema operacional e linguagem de programação de software). Passa pelo maior detalhamento do funcionamento do software, acrescentando atributos e métodos que envolvem a solução de problemas específicos não identificados durante a fase puramente conceitual.

Envolve a otimização da estrutura de dados e dos algoritmos, a minimização do tempo de execução, da alocação de memória e de custos computacionais. Existe um desvio de ênfase para os conceitos da plataforma selecionada (C++11 e *framework* Qt).

Efeitos do projeto no modelo estrutural

- Adicionar nos diagramas de pacotes as bibliotecas e subsistemas selecionados no projeto do sistema:
 - Neste projeto, o modelo estrutural foi expandido para acomodar os pacotes externos do *framework* Qt. Foram incluídos os pacotes lógicos `<QtWidgets>`

(para a interface do utilizador), `<QTextDocument>` e `<QPdfWriter>` (para o subsistema de relatórios nativos), bem como a biblioteca gráfica de terceiros `QCustomPlot`, que passou a atuar como um pacote utilitário de renderização bidimensional.

- Novas classes e associações oriundas das bibliotecas selecionadas e da linguagem escolhida devem ser acrescentadas ao modelo:
 - Neste projeto, a classe de geração de relatórios (`CRelatorio`) foi adaptada para manipular objetos visuais diretamente através da inclusão da classe utilitária `QPixmap`, convertendo gráficos em fluxos de bytes (`QByteArray` e `QBuffer`) codificados em Base64 para injeção direta no HTML do relatório.
- Estabelecer as dependências e restrições associadas à plataforma escolhida:
 - Neste projeto, estabeleceu-se a restrição de separação estrita (desacoplamento) entre o domínio (lógica física) e a interface. As classes do núcleo matemático (`CSolver`, `CWelge`) não possuem qualquer dependência das bibliotecas gráficas do Qt, garantindo que o motor de cálculo permaneça puramente em C++ padrão (*std*), facilitando manutenções ou migrações futuras.

Efeitos do projeto no modelo dinâmico

- Revisar os diagramas de sequência e de comunicação considerando a plataforma escolhida:
 - Neste projeto, os diagramas de sequência foram atualizados para refletir o paradigma de comunicação do Qt (*Signals and Slots*). As chamadas de cálculo não ocorrem de forma espontânea; o diagrama dinâmico passa a exibir eventos de gatilho, como `on_btnPlotarSolucao_clicked()`, que invocam métodos de sincronização (`sincronizarDadosComSimulador()`) antes de despoletar a sequência de resolução numérica.
- Verificar a necessidade de se revisar, ampliar e adicionar novos diagramas de máquinas de estado e de atividades:
 - Neste projeto, as máquinas de estado da interface gráfica foram refinadas. Foi adicionado um controlo de estado booleano (`isDarkMode`) à classe `MainWindow`, que gere a transição visual das folhas de estilo (QSS) dinâmicas, alternando paletas de cores e ícones (recursos `.qrc`) em tempo de execução sem necessitar da reinicialização do software.

Efeitos do projeto nos atributos

- Atributos novos podem ser adicionados a uma classe, como atributos específicos de uma determinada linguagem de programação:

- Neste projeto, os vetores matemáticos concebidos na análise foram traduzidos para estruturas otimizadas da linguagem C++. Utilizou-se o `QVector<double>` para armazenamento contíguo de coordenadas de plotagem (posições adimensionais e tempos), e o iterador `std::map<double, double>` para garantir a ordenação automática e a eliminação de dados sobrepostos na construção da curva de eficiência de deslocamento (E_d). Constantes de precisão (`SWI_EPS`) foram explicitamente declaradas.

Efeitos do projeto nos métodos

- Em função da plataforma escolhida, verifique as possíveis alterações nos métodos:
 - Neste projeto, os métodos de acesso a ficheiros foram completamente alterados para utilizar as caixas de diálogo nativas do sistema operativo através da classe `QFileDialog`. A exportação em formato PDF encapsulou a necessidade de integração entre a lógica de layout de página (`QPageLayout`) e as métricas físicas dentro do método `exportarParaPDF()`.
- Divisão de métodos (Tomada de decisões/controlo vs. Processamento otimizado):
 - Neste projeto, implementou-se uma clara bifurcação. Métodos de decisão e controlo operam na classe `MainWindow`, sendo responsáveis por validar entradas e lidar com exceções (`try...catch`). Em contrapartida, os métodos de processamento, como `calcularTangente` na classe `CWelge`, evitam o polimorfismo excessivo nos seus laços (*loops*) de cálculo pesado para não comprometer o desempenho da varredura, focando na computação pura de diferenças finitas para derivadas pontuais de alta precisão.
- Algoritmos complexos podem ser subdivididos e otimizados utilizando STL:
 - Neste projeto, o algoritmo clássico de identificação da velocidade da frente de choque foi subdividido para ancorar rigorosamente o ponto inicial da secante na saturação de água irreduzível (S_{wi}), utilizando bibliotecas matemáticas padrão (`<algorithm>` e `<cmath>`) como o `std::max` para filtros lógicos de monotonicidade física (garantindo que a eficiência E_d nunca regrida ao longo do tempo).
- Os métodos das classes estão a dar resposta às responsabilidades da classe?
 - Neste projeto, reviu-se o Princípio da Responsabilidade Única (SRP:). Asseverou-se que a classe da janela principal não realiza cálculos físicos, delegando toda a montagem do problema transiente à classe `CSolver` e os cálculos da descon-tinuidade hiperbólica à classe `CWelge`.

Efeitos do projeto nas heranças

- Reorganização das classes e dos métodos:

- Neste projeto, consolidou-se a hierarquia baseada em classes abstratas. Os métodos de cálculo do fluxo fracionário (f_w) foram generalizados para trabalhar de forma abstrata com ponteiros, permitindo que a injeção do modelo de permeabilidade escolhido decorresse em tempo de execução, sem necessidade de duplicação de métodos genéricos para cenários bifásicos distintos.
- Abstração do comportamento comum:
 - Neste projeto, a interface `ICurvasPermeabilidade` garantiu que todas as classes concretas de modelos (Corey, LET, Chierici e Tabelada) assumissem o contrato de fornecimento obrigatório das permeabilidades relativas (k_{rw} e k_{ro}), garantindo que as expansões futuras da física do reservatório herdem obrigatoriamente essa mesma estrutura formal.
- Utilização de delegação para partilhar a implementação:
 - Neste projeto, evitou-se a herança profunda e irrealista através da delegação (composição). A classe `CSimulador` não herda do `CSolver` ou do `CWelge`; ela possui ponteiros para essas instâncias delegando-lhes as chamadas numéricas, o que reduziu drasticamente o acoplamento rígido do código-fonte.

Efeitos do projeto nas associações

- Definição de como as associações serão implementadas:
 - Neste projeto, as associações fortes de composição (tempo de vida dependente) foram implementadas com alocação dinâmica de ponteiros (*new*) na instanciamento do simulador e libertadas (*delete*) no destrutor, de forma a garantir a segurança e a conformidade da gestão manual de memória exigida pela linguagem C++.
- Associações diretas (evitar percorrer vários nós de classes distantes):
 - Neste projeto, evitou-se a transgressão da Lei de Demeter (ex: `Janela->Simulador->Welge->Calculadora->derivada()`). A passagem de condições de contorno cruciais (como saturação inicial e máxima) foi refatorada para ser transferida explicitamente através das assinaturas dos métodos (e.g., `_solver->calcularPerfilSaturacao(swi, sw_max)`), estabelecendo associações de fluxo de dados diretas e extremamente legíveis, eliminando desvios longos e instáveis de arquitetura.

Efeitos do projeto nas otimizações

- Análise de aspetos relativos à otimização do sistema:
 - Neste projeto, a principal otimização foi realizada na zona pós-*breakthrough* da curva de recuperação. Como o Método das Características exige encontrar a saturação cuja derivada direcional seja igual ao inverso do tempo ($f'_w(S_w) = 1/t_D$), os métodos baseados em procura de raízes causavam colapso numérico

(ruído) quando a derivada tendia a zero. A otimização arquitetável inverteu a lógica, varrendo parametricamente a saturação espacial para deduzir diretamente o tempo de saída exato.

- Inclusão de bibliotecas otimizadas e atributos auxiliares:
 - Neste projeto, foram criados atributos auxiliares calculados prematuramente em memória (como o `max_ed` gerado num mapa de estados de eficiência temporal), descartando ciclos computacionais repetitivos que desenhariam "quebras" irreais (degraus) no gráfico da eficiência de deslocamento aquando do cálculo da transição abrupta da frente de choque. A ordem de execução garante sempre que a extração em formato imagem (`QPixmap`) só decorra após as rotinas de atualização visual de eixos (`QCustomPlot`) estarem plenamente concluídas e escaladas.

5.3 Diagrama de componentes

O diagrama de componentes mostra a forma como os componentes do software se relacionam e suas dependências arquiteturais estruturais. Inclui itens como bibliotecas estáticas, bibliotecas dinâmicas (DLLs), executáveis, arquivos de disco e pacotes de código-fonte.

Neste projeto, o diagrama de componentes reflete a arquitetura modular e multiplataforma do *framework* Qt aliada ao núcleo matemático nativo em C++. O sistema é composto pelos seguintes agrupamentos de componentes lógicos e físicos:

- **Executável Principal (`Simulador.exe`):** Componente central gerado pela compilação, que engloba a lógica de negócio, as instâncias da interface gráfica (`MainWindow`) e as rotinas do controlador (`CSimulador`). Ele atua como o maestro do fluxo de execução.
- **Subsistema de Interface e Gráficos:** Depende diretamente das bibliotecas dinâmicas do Qt (como `Qt5Core.dll`, `Qt5Gui.dll` e `Qt5Widgets.dll` no windows). Para a renderização visual das curvas, incorpora estaticamente a biblioteca de terceiros `QCustomPlot`, que atua como um componente interno para transformar matrizes de dados em gráficos bidimensionais interativos.
- **Subsistema de Geração de Relatórios:** Depende das bibliotecas nativas de impressão e formatação de texto do Qt (`Qt5PrintSupport.dll`), sendo responsável por injetar os fluxos de bytes das imagens (`Pixmaps`) capturadas da camada gráfica em um documento formatado em HTML e, posteriormente, convertido em PDF.
- **Subsistema Numérico e Lógico:** Composto inteiramente por classes padrão da linguagem C++ (`CSolver`, `CWedge`, `CCalculadoraFluxoFracionario`), garantindo independência de bibliotecas externas para a execução da matemática do escoamento.

- **Arquivos de Entrada e Saída (I/O):** O sistema possui dependências de arquivos de disco plano do tipo `.txt` para a importação opcional de parâmetros de permeabilidade relativa tabelada, e gera de forma autônoma documentos executivos de saída do tipo `.pdf`.

Esta divisão clara garante que alterações nas regras matemáticas do escoamento bifásico não exijam a recompilação ou substituição de componentes visuais, garantindo a manutenibilidade do código.

Veja na Figura 5.1 o diagrama de componentes:

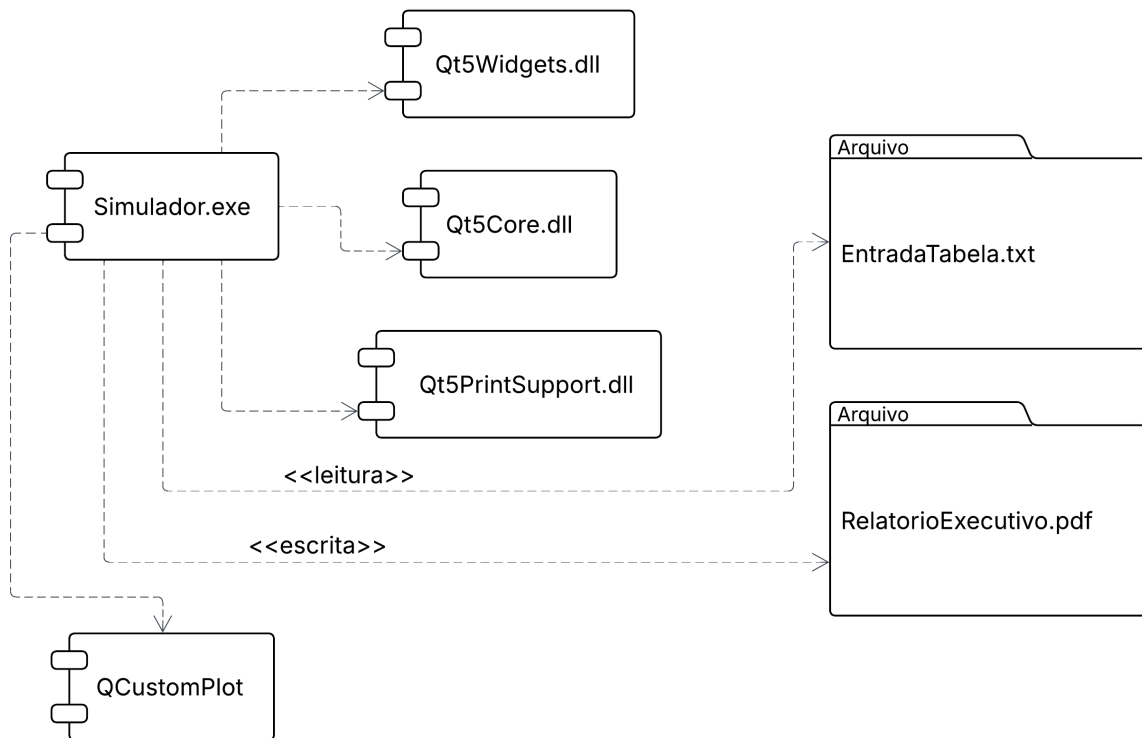


Figura 5.1: Diagrama de componentes

5.4 Diagrama de implantação

O diagrama de implantação é um diagrama de alto nível que ilustra as relações entre o sistema lógico e o hardware, focando nos aspectos da arquitetura computacional escolhida e na configuração dos nós em tempo de execução.

Neste projeto, devido à opção metodológica por uma ferramenta de natureza analítica e resposta em tempo real, dispensou-se a infraestrutura de rede complexa. O diagrama de implantação concentra-se em um único nó principal de execução (*Node*), caracterizado como a Estação de Trabalho do Engenheiro (*Desktop*).

- **Nó de Execução (*Workstation*):** Representa o hardware do usuário (processador x86/x64, memória RAM primária). Neste nó reside o Sistema Operacional hospedeiro (Windows, Linux ou macOS), o qual fornece a plataforma de execução para o software.

- **Artefatos Implantados:** Dentro deste nó, o sistema é implantado contendo o pacote executável principal, ladeado pelos componentes dinâmicos do *framework* Qt (.dlls essenciais) encapsulados no mesmo diretório local para evitar problemas de dependência ou instalação de ferramentas de terceiros no computador do cliente.
- **Dispositivos de I/O Temporários:** O sistema interage diretamente com os discos de armazenamento rígido locais (HDD/SSD) exclusivamente durante a leitura dos modelos físicos (.txt) e na gravação do relatório de diagnóstico (.pdf), operando isoladamente da internet ou de servidores de banco de dados locais.

Veja na Figura ??, ?? o de diagrama de implantação.

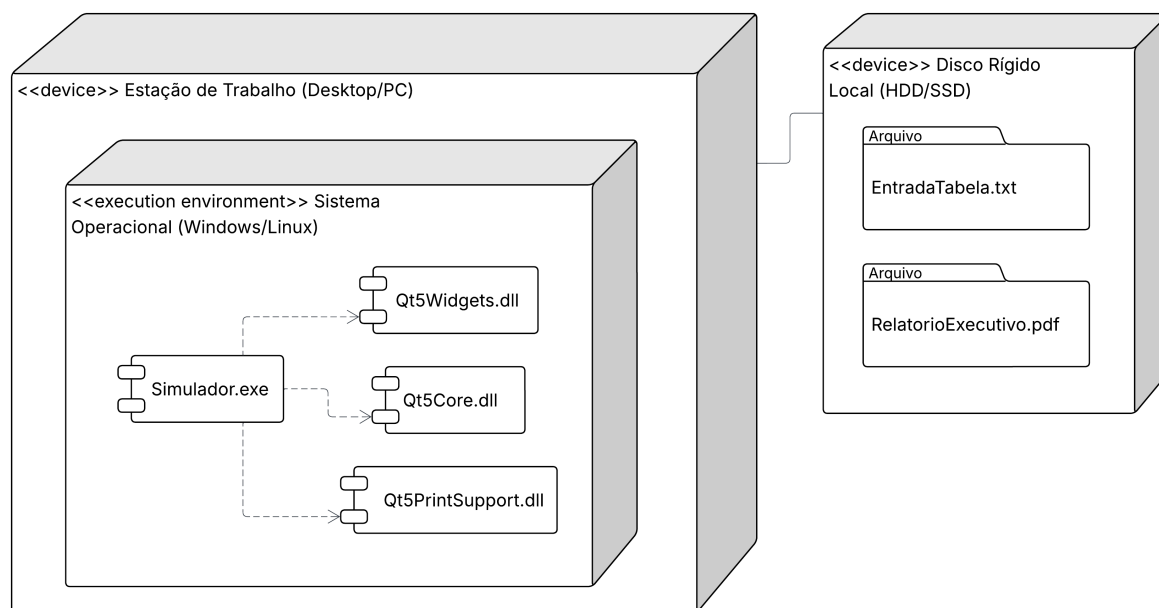


Figura 5.2: Diagrama de implantação

5.4.1 Lista de características <<features>>

No final do ciclo de concepção e análise, consolidou-se uma lista iterativa de características (*features*) fundamentais para a estabilidade e usabilidade da ferramenta, implementadas através de ciclos curtos de desenvolvimento e validação analítica:

- v2.1 - Fundação Estrutural e Interatividade Básica
 - Construção da interface principal com o *framework* Qt *Widgets*.
 - Implementação da calculadora geométrica e modelos de permeabilidade relativa (Padrão *Strategy*).
 - Testes lógicos: Validação da plotagem contínua da Curva de Fluxo Fracionário (\$f_w\$).
- v2.1 - Motor Matemático e Física do Deslocamento

- Implementação da lógica sequencial do Método das Características (MOC) e malha discreta.
 - Cálculo algorítmico da derivada pontual e construção da tangente geométrica de Welge.
 - Testes de Estabilidade: Prevenção de múltiplos valores de saturação (Frente de Choque).
- v2.3 - Ferramenta Executiva e Otimização Visual
 - Integração do subsistema dinâmico de temas UI (Modo Escuro / Claro com QSS).
 - Implementação do módulo avançado de exportação vetorial de gráficos e relatórios técnicos (PDF).
 - Testes Finais: Validação da continuidade assintótica da Curva de Eficiência de Deslocamento (E_d) pós-breakthrough.

5.4.2 Tabela classificação sistema

A Tabela a seguir classifica a natureza estrutural e o domínio de aplicação do sistema de engenharia desenvolvido:

Tabela 5.1: Tabela classificação sistema

Licença:	☒ livre GPL-v3 ☐ proprietária
Engenharia de software:	☐ tradicional ☒ ágil ☐ outras
Paradigma de programação:	☐ estruturada ☒ orientado a objeto - POO ☐ funcional
Modelagem UML:	☒ básica ☒ intermediária ☐ avançada
Algoritmos:	☒ alto nível ☒ baixo nível
	implementação: ☐ recursivo ou ☒ iterativo; ☒ determinístico ou ☐ não-determinístico; ☐ exato ou ☒ aproximado
	concorrências: ☒ serial ☐ concorrente ☐ paralelo
	paradigma: ☒ dividir para conquistar ☐ programação linear ☐ transformação/ redução ☐ busca e enumeração ☐ heurístico e probabilístico ☐ baseados em pilhas
Software:	☐ de base ☒ aplicados ☐ de cunho geral ☒ específicos para determinada área ☒ educativo ☒ científico
	instruções: ☒ alto nível ☐ baixo nível
	otimização: ☐ serial não otimizado ☒ serial otimizado ☐ concorrente ☐ paralelo ☐ vetorial
	interface do usuário: ☐ kernel numérico ☐ linha de comando ☐ modo texto ☒ híbrida (texto e saídas gráficas) ☐ modo gráfico (ex: Qt) ☐ navegador
Recursos de C++:	☒ C++ básico (FCC): variáveis padrões da linguagem, estruturas de controle e repetição, estruturas de dados, struct, classes(objetos, atributos, métodos), funções; entrada e saída de dados (<i>streams</i>), funções de cmath
	☒ C++ intermediário: funções lambda. Ponteiros, referências, herança, herança múltipla, polimorfismo, sobrecarga de funções e de operadores, tipos genéricos (templates), <i>smarth pointers</i> . Diretrizes de pré-processador, classes de armazenamento e modificadores de acesso. Estruturas de dados: enum, uniões. Bibliotecas: entrada e saída acesso com arquivos de disco, redirecionamento. Bibliotecas: <i>filesystem</i>
	☒ C++ intermediário 2: A biblioteca de gabaritos de C++ (a STL), containers, iteradores, objetos funções e funções genéricas. Noções de processamento paralelo (múltiplas threads, uso de <i>thread</i> , <i>join</i> e <i>mutex</i>). Bibliotecas: <i>random</i> , <i>threads</i>
	☐ C++ avançado: Conversão de tipos do usuário, especializações de templates, excessões. Cluster de computadores, processamento paralelo e concorrente, múltiplos processos (pipes, memória compartilhada, sinais). Bibliotecas: <i>expressões regulares</i> , <i>múltiplos processos</i>
Bibliotecas de C++:	☒ Entrada e saída de dados (<i>streams</i>) ☒ cmath ☒ <i>filesystem</i> ☐ <i>random</i> ☐ <i>threads</i> ☐ <i>expressões regulares</i> ☐ <i>múltiplos processos</i>
Bibliotecas externas:	☐ CGnuplot ☒ QCustomPlot ☐ Qt diálogos ☐ QT Janelas/menus/BT
Ferramentas auxiliares:	Montador: ☒ make ☐ cmake ☒ qmake
IDE:	☐ Editor simples: kate/gedit/emacs ☐ kdevelop ☒ QT-Creator ☐
SCV:	☐ cvs ☐ svn ☒ git
Disciplinas correlacionadas	☐ estatística ☒ cálculo numérico ☒ modelamento numérico ☐ análise e processamento de imagens

Capítulo 6

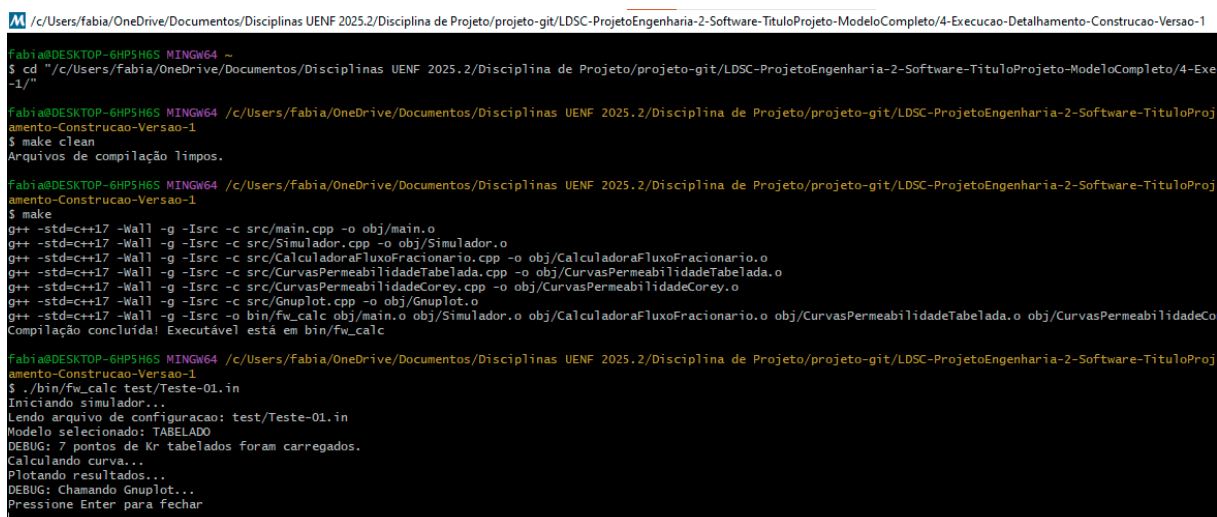
Ciclos de Planejamento/Detalhamento

Apresenta-se neste capítulo os ciclos de planejamento/detalhamento para as diferentes versões desenvolvidas.

6.1 Versão 1.0 - Prototipagem do Núcleo Analítico

Na primeira versão, Figura 6.1, o foco foi a validação do núcleo matemático fundamental. O software foi concebido como uma ferramenta de linha de comando (CLI - *Command Line Interface*) simples, voltada para a plotagem automatizada da curva de fluxo fracionário (f_w).

- Escopo: Implementação dos modelos de permeabilidade relativa de Corey e Tabulador.
- Funcionalidades: Execução via arquivo de configuração (.in), leitura de dados tabulares de permeabilidade e integração externa com o *Gnuplot* para a geração de gráficos.
- Resultados: Esta versão provou a viabilidade da solução analítica, permitindo a validação dos cálculos matemáticos contra os dados experimentais carregados, servindo como o alicerce fundamental para a transição para um ambiente gráfico.



```

/c/Users/fabia/OneDrive/Documents/Disiplinas UENF 2025.2/Disiplina de Projeto/projeto-git/LDSC-ProjetoEngenharia-2-Software-TituloProjeto-ModeloCompleto/4-Execucao-Detalhamento-Construcao-Versao-1
Fabia@DESKTOP-6HP5H6S MINGW64 ~
$ cd "/c/Users/fabia/OneDrive/Documents/Disiplinas UENF 2025.2/Disiplina de Projeto/projeto-git/LDSC-ProjetoEngenharia-2-Software-TituloProjeto-ModeloCompleto/4-Execucao-Detalhamento-Construcao-Versao-1/"
Fabia@DESKTOP-6HP5H6S MINGW64 /c/Users/fabia/OneDrive/Documents/Disiplinas UENF 2025.2/Disiplina de Projeto/projeto-git/LDSC-ProjetoEngenharia-2-Software-TituloProjeto-ModeloCompleto/4-Execucao-Detalhamento-Construcao-Versao-1
$ make clean
Arquivos de compilaco limpos.

Fabia@DESKTOP-6HP5H6S MINGW64 /c/Users/fabia/OneDrive/Documents/Disiplinas UENF 2025.2/Disiplina de Projeto/projeto-git/LDSC-ProjetoEngenharia-2-Software-TituloProjeto-ModeloCompleto/4-Execucao-Detalhamento-Construcao-Versao-1
$ make
g++ -std=c++17 -Wall -g -Isrc -c src/main.cpp -o obj/main.o
g++ -std=c++17 -Wall -g -Isrc -c src/Simulador.cpp -o obj/Simulador.o
g++ -std=c++17 -Wall -g -Isrc -c src/CalculadoraFluxoFracionario.cpp -o obj/CalculadoraFluxoFracionario.o
g++ -std=c++17 -Wall -g -Isrc -c src/CurvasPermeabilidadeTabelada.cpp -o obj/CurvasPermeabilidadeTabelada.o
g++ -std=c++17 -Wall -g -Isrc -c src/CurvasPermeabilidadeCorey.cpp -o obj/CurvasPermeabilidadeCorey.o
g++ -std=c++17 -Wall -g -Isrc -c src/Gnuplot.cpp -o obj/Gnuplot.o
g++ -std=c++17 -Wall -g -Isrc -o bin/fw_calc obj/main.o obj/Simulador.o obj/CalculadoraFluxoFracionario.o obj/CurvasPermeabilidadeTabelada.o obj/CurvasPermeabilidadeCorey.o
Compilaco concluída! Executável está em bin/fw_calc

Fabia@DESKTOP-6HP5H6S MINGW64 /c/Users/fabia/OneDrive/Documents/Disiplinas UENF 2025.2/Disiplina de Projeto/projeto-git/LDSC-ProjetoEngenharia-2-Software-TituloProjeto-ModeloCompleto/4-Execucao-Detalhamento-Construcao-Versao-1
$ ./bin/fw_calc test/Teste-01.in
Iniciando simulador...
Lendo arquivo de configuracao: test/Teste-01.in
Modelo selecionado: TABELADO
DEBUG: 7 pontos de Kr tabelados foram carregados.
Calculando curva...
Plotando resultados...
DEBUG: Chamando Gnuplot...
Pressione Enter para fechar

```

Figura 6.1: Versão 1.0, imagem do programa rodando

6.2 Versão 2.1 - Migração para Interface Gráfica e Unificação

Nesta segunda versão, Figura 6.2, ocorreu a migração do paradigma CLI para uma aplicação *Desktop* robusta, utilizando o *framework* Qt. O objetivo central foi a unificação dos modelos de cálculo sob uma única interface visual e a eliminação da dependência de renderizadores externos (*Gnuplot*).

- Escopo: Criação da estrutura de classes `MainWindow` e integração dos modelos LET e Chierici.
- Funcionalidades: Implementação do motor de simulação dinâmica, dos gráficos interativos via `QCustomPlot` e dos campos de entrada de dados escalares (reservatório e fluidos).
- Resultados: O sistema tornou-se uma ferramenta de engenharia completa, onde o usuário passou a interagir com os dados em tempo real. Apesar da funcionalidade robusta, a interface ainda mantinha o padrão estético básico dos componentes do sistema operacional (*default*).

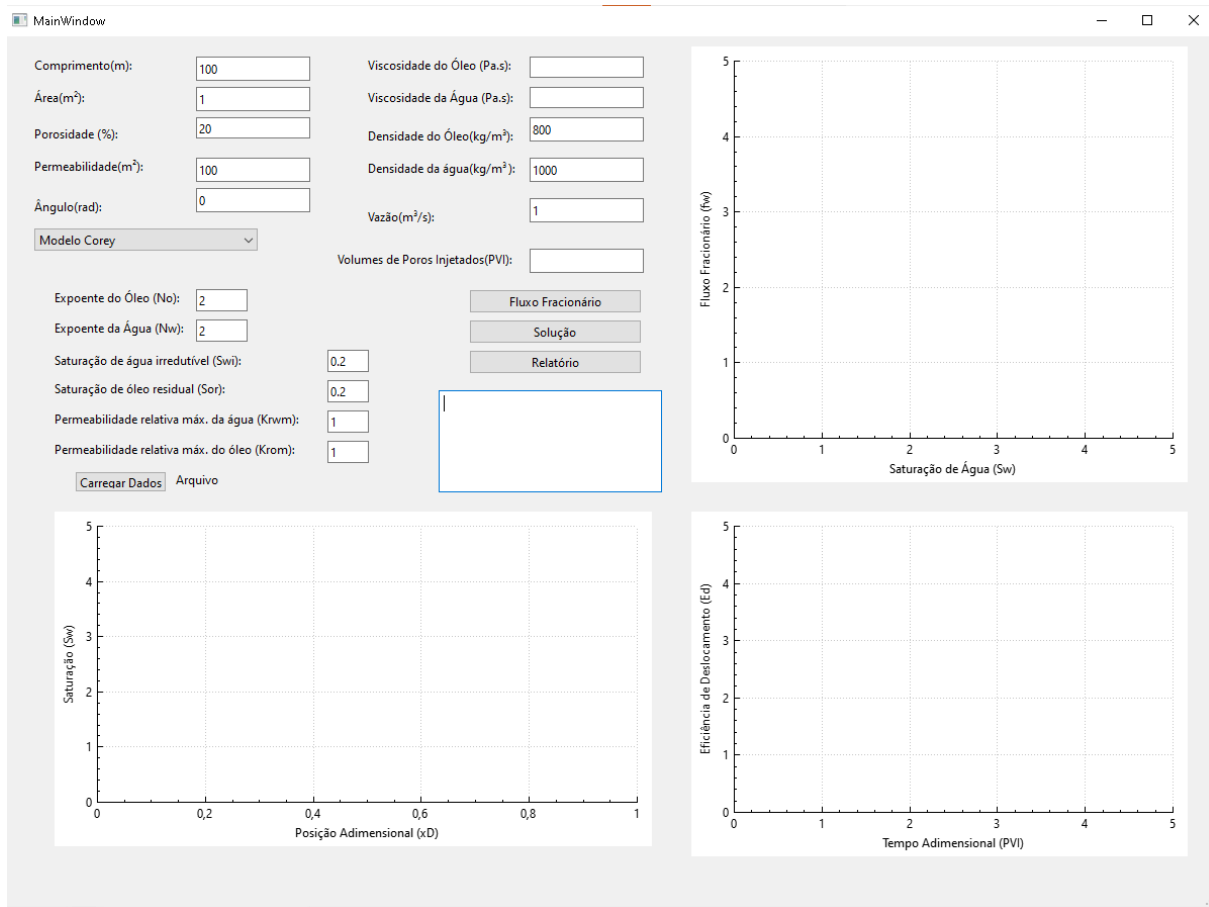


Figura 6.2: Versão 2.1, imagem do programa rodando

6.3 Versão 2.3 - Refinamento de Engenharia e Experiência do Usuário (UX)

Nesta terceira versão, Figura 6.3, o desenvolvimento foi voltado para o refinamento da experiência do utilizador e a formalização do projeto como um produto educativo e profissional. A interface foi submetida a um processo de redesenho completo, focando na ergonomia visual e na acessibilidade.

- Escopo: Padronização estética, otimização da usabilidade e implementação de recursos profissionais de relatório técnico.
- Funcionalidades: Implementação do sistema dinâmico de temas (Modo Claro/Esuro), padronização tipográfica (Segoe UI), arredondamento ergonômico de componentes, ícones vetoriais em tempo de execução e a funcionalidade de exportação de diagnósticos físicos para PDF, com embutimento automático dos gráficos simulados.
- Resultados: O sistema atingiu um nível de acabamento profissional, com alta conformidade às boas práticas de UI/UX para engenharia. A versão atual consolidou o simulador como uma ferramenta educativa completa, unindo estabilidade numérica

(solução do *breakthrough* e tangente de Welge) a uma interface intuitiva e adaptável às condições de iluminação do ambiente de trabalho.

Para garantir a coerência visual e a alternância intuitiva entre os modos de iluminação (Claro e Escuro), foram desenvolvidos recursos gráficos exclusivos (ícones vetoriais) utilizando a plataforma Canva, conforme ilustrado nas Figura 6.4 e 6.5. Estes ícones foram embutidos diretamente no executável final através do sistema de recursos do Qt (.qrc). Maiores detalhes da plataforma Canva podem ser obtidos no site https://www.canva.com/pt_br/.

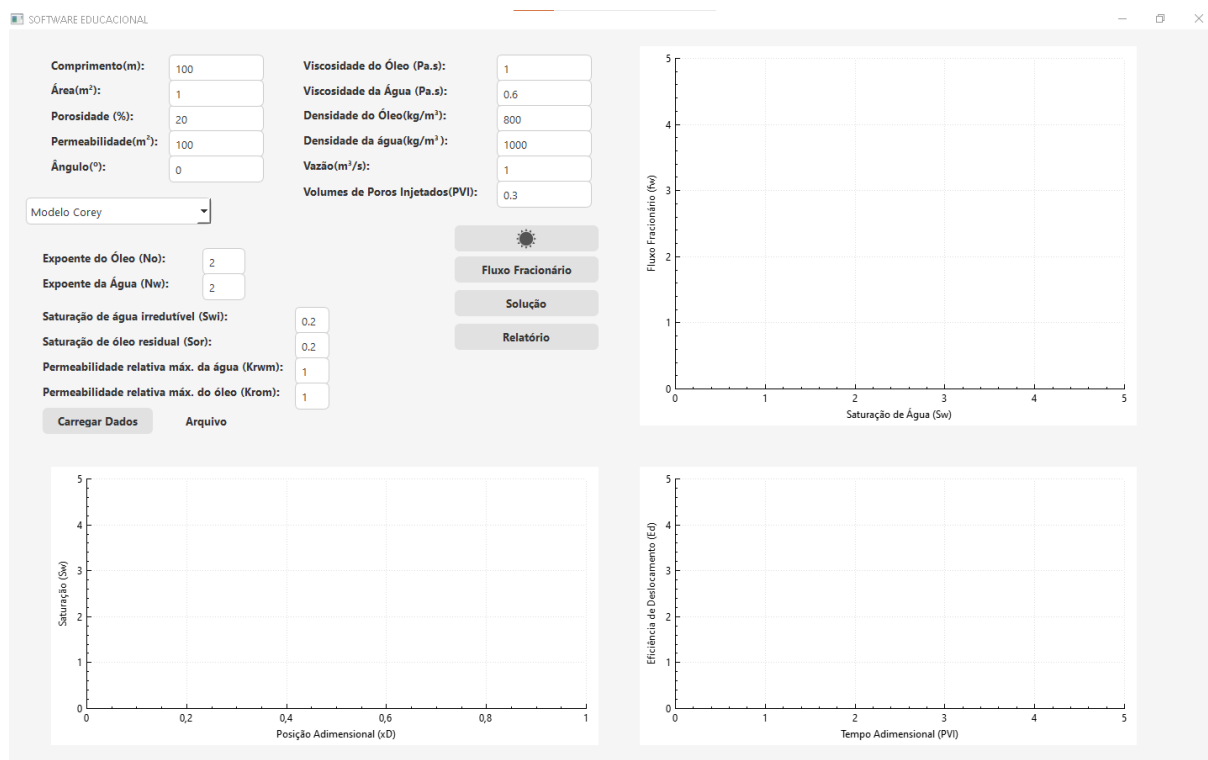


Figura 6.3: Versão 2.3, imagem do programa rodando

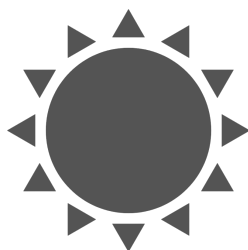


Figura 6.4: Ícone para tema claro



Figura 6.5: Ícone para tema escuro

Capítulo 7

Ciclos Construção - Implementação

Neste capítulo, são apresentados os códigos fonte implementados.

7.1 Código fonte

Apresenta-se na listagem 7.1 o arquivo de cabeçalho da classe `CCelula`, contendo as definições dos atributos do volume elementar do reservatório.

Listagem 7.1: Arquivo de Cabeçalho (`CCelula.h`)

```
#ifndef CCELULA_H
#define CCELULA_H

/**
5 * @file CCelula.h
 * @brief Definição da classe CCelula..
 */

/**
10 * @class CCelula
 * @brief Representa um ponto discreto (volume elementar) no domínio
 *       do reservatório.
 *
 * Na abordagem analítica, a célula funciona como um container de
 * dados que armazena
 * o estado de uma saturação específica (Sw) e sua posição (x)
 * correspondente em um
15 * determinado tempo (t).
 *
 * @author Fabiane da Silva Barros
 * @date Janeiro 2026
 */
20 class CCelula {
private:
    double _saturacao;    ///< Saturação de água (Sw) neste ponto.
    double _posicao;       ///< Posição espacial (x) no reservatório
                           [m].
}
```

```

    double _derivadaFluxo;    ///< Velocidade adimensional da onda (dfw
                             /dSw).

25
public:
    /**
     * @brief Construtor padrão da classe CCellula.
     * Inicializa todos os atributos numéricos com valor zero.
30     */
    CCellula();

    /**
     * @brief Construtor parametrizado da classe CCellula.
35     *
     * @param sw Valor da saturação de água da célula.
     * @param x Posição inicial no espaço (adimensional ou em metros)
     *
     * @param dfw Valor da derivada do fluxo fracionário (velocidade
     *           da onda associada).
     */
40    CCellula(double sw, double x, double dfw);

    /**
     * @brief Destrutor virtual da classe CCellula.
     */
45    virtual ~CCellula();

    // --- Getters (Métodos de Acesso) ---

    /**
50     * @brief Retorna a saturação de água atual da célula.
     * @return O valor da saturação (Sw) compreendido entre 0 e 1.
     */
    double getSaturacao() const;

55    /**
     * @brief Retorna a posição espacial atual da célula.
     * @return O valor da posição (x).
     */
    double getPosicao() const;

60    /**
     * @brief Retorna a derivada do fluxo fracionário armazenada na
     *       célula.
     * @return O valor da taxa de variação (dfw/dSw).
     */
65    double getDerivadaFluxo() const;

    // --- Setters (Métodos de Modificação) ---

```

```

70  /**
    * @brief Define uma nova saturação de água para a célula.
    * @param sw 0 novo valor da saturação de água.
    */
    void setSaturacao(double sw);

75  /**
    * @brief Define uma nova posição espacial para a célula.
    * @param x 0 novo valor da posição.
    */
    void setPosicao(double x);

80  /**
    * @brief Define uma nova derivada de fluxo para a célula.
    * @param dfw 0 novo valor da derivada do fluxo fracionário.
    */
85  void setDerivadaFluxo(double dfw);
};

#endif // CCELULA_H

```

Apresenta-se na listagem 7.2 o arquivo de implementação da classe C Celula.

Listagem 7.2: Arquivo de Implementação C Celula.cpp

```

/**
 * @file C Celula.cpp
 * @brief Implementação dos métodos da classe C Celula.
 * @author Fabiane da Silva Barros
5  * @date Janeiro 2026
 */

#include "C Celula.h"

10 // --- Construtor Padrão ---
C Celula::C Celula()
    : _saturacao(0.0), _posicao(0.0), _derivadaFluxo(0.0)
{
}

15 // --- Construtor Parametrizado ---
C Celula::C Celula(double sw, double x, double dfw)
    : _saturacao(sw), _posicao(x), _derivadaFluxo(dfw)
{
20 }

// --- Destrutor ---
C Celula::~C Celula() {
}

25 // --- Getters ---

```

```

double CCelula::getSaturacao() const {
    return _saturacao;
}

30 double CCelula::getPosicao() const {
    return _posicao;
}

35 double CCelula::getDerivadaFluxo() const {
    return _derivadaFluxo;
}

// --- Setters ---
40 void CCelula::setSaturacao(double sw) {
    _saturacao = sw;
}

void CCelula::setPosicao(double x) {
45     _posicao = x;
}

void CCelula::setDerivadaFluxo(double dfw) {
    _derivadaFluxo = dfw;
50 }

```

Apresenta-se na listagem 7.3 o arquivo de cabeçalho da classe **CMalha**, responsável por agregar e organizar espacialmente as células do reservatório em um instante de tempo.

Listagem 7.3: Arquivo de Cabeçalho CMalha.h

```

#ifndef CMALHA_H
#define CMALHA_H

/**
5 * @file CMalha.h
 * @brief Definição da classe CMalha.
 */

#include <vector>
10 #include "CCelula.h"

/**
 * @class CMalha
 * @brief Representa o domínio discretizado do reservatório (o
    conjunto de pontos).
15 *
 * Na abordagem analítica, a malha armazena o "perfil instantâneo" da
    simulação.
 * Ela contém um vetor de células, onde cada célula representa um
    estado (Posição, Saturação)
 * calculado para o tempo atual de injeção da água.

```

```

*
20 * @author Fabiane da Silva Barros
* @date Janeiro 2026
*/
class CMalha {
private:
25     std::vector<CCelula> _celulas; ///< Vetor dinâmico contendo os
        pontos do perfil espacial.
        double _tempoAtual;          ///< O tempo adimensional (em PVI)
        correspondente a este perfil.

public:
    /**
30     * @brief Construtor padrão da classe CMalha.
    * Inicializa a malha com um vetor vazio e tempo zero.
    */
    CMalha();

35     /**
    * @brief Destrutor virtual da classe CMalha.
    * Garante a liberação adequada dos recursos do vetor.
    */
    virtual ~CMalha();

40     // --- Gerenciamento de Dados ---

    /**
    * @brief Remove todas as células da malha.
45     * Método utilizado antes de recalcular um novo perfil espacial
    para um tempo diferente.
    */
    void limpar();

    /**
50     * @brief Adiciona uma nova célula ao final da malha.
    * @param celula Objeto CCelula já configurado (contendo x, Sw e
    derivada).
    */
    void adicionarCelula(const CCelula& celula);

55     /**
    * @brief Ordena as células com base na posição espacial (x) de
    forma crescente.
    * Fundamental para garantir a plotagem correta dos gráficos,
    evitando que as linhas fiquem embaralhadas.
    */
    void ordenarPorPosicao();

60     // --- Setters e Getters ---

```

```

/**
 * @brief Define o tempo atual associado ao perfil da malha.
65  * @param tempo O valor do tempo adimensional (PVI).
 */
void setTempoAtual(double tempo);

/**
70  * @brief Retorna o tempo atual do perfil armazenado na malha.
 * @return O tempo em PVI.
 */
double getTempoAtual() const;

75  /**
 * @brief Acesso direto ao vetor de células (somente leitura).
 * @return Referência constante ao vetor de células da malha.
 */
const std::vector<CCelula>& getCelulas() const;

80  /**
 * @brief Acesso direto ao vetor de células (leitura e escrita).
 * Útil se a classe CSolver precisar manipular ou iterar sobre o
 * vetor diretamente para aplicar correções.
 * @return Referência modificável ao vetor de células.
85  */
std::vector<CCelula>& getCelulas();
};

#endif // CMALHA_H

```

Apresenta-se na listagem 7.4 o arquivo de implementação da classe `CMalha`, demonstrando a utilização da biblioteca padrão de algoritmos genéricos (STL) para ordenação espacial.

Listagem 7.4: Arquivo de Implementação `CMalha.cpp`

```

/**
 * @file CMalha.cpp
 * @brief Implementação dos métodos da classe CMalha.
 * @author Fabiane da Silva Barros
5  * @date Janeiro 2026
 */

#include "CMalha.h"
#include <algorithm> // Necessário para a função std::sort
10 // --- Construtor ---
CMalha::CMalha() : _tempoAtual(0.0) {
    // Vetor de células inicia vazio automaticamente
}
15

```

```

// --- Destrutor ---
CMalha::~CMalha() {
    limpar();
}
20
// --- Gerenciamento ---

void CMalha::limpar() {
    _celulas.clear();
25}

void CMalha::adicionarCelula(const CCelula& celula) {
    _celulas.push_back(celula);
}

30
void CMalha::ordenarPorPosicao() {
    // Utiliza std::sort da STL com uma função lambda para comparar
    // as posições (x)
    std::sort(_celulas.begin(), _celulas.end(),
        [](const CCelula& a, const CCelula& b) {
35         return a.getPosicao() < b.getPosicao();
        }
    );
}

40// --- Setters e Getters ---

void CMalha::setTempoAtual(double tempo) {
    _tempoAtual = tempo;
}

45
double CMalha::getTempoAtual() const {
    return _tempoAtual;
}

50const std::vector<CCelula>& CMalha::getCelulas() const {
    return _celulas;
}

std::vector<CCelula>& CMalha::getCelulas() {
55    return _celulas;
}

```

Apresenta-se na listagem 7.5 a interface pura (classe abstrata) que dita as regras para a aplicação do Padrão de Projeto *Strategy* no cálculo das permeabilidades relativas.

Listagem 7.5: Arquivo de Cabeçalho ICurvasPermeabilidade.h

```

#ifndef ICURVASPERMEABILIDADE_H
#define ICURVASPERMEABILIDADE_H

```

```

/**
5  * @file ICurvasPermeabilidade.h
  * @brief Definição da interface ICurvasPermeabilidade.
  */

#include <string>

10 /**
   * @class ICurvasPermeabilidade
   * @brief Interface (Classe Abstrata) para modelos empíricos de
     permeabilidade relativa.
   *
15  * Define o contrato rigoroso que todos os modelos de permeabilidade
     (Corey, LET,
   * Chierici e Tabelado) devem implementar. A arquitetura baseia-se no
     padrão de
   * projeto comportamental Strategy, permitindo que o simulador
     principal e a
   * calculadora geométrica troquem o modelo matemático em tempo de
     execução sem
   * necessitar de qualquer alteração estrutural na sua lógica interna.
20  *
   * @author Fabiane da Silva Barros
   * @date Janeiro 2026
   */
class ICurvasPermeabilidade {
25 public:
    /**
      * @brief Destrutor virtual da interface.
      * Fundamental em polimorfismo C++ para garantir a chamada em
        cascata dos destrutores
      * e a destruição segura e completa das instâncias das classes
        derivadas na memória dinâmica.
30    */
    virtual ~ICurvasPermeabilidade() {}

    /**
      * @brief Carrega parâmetros ou dados discretizados a partir de
        um arquivo de disco externo.
35    * @param arquivo Caminho absoluto ou relativo para o arquivo de
        texto (.txt) contendo os coeficientes empíricos ou a tabela
        de dados.
      */
    virtual void carregarDados(const std::string& arquivo) = 0;

    /**
40    * @brief Calcula a permeabilidade relativa da fase aquosa (krw)
        para uma dada saturação.
      * @param sw Saturação de água atual na célula (variável

```

```

        independente).
    * @return Valor adimensional da permeabilidade relativa da água
      (frequentemente no intervalo de 0 a 1).
    */
    virtual double getKrw(double sw) const = 0;
45
    /**
    * @brief Calcula a permeabilidade relativa da fase oleica (kro)
      para uma dada saturação.
    * @param sw Saturação de água atual na célula (variável
      independente).
    * @return Valor adimensional da permeabilidade relativa do óleo
      (frequentemente no intervalo de 0 a 1).
50    */
    virtual double getKro(double sw) const = 0;
};

#endif // ICURVASPERMEABILIDADE_H

```

Apresenta-se na listagem 7.6 o arquivo de cabeçalho da classe CCurvasPermeabilidadeCorey, que concretiza o modelo empírico de potência para o cálculo bifásico.

Listagem 7.6: Arquivo de Cabeçalho CCurvasPermeabilidadeCorey.h

```

#ifndef CCURVASPERMEABILIDADECOREY_H
#define CCURVASPERMEABILIDADECOREY_H

/**
5  * @file CCurvasPermeabilidadeCorey.h
  * @brief Definição da classe CCurvasPermeabilidadeCorey.
  */

#include "ICurvasPermeabilidade.h"
10#include <string>
#include <cmath>

/**
  * @class CCurvasPermeabilidadeCorey
15 * @brief Implementação do modelo analítico de permeabilidade
      relativa de Corey.
  *
  * O modelo de Corey é uma correlação empírica clássica do tipo
    potência,
  * amplamente utilizada na engenharia de reservatórios para sistemas
    bifásicos.
  * Ele assume que as permeabilidades relativas são proporcionais a
    uma potência
20 * da saturação normalizada de água ( $S_{wn}$ ).
  *
  * As equações fundamentais são dadas por:
  * -  $k_{rw} = k_{rw,max} \cdot (S_{wn})^{n_w}$ 

```

```

* - \f$ k_{ro} = k_{ro,max} \cdot (1 - S_{wn})^{n_o} \f$
25 *
* Sendo a saturação normalizada:
* \f$ S_{wn} = \frac{S_w - S_{wir}}{1 - S_{wir} - S_{or}} \f$
*
* @author Fabiane da Silva Barros
30 * @date Janeiro 2026
*/

class CCurvasPermeabilidadeCorey : public ICurvasPermeabilidade {
private:
    double _swir;      ///< Saturação irreduzível de água ($S_{wir}$).
35    double _sorw;      ///< Saturação residual de óleo ($S_{or}$).
    double _krw_max;   ///< Permeabilidade relativa máxima da água (
        endpoint).
    double _kro_max;   ///< Permeabilidade relativa máxima do óleo (
        endpoint).
    double _nw;        ///< Expoente empírico de Corey para a fase
        aquosa.
    double _no;        ///< Expoente empírico de Corey para a fase
        oleica.

40    /**
    * @brief Calcula a saturação normalizada efetiva de água.
    * @param sw Saturação real instantânea.
    * @return Saturação normalizada ($S_{wn}$), limitada
    *         estritamente entre 0.0 e 1.0.
45    */
    double calcularSwNorm(double sw) const;

public:
    /**
50    * @brief Construtor parametrizado completo.
    *
    * @param kroMax Permeabilidade relativa máxima do óleo.
    * @param krwMax Permeabilidade relativa máxima da água.
    * @param no Expoente da curva de óleo.
55    * @param nw Expoente da curva de água.
    * @param swir Saturação irreduzível de água.
    * @param sor Saturação residual de óleo.
    */
    CCurvasPermeabilidadeCorey(double kroMax, double krwMax, double
        no, double nw, double swir, double sor);

60    /**
    * @brief Construtor padrão.
    * Inicializa instâncias temporárias com todos os coeficientes
    * zerados.
    */
65    CCurvasPermeabilidadeCorey();

```

```

/**
 * @brief Destrutor virtual.
 */
70 virtual ~CCurvasPermeabilidadeCorey();

/**
 * @brief Carrega os coeficientes do modelo a partir de um
        arquivo externo de texto.
 *
75 * Espera-se um arquivo plano estruturado com 6 valores numéricos
        separados por espaços:
 * Swir, Sor, KroMax, KrwMax, No, Nw.
 *
 * @param arquivo Caminho absoluto ou relativo do arquivo `.txt`.
 * @throws std::runtime_error Se o arquivo for inacessível ou
        estiver corrompido.
80 */
void carregarDados(const std::string& arquivo) override;

/**
 * @brief Calcula a Permeabilidade Relativa da fase Água ($k_{rw}$).
85 * @param sw Saturação espacial de água atual.
 * @return Valor adimensional interpolado.
 */
double getKrw(double sw) const override;

90 /**
 * @brief Calcula a Permeabilidade Relativa da fase Óleo ($k_{ro}$).
 * @param sw Saturação espacial de água atual.
 * @return Valor adimensional interpolado.
 */
95 double getKro(double sw) const override;

        /** @brief Retorna a Saturação Irredutível. */
double getSwi() const { return _swir; }

100 /** @brief Retorna a Saturação de óleo residual. */
double getSor() const { return _sor; }
};

#endif // CCURVASPERMEABILIDADECOREY_H

```

Apresenta-se na listagem 7.7 o arquivo de implementação, detalhando os limitadores numéricos implementados para garantir estabilidade matemática na normalização das saturações.

Listagem 7.7: Arquivo de Implementação CCurvasPermeabilidadeCorey.cpp

```

/**
 * @file CCurvasPermeabilidadeCorey.cpp
 * @brief Implementação dos métodos do modelo de permeabilidade
 *        relativa de Corey.
 * @author Fabiane da Silva Barros
5  * @date Janeiro 2026
 * /

#include "CCurvasPermeabilidadeCorey.h"
#include <algorithm> // Para os limitadores std::max, std::min
10 #include <fstream> // Para leitura de streams de disco (std::
    ifstream)
#include <stdexcept> // Para lançamento de exceções em tempo de
    execução

// --- Construtor Parametrizado ---
CCurvasPermeabilidadeCorey::CCurvasPermeabilidadeCorey(double kroMax,
    double krwMax, double no, double nw, double swir, double sor)
15 : _swir(swir), _sorw(sor), _krw_max(krwMax), _kro_max(kroMax),
    _nw(nw), _no(no)
{
}

// --- Construtor Padrão ---
20 CCurvasPermeabilidadeCorey::CCurvasPermeabilidadeCorey()
    : _swir(0.0), _sorw(0.0), _krw_max(0.0), _kro_max(0.0), _nw(0.0),
    _no(0.0)
{
}

25 // --- Destrutor ---
CCurvasPermeabilidadeCorey::~CCurvasPermeabilidadeCorey() {
}

// --- Método Auxiliar Privado ---
30 double CCurvasPermeabilidadeCorey::calcularSwNorm(double sw) const {
    double denominador = 1.0 - _swir - _sorw;

    // Tratamento numérico para evitar singularidade e divisão por
    zero
    if (denominador <= 1e-6) return 0.0;

35     double sw_norm = (sw - _swir) / denominador;

    // Assegura robustez física: Swn nunca será negativo ou maior que
    a unidade
    return std::max(0.0, std::min(1.0, sw_norm));
40 }

```

```

// --- Implementação da Interface ICurvasPermeabilidade ---

void CCurvasPermeabilidadeCorey::carregarDados(const std::string&
arquivo) {
45   std::ifstream in(arquivo);
   if (!in.is_open()) {
       throw std::runtime_error("Exceção_Corey: Falha na montagem do
           arquivo->" + arquivo);
   }

50   // Processo de extração serial dos 6 coeficientes estruturais
   if (!(in >> _swir >> _sorw >> _kro_max >> _krw_max >> _no >> _nw)
       ) {
       throw std::runtime_error("Exceção_Corey: Erro no formato
           numérico de entrada.");
   }

55   in.close();
}

double CCurvasPermeabilidadeCorey::getKrw(double sw) const {
   double sw_norm = calcularSwNorm(sw);
60   // Eq: krw = krw_max * (Swn)^nw
   return _krw_max * std::pow(sw_norm, _nw);
}

double CCurvasPermeabilidadeCorey::getKro(double sw) const {
65   double sw_norm = calcularSwNorm(sw);
   // Eq: kro = kro_max * (1 - Swn)^no
   return _kro_max * std::pow(1.0 - sw_norm, _no);
}

```

Apresenta-se na listagem 7.8 o arquivo de cabeçalho da classe que implementa o modelo numérico tabelado.

Listagem 7.8: Arquivo de Cabeçalho CCurvasPermeabilidadeTabelada.h

```

#ifndef CCURVASPERMEABILIDADETABELADA_H
#define CCURVASPERMEABILIDADETABELADA_H

/**
5 * @file CCurvasPermeabilidadeTabelada.h
 * @brief Definição da classe CCurvasPermeabilidadeTabelada.
 */

#include "ICurvasPermeabilidade.h"
10#include <vector>
#include <string>

/**

```

```

* @class CCurvasPermeabilidadeTabelada
15 * @brief Implementação empírica de curvas de permeabilidade baseadas
    em dados laboratoriais discretos.
*
* Esta classe concreta da interface ICurvasPermeabilidade permite a
    inserção
* de dados experimentais arbitrários (pontos discretos retirados de
    ensaios de
* laboratório *Special Core Analysis* - SCAL) para definir as
    funções
20 * de permeabilidade relativa.
*
* A continuidade das funções para os cálculos diferenciais de fluxo
    fracionário
* é garantida por meio de interpolação linear entre os nós discretos
    do domínio de saturação.
*
25 * @author Fabiane da Silva Barros
    * @date Janeiro 2026
    */
class CCurvasPermeabilidadeTabelada : public ICurvasPermeabilidade {
private:
30     std::vector<double> _sw; ///< Vetor de domínio: Valores da
        Saturação da fase aquosa ($S_w$).
        std::vector<double> _krw; ///< Vetor imagem: Valores da
        Permeabilidade relativa da Água ($k_{rw}$).
        std::vector<double> _kro; ///< Vetor imagem: Valores da
        Permeabilidade relativa do Óleo ($k_{ro}$).

    /**
35     * @brief Executa o algoritmo numérico de Interpolação e
        Extrapolação Linear plana.
    *
    * Este método privado procura a faixa de domínio  $[x_i, x_{i+1}]$  onde o parâmetro
    *  $x_{\text{desejado}}$  se encontra. O valor da imagem é obtido pela
        equação geométrica da secante:
    * 
$$y = y_i + \frac{(x - x_i) \cdot (y_{i+1} - y_i)}{x_{i+1} - x_i}$$

40     * Valores fora dos limites tabelados são extrapolados mantendo a
        assíntota plana da borda (constante).
    *
    * @param x_desejado O valor interpolante (Saturação de água alvo
        ).
    * @param vec_x A referência constante do vetor de coordenadas
        independentes (Domínio).
    * @param vec_y A referência constante do vetor de coordenadas
        dependentes (Imagem).
45     * @return O valor escalar aproximado  $y(x)$ .

```

```

    */
    double interpolar(double x_desejado, const std::vector<double>&
        vec_x, const std::vector<double>& vec_y) const;

public:
50    /**
        * @brief Construtor padrão da classe
        *       CCurvasPermeabilidadeTabelada.
        */
    CCurvasPermeabilidadeTabelada() = default;

55    /**
        * @brief Destrutor virtual da classe.
        */
    virtual ~CCurvasPermeabilidadeTabelada() = default;

60    /**
        * @brief Executa a leitura sequencial ou paralela de dados
        *       estruturados em arquivo ASCII.
        *
        * O leitor (parser) varre o arquivo em busca das diretrizes de
        * controle:
        * `DADOS_KR_INICIO` e `FIM_DADOS`. Os dados internos devem estar
        * alinhados
65    * matricialmente como: `[Sw] [Krw] [Kro]`.
        *
        * @param arquivo Caminho relativo ou absoluto do diretório do
        *       arquivo `.txt`.
        * @throws std::runtime_error Lança exceção bloqueante caso o
        *       arquivo seja inacessível ou se
        *       a demarcação das colunas não for localizada.
70    */
    void carregarDados(const std::string& arquivo) override;

    /**
        * @brief Computa a permeabilidade relativa aquosa contínua.
75    * @param sw Saturação atual de água da malha ($S_w$).
        * @return A interpolação de \f$ k_{rw}(S_w) \f$.
        */
    double getKrw(double sw) const override;

80    /**
        * @brief Computa a permeabilidade relativa oleica contínua.
        * @param sw Saturação atual de água da malha ($S_w$).
        * @return A interpolação de \f$ k_{ro}(S_w) \f$.
        */
85    double getKro(double sw) const override;

    /** @brief Retorna a Saturação Irredutível (primeiro ponto da

```

```

        tabela). */
double getSwi() const {
    return _sw.empty() ? 0.0 : _sw.front();
90 }

    /** @brief Retorna a Saturação Máxima de Água (último ponto
        da tabela). */
double getSwMax() const {
    return _sw.empty() ? 1.0 : _sw.back();
95 }
};

#endif // CCURVASPERMEABILIDADETABELADA_H

```

Apresenta-se na listagem 7.9 o arquivo de implementação correspondente, contendo as funções de manipulação de disco ASCII e interpolação linear.

Listagem 7.9: Arquivo de Implementação CCurvasPermeabilidadeTabelada.cpp

```

/**
 * @file CCurvasPermeabilidadeTabelada.cpp
 * @brief Implementação das lógicas de leitura (parser) e
 *        interpolação numérica.
 * @author Fabiane da Silva Barros
5 * @date Janeiro 2026
 */

#include "CCurvasPermeabilidadeTabelada.h"
#include <iostream>
10#include <fstream>
#include <sstream>
#include <stdexcept>

// --- Implementação da Interface ICurvasPermeabilidade ---
15
void CCurvasPermeabilidadeTabelada::carregarDados(const std::string&
    arquivo) {
    std::ifstream arq(arquivo);
    if (!arq.is_open()) {
        throw std::runtime_error("Exceção_IO_(Tabelado):_Impossível_
            resolver_caminho_ou_abrir_arquivo:_ " + arquivo);
20 }

    std::string linha;
    bool lendoDados = false;

25 // Limpeza de buffer de dados antigos antes do carregamento
    _sw.clear();
    _krw.clear();
    _kro.clear();

```

```

30 // Rotina de varredura top-down e limpeza de ruídos no ASCII
while (std::getline(arq, linha)) {
    if (linha.empty() || linha[0] == '#') continue;

    std::stringstream ss(linha);
35 std::string palavraChave;
    ss >> palavraChave;

    // Máquina de estados simples: Início de bloco
    if (palavraChave == "DADOS_KR_INICIO") {
40         lendoDados = true;
        continue;
    }

    // Fim de bloco de captura
45 if (palavraChave == "FIM_DADOS") {
        lendoDados = false;
        break;
    }

50 if (lendoDados) {
        std::stringstream ss_linha(linha);
        double sw, krw, kro;
        ss_linha >> sw >> krw >> kro;

55         _sw.push_back(sw);
        _krw.push_back(krw);
        _kro.push_back(kro);
    }
}

60 // Validação Pós-Leitura de Integridade de Tamanho
if (_sw.empty()) {
    throw std::runtime_error("Exceção Arquitetural: Arquivo
        malformatado. Ausência dos delimitadores 'DADOS_KR_INICIO' e
        'FIM_DADOS'.");
}

65 }

double CCurvasPermeabilidadeTabelada::getKrw(double sw) const {
    return interpolar(sw, _sw, _krw);
}

70 double CCurvasPermeabilidadeTabelada::getKro(double sw) const {
    return interpolar(sw, _sw, _kro);
}

75 // --- Métodos Numéricos Auxiliares ---

```

```

double CCurvasPermeabilidadeTabelada::interpolar(double x_desejado,
    const std::vector<double>& vec_x, const std::vector<double>& vec_y
) const {

    // Cenário Extremo de Contorno Esquerdo: Extrapolação inferior
    plana
80    if (x_desejado <= vec_x.front()) {
        return vec_y.front();
    }

    // Cenário Extremo de Contorno Direito: Extrapolação superior
    plana
85    if (x_desejado >= vec_x.back()) {
        return vec_y.back();
    }

    // Busca linear de ancoragem intervalar (Interval-matching)
90    for (size_t i = 0; i < vec_x.size() - 1; ++i) {

        // Limitação lógica:  $x_i \leq x \leq x_{i+1}$ 
        if (x_desejado >= vec_x[i] && x_desejado <= vec_x[i+1]) {

95            double x0 = vec_x[i];
            double y0 = vec_y[i];
            double x1 = vec_x[i+1];
            double y1 = vec_y[i+1];

100            // Proteção contra anomalias geométricas em matrizes
                densas (divisão por zero)
            if (x1 == x0) {
                return y0;
            }

105            // Lei das proporções geométricas (Semelhança de
                triângulos na secante)
            return y0 + (x_desejado - x0) * (y1 - y0) / (x1 - x0);
        }
    }

110    // Falha em cascata final de segurança (*fallback*)
    return vec_y.back();
}

```

Apresenta-se na listagem 7.10 o cabeçalho da classe referente ao modelo empírico de Lombez, Escovedo e Trindade (LET), notório por sua flexibilidade paramétrica.

Listagem 7.10: Arquivo de Cabeçalho (CCurvasPermeabilidadeLET.h)

```

#ifndef CCURVASPERMEABILIDADELET_H
#define CCURVASPERMEABILIDADELET_H

```

```

/**
5  * @file CCurvasPermeabilidadeLET.h
  * @brief Definição da classe CCurvasPermeabilidadeLET.
  */

#include "ICurvasPermeabilidade.h"
10#include <cmath>
#include <string>

/**
  * @class CCurvasPermeabilidadeLET
15 * @brief Implementação empírica do modelo de permeabilidade relativa
    LET.

  *
  * O modelo LET (Lombez, Escovedo e Trindade, 2008) é uma correlação
    matemática
  * altamente flexível. Ele utiliza três parâmetros empíricos (L, E, T
    ) para descrever
  * com precisão a forma (shape), elevação (elevation) e translação (
    translation)
20 * das curvas de permeabilidade relativa, ajustando-se a dados
    laboratoriais complexos.

  *
  * As equações fundamentais são dadas por:
  * -  $k_{rw} = \frac{(S_{wn})^{L_w}}{(S_{wn})^{L_w} + E_w \cdot (1 - S_{wn})^{T_w}}$ 
  * -  $k_{ro} = \frac{(1 - S_{wn})^{L_o}}{(1 - S_{wn})^{L_o} + E_o \cdot (S_{wn})^{T_o}}$ 
25 *
  * @author Fabiane da Silva Barros
  * @date Janeiro 2026
  */

class CCurvasPermeabilidadeLET : public ICurvasPermeabilidade {
30private:
    // --- Parâmetros da Fase Aquosa ---
    double _Lw; ///< Parâmetro empírico de forma (L) para a água.
    double _Ew; ///< Parâmetro empírico de elevação (E) para a água.
    double _Tw; ///< Parâmetro empírico de translação (T) para a água
    .

35    // --- Parâmetros da Fase Oleica ---
    double _Lo; ///< Parâmetro empírico de forma (L) para o óleo.
    double _Eo; ///< Parâmetro empírico de elevação (E) para o óleo.
    double _To; ///< Parâmetro empírico de translação (T) para o óleo
    .

40    // --- Saturações de Contorno ---
    double _Swirr; ///< Saturação irreduzível de água ($S_{wirr}$).
    double _Sor;   ///< Saturação residual de óleo ($S_{or}$).

```

```

45 public:
    /**
     * @brief Construtor principal parametrizado da classe LET.
     *
     * @param Lw Parâmetro L para água.
     * @param Ew Parâmetro E para água.
50  * @param Tw Parâmetro T para água.
     * @param Lo Parâmetro L para óleo.
     * @param Eo Parâmetro E para óleo.
     * @param To Parâmetro T para óleo.
55  * @param Swirr Saturação irreduzível de água.
     * @param Sor Saturação residual de óleo.
     */
    CCurvasPermeabilidadeLET(double Lw, double Ew, double Tw,
                               double Lo, double Eo, double To,
60                               double Swirr, double Sor);

    /**
     * @brief Construtor padrão.
     * Inicializa todos os coeficientes empíricos e saturações
     * extremas com zero.
65  */
    CCurvasPermeabilidadeLET();

    /**
     * @brief Destrutor virtual da classe.
70  */
    virtual ~CCurvasPermeabilidadeLET();

    /**
     * @brief Carrega os coeficientes estruturais do modelo LET a
     * partir de um arquivo de disco.
75  *
     * A leitura espera a extração sequencial de 8 parâmetros no
     * formato ASCII:
     * `[Lw] [Ew] [Tw] [Lo] [Eo] [To] [Swirr] [Sor]`
     *
     * @param arquivo Caminho relativo ou absoluto do arquivo de
     * configuração (.txt).
80  * @throws std::runtime_error Se o arquivo for inválido ou não
     * contiver os 8 parâmetros.
     */
    void carregarDados(const std::string& arquivo) override;

    /**
85  * @brief Computa a permeabilidade relativa da fase aquosa ($k_{rw}$) usando a formulação LET.
     * @param Sw Saturação atual de água ($S_w$).

```

```

    * @return O valor adimensional de $k_{rw}$.
    */
double getKrw(double Sw) const override;

90 /**
    * @brief Computa a permeabilidade relativa da fase oleica ($k_{ro}$) usando a formulação LET.
    * @param Sw Saturação atual de água ($S_w$).
    * @return O valor adimensional de $k_{ro}$.
95 */
double getKro(double Sw) const override;

    /** @brief Retorna a Saturação Irredutível. */
double getSwi() const { return _Swirr; }

100 /** @brief Retorna a Saturação de óleo residual. */
double getSor() const { return _Sor; }
};

105 #endif // CCURVASPERMEABILIDADELET_H

```

Apresenta-se na listagem 7.11 o correspondente arquivo de implementação com os cálculos numéricos das permeabilidades relativas.

Listagem 7.11: Arquivo de Implementação CCurvasPermeabilidadeLET.cpp

```

/**
 * @file CCurvasPermeabilidadeLET.cpp
 * @brief Implementação dos métodos e equações do modelo
        CCurvasPermeabilidadeLET.
 * @author Fabiane da Silva Barros
5 * @date Janeiro 2026
 */

#include "CCurvasPermeabilidadeLET.h"
#include <iostream>
10 #include <fstream>
#include <stdexcept>

// --- Construtor Principal ---
CCurvasPermeabilidadeLET::CCurvasPermeabilidadeLET(double Lw, double
    Ew, double Tw,
15 double Lo, double
        Eo, double To,
        double Swirr,
        double Sor)

: _Lw(Lw), _Ew(Ew), _Tw(Tw),
  _Lo(Lo), _Eo(Eo), _To(To),
  _Swirr(Swirr), _Sor(Sor)
20 {
}

```

```

// --- Construtor Vazio ---
CCurvasPermeabilidadeLET::CCurvasPermeabilidadeLET()
25 : _Lw(0.0), _Ew(0.0), _Tw(0.0),
    _Lo(0.0), _Eo(0.0), _To(0.0),
    _Swirr(0.0), _Sor(0.0)
{
}
30
// --- Destrutor ---
CCurvasPermeabilidadeLET::~~CCurvasPermeabilidadeLET() {
}

35 // --- Implementação da Interface ICurvasPermeabilidade ---

void CCurvasPermeabilidadeLET::carregarDados(const std::string&
arquivo) {
    std::ifstream in(arquivo);
    if (!in.is_open()) {
40         throw std::runtime_error("Exceção LET: Não foi possível
            localizar ou abrir o arquivo: " + arquivo);
    }

    // Leitura estrita dos 8 parâmetros do modelo LET
    if (!(in >> _Lw >> _Ew >> _Tw >> _Lo >> _Eo >> _To >> _Swirr >>
        _Sor)) {
45         throw std::runtime_error("Exceção LET: Formato numérico
            corrompido. Esperado: _Lw _Ew _Tw _Lo _Eo _To _Swirr _Sor");
    }

    in.close();
}

50
double CCurvasPermeabilidadeLET::getKrw(double Sw) const {
    double denominador = 1.0 - _Swirr - _Sor;
    if (denominador <= 1e-6) return 0.0;

55 // Normalização da saturação
    double Swn = (Sw - _Swirr) / denominador;

    // Condições físicas de contorno
    if (Swn <= 0.0) return 0.0;
    if (Swn >= 1.0) return 1.0;
60

    // Motor matemático LET para a água
    double num = std::pow(Swn, _Lw);
    double term2 = _Ew * std::pow(1.0 - Swn, _Tw);

65
    return num / (num + term2);
}

```

```

}

double CCurvasPermeabilidadeLET::getKro(double Sw) const {
70   double denominador = 1.0 - _Swirr - _Sor;
      if (denominador <= 1e-6) return 0.0;

      // Normalização da saturação
      double Swn = (Sw - _Swirr) / denominador;

75   // Condições físicas de contorno
      if (Swn <= 0.0) return 1.0;
      if (Swn >= 1.0) return 0.0;

80   // Motor matemático LET para o óleo
      double num = std::pow(1.0 - Swn, _Lo);
      double term2 = _Eo * std::pow(Swn, _To);

      return num / (num + term2);
85 }

```

Apresenta-se na listagem 7.12 o cabeçalho da classe que representa o modelo exponencial flexível de (CHIERICI, 1984).

Listagem 7.12: Arquivo de Cabeçalho CCurvasPermeabilidadeChierici.h

```

#ifndef CCURVASPERMEABILIDADECHIERICI_H
#define CCURVASPERMEABILIDADECHIERICI_H

/**
5  * @file CCurvasPermeabilidadeChierici.h
  * @brief Definição da classe CCurvasPermeabilidadeChierici.
  */

#include "ICurvasPermeabilidade.h"
10 #include <cmath>
    #include <string>

/**
  * @class CCurvasPermeabilidadeChierici
15  * @brief Implementação empírica do modelo exponencial de
      permeabilidade relativa de Chierici (1984).
  *
  * A classe concretiza as equações propostas por Chierici para
      descrever o
  * escoamento bifásico no meio poroso. Diferentemente do modelo de
      Corey, o
  * modelo de Chierici baseia-se em funções exponenciais de decaimento
      , sendo
20  * reconhecido pela elevada flexibilidade no ajuste da concavidade (
      curvature)
  * das curvas experimentais.

```

```

*
* As equações adotadas são:
* - \f$ k_{rw} = k_{rw,max} \cdot \exp\left(-A_w \cdot S_{wn}^{-B_w}\right) \f$
* - \f$ k_{ro} = k_{ro,max} \cdot \exp\left(-A_o \cdot (1 - S_{wn})^{-B_o}\right) \f$
25
*
* @author Fabiane da Silva Barros
* @date Janeiro 2026
*/
30 class CCurvasPermeabilidadeChierici : public ICurvasPermeabilidade {
private:
    double _Aw;        ///< Parâmetro empírico exponencial A para a
                        fase aquosa.
    double _Bw;        ///< Parâmetro empírico exponencial B para a
                        fase aquosa.
    double _Ao;        ///< Parâmetro empírico exponencial A para a
                        fase oleica.
35    double _Bo;        ///< Parâmetro empírico exponencial B para a
                        fase oleica.
    double _Swirr;     ///< Saturação irreduzível de água ($S_{wirr}$).
    double _Sor;       ///< Saturação residual de óleo ($S_{or}$).
    double _kroMax;    ///< Ponto final (endpoint) da permeabilidade
                        relativa do óleo.
    double _krwMax;    ///< Ponto final (endpoint) da permeabilidade
                        relativa da água.
40
public:
    /**
     * @brief Construtor principal parametrizado do modelo de
     Chierici.
     *
     * @param Aw Parâmetro de decaimento (shape) para a água.
     * @param Bw Parâmetro de curvatura (curvature) para a água.
     * @param Ao Parâmetro de decaimento (shape) para o óleo.
     * @param Bo Parâmetro de curvatura (curvature) para o óleo.
     * @param Swirr Saturação irreduzível de água.
     * @param Sor Saturação residual de óleo.
50    * @param krwMax Valor escalar máximo admitido para $k_{rw}$.
     * @param kroMax Valor escalar máximo admitido para $k_{ro}$.
     */
    CCurvasPermeabilidadeChierici(double Aw, double Bw, double Ao,
                                   double Bo,
55                                   double Swirr, double Sor,
                                   double krwMax, double kroMax);

    /**
     * @brief Construtor padrão.
60    * Inicializa instâncias temporárias com coeficientes nulos em

```

```

        memória.
    */
    CCurvasPermeabilidadeChierici();

    /**
65     * @brief Destrutor virtual da classe.
    */
    virtual ~CCurvasPermeabilidadeChierici();

    /**
70     * @brief Carrega os coeficientes do modelo Chierici a partir de
        um arquivo em disco.
    *
    * Lê 8 parâmetros na seguinte ordem estrita:
    * `Aw Bw Ao Bo Swirr Sor KroMax KrwMax`
    *
75     * @param arquivo Caminho para o arquivo `.txt`.
    * @throws std::runtime_error Em caso de falha de I/O ou
        incompatibilidade numérica.
    */
    void carregarDados(const std::string& arquivo) override;

80    /**
    * @brief Computa a permeabilidade relativa contínua da água ($k_{rw}$).
    * @param Sw Saturação real da fase aquosa na célula analítica.
    * @return O escalar calculado de \f$k_{rw}\f$.
    */
85    double getKrw(double Sw) const override;

    /**
    * @brief Computa a permeabilidade relativa contínua do óleo ($k_{ro}$).
    * @param Sw Saturação real da fase aquosa na célula analítica.
90     * @return O escalar calculado de \f$k_{ro}\f$.
    */
    double getKro(double Sw) const override;

    /** @brief Retorna a Saturação Irredutível. */
95    double getSwi() const { return _Swirr; }

    /** @brief Retorna a Saturação de óleo residual. */
    double getSor() const { return _Sor; }
};
100 #endif // CCURVASPERMEABILIDADECHIERICI_H

```

Apresenta-se na listagem 7.13 o arquivo de implementação do modelo, detalhando as proteções lógicas inseridas para evitar a singularidade exponencial inerente ao equaciona-

mento de Chierici.

Listagem 7.13: Arquivo de Implementação CCurvasPermeabilidadeChierici.cpp

```

/**
 * @file CCurvasPermeabilidadeChierici.cpp
 * @brief Implementação analítica das equações exponenciais do modelo
 *        Chierici.
 * @author Fabiane da Silva Barros
5  * @date Janeiro 2026
 * /

#include "CCurvasPermeabilidadeChierici.h"
#include <iostream>
10 #include <fstream>
#include <stdexcept>

// --- Construtor Principal ---
CCurvasPermeabilidadeChierici::CCurvasPermeabilidadeChierici(double
    Aw, double Bw, double Ao, double Bo,
15                                     double
                                           Swirr
                                           ,
                                           double
                                           Sor,
double
    krwMax
    ,
    double
                                           kroMax
                                           )
    : _Aw(Aw), _Bw(Bw), _Ao(Ao), _Bo(Bo),
      _Swirr(Swirr), _Sor(Sor), _kroMax(kroMax), _krwMax(krwMax)
{
20 }

// --- Construtor Vazio ---
CCurvasPermeabilidadeChierici::CCurvasPermeabilidadeChierici()
    : _Aw(0.0), _Bw(0.0), _Ao(0.0), _Bo(0.0),
25     _Swirr(0.0), _Sor(0.0), _kroMax(0.0), _krwMax(0.0)
{
}

// --- Destrutor ---
30 CCurvasPermeabilidadeChierici::~CCurvasPermeabilidadeChierici() {
}

// --- Implementação da Interface ICurvasPermeabilidade ---
35 void CCurvasPermeabilidadeChierici::carregarDados(const std::string&

```

```

arquivo) {
    std::ifstream in(arquivo);
    if (!in.is_open()) {
        throw std::runtime_error("Exceção Chierici: Falha de acesso_
        ao_arquivo_de_entrada:" + arquivo);
    }
40
    // Leitura estrita de 8 parâmetros formatados
    if (!(in >> _Aw >> _Bw >> _Ao >> _Bo >> _Swirr >> _Sor >> _kroMax
        >> _krwMax)) {
        throw std::runtime_error("Exceção Chierici: Avaria no formato
        dos dados. Esperado: Aw Bw Ao Bo Swirr Sor KroMax KrwMax"
        );
    }
45
    in.close();
}

double CCurvasPermeabilidadeChierici::getKrw(double Sw) const {
50
    double denominador = 1.0 - _Swirr - _Sor;

    // Proteção de estabilidade estrutural
    if (denominador <= 1e-6) return 0.0;

55
    double Swn = (Sw - _Swirr) / denominador;

    // Condições de contorno (Singularidade assintótica prevenida)
    if (Swn <= 0.0) return 0.0;
    if (Swn >= 1.0) return _krwMax;
60
    // Matemática Exponencial:  $Krw = K_{rwMax} * \exp(-Aw * Swn^{-Bw})$ 
    return _krwMax * std::exp(-_Aw * std::pow(Swn, -_Bw));
}

65 double CCurvasPermeabilidadeChierici::getKro(double Sw) const {
    double denominador = 1.0 - _Swirr - _Sor;

    if (denominador <= 1e-6) return 0.0;

70
    double Swn = (Sw - _Swirr) / denominador;

    // Condições de contorno (Singularidade assintótica prevenida)
    if (Swn >= 1.0) return 0.0;
    if (Swn <= 0.0) return _kroMax;
75
    // Matemática Exponencial:  $Kro = K_{kroMax} * \exp(-Ao * (1-Swn)^{-Bo})$ 
    return _kroMax * std::exp(-_Ao * std::pow(1.0 - Swn, -_Bo));
}

```

Apresenta-se na listagem 7.14 o cabeçalho da classe CCalculadoraFluxoFracionario, responsável pelo processamento analítico do escoamento imiscível segundo a formulação de Buckley-Leverett com as correções adimensionais gravitacionais.

Listagem 7.14: Arquivo de Cabeçalho CCalculadoraFluxoFracionario.h

```

#ifndef CCALCULADORAFLUXOFRACIONARIO_H
#define CCALCULADORAFLUXOFRACIONARIO_H

/**
5 * @file CCalculadoraFluxoFracionario.h
  * @brief Definição da classe CCalculadoraFluxoFracionario.
  */

#include "ICurvasPermeabilidade.h"
10 #include <map>
#include <cmath>

/**
  * @class CCalculadoraFluxoFracionario
15 * @brief Responsável pelos cálculos pontuais do fluxo fracionário e
    suas derivadas numéricas.
  *
  * Esta classe implementa a equação constitutiva do fluxo fracionário
    (Buckley-Leverett
  * generalizado), acoplando os efeitos de forças viscosas e
    gravitacionais no escoamento
  * bifásico incompressível.
20 *
  * A equação matemática governante implementada é:
  * 
$$f_w = \frac{1 - \frac{k \cdot k_{ro}}{u_t \cdot \mu_o} \Delta \rho \cdot g \cdot \sin(\alpha)}{1 + \frac{k_{ro}}{k_{rw}} \frac{\mu_w}{\mu_o}} f$$

  *
  * Onde:
25 * -  $f_w$  é a fração do fluxo total correspondente à água.
  * -  $\alpha$  é o ângulo de mergulho do reservatório.
  * -  $u_t$  é a velocidade total de Darcy.
  *
  * @author Fabiane da Silva Barros
30 * @date Janeiro 2026
  */
class CCalculadoraFluxoFracionario {
private:
  // --- Propriedades PVT dos Fluidos ---
35 double _mi_o;    ///< Viscosidade dinâmica da fase oleica ( $\mu_o$  [cP].
    double _mi_w;    ///< Viscosidade dinâmica da fase aquosa ( $\mu_w$  [cP].
    double _rho_o;    ///< Densidade da fase oleica ( $\rho_o$  [

```

```

    kg/m³].
double _rho_w;    ///< Densidade da fase aquosa (\f$ \rho_w \f$) [
    kg/m³].

40 // --- Propriedades Estruturais e Operacionais ---
double _k;        ///< Permeabilidade absoluta do meio poroso (\f$
    k \f$) [mD].
double _qt;        ///< Vazão volumétrica de injeção total (\f$ q_t
    \f$) [m³/d].
double _A;         ///< Área transversal hidráulica do reservatório
    (\f$ A \f$) [m²].
double _ut;        ///< Velocidade total aparente de Darcy (\f$ u_t
    = q_t/A \f$) [m/d].
45 double _angulo;  ///< Ângulo de mergulho estrutural (\f$ \alpha \f$)
    [radianos].
double _g;         ///< Aceleração gravitacional local (\f$ g \f$) [
    m/s²].

// --- Acoplamento de Domínio (Padrão Strategy) ---
ICurvasPermeabilidade* _modeloKr; ///< Ponteiro polimórfico para
    o modelo de permeabilidade relativa ativo.

50 public:
    /**
     * @brief Construtor padrão da calculadora física.
     *
     * @param mu_o Viscosidade do óleo.
     * @param mu_w Viscosidade da água.
     * @param modelo Ponteiro alocado para o modelo matemático de
     *   rocha-fluido.
     * @param g Constante de aceleração gravitacional (Default =
     *   9.80665 m/s²).
     */
60 CCalculadoraFluxoFracionario(double mu_o, double mu_w,
    ICurvasPermeabilidade* modelo, double g = 9.80665);

    /**
     * @brief Injeta e atualiza as propriedades físicas necessárias
     *   para o balanço de forças.
     *
     * @param mi_w Viscosidade da água [cP].
     * @param mi_o Viscosidade do óleo [cP].
     * @param rho_w Densidade da água [kg/m³].
     * @param rho_o Densidade do óleo [kg/m³].
     * @param k Permeabilidade absoluta [mD].
     * @param angulo Inclinação do estrato em graus (convertido para
     *   radianos internamente).
     * @param qt Vazão de injeção [m³/d].
     * @param A Área da seção [m²].

```

```

    */
    void setPropriedades(double mi_w, double mi_o, double rho_w,
        double rho_o, double k, double angulo, double qt, double A);
75
    /**
     * @brief Modifica dinamicamente a estratégia de permeabilidade
     *         relativa.
     * @param modelo Ponteiro para a nova interface
     *         ICurvasPermeabilidade.
     */
80 void setModeloPermeabilidade(ICurvasPermeabilidade* modelo);

    // --- Diagnósticos Dimensionais e Adimensionais ---

    /**
85  * @brief Avalia o critério macroscópico de estabilidade capilar
     *         (Rapoport-Leas).
     * \f$ N_{RL} = \left( \frac{\phi}{k} \right)^{1/2} \frac{L \cdot u_t \cdot \mu_w}{\sigma \cdot k_{rw}^0 \cdot \cos(\theta)} \f$
     * @param L Comprimento característico do sistema [m].
     * @param phi Porosidade efetiva da rocha [fração].
     * @param sigma Tensão interfacial água-óleo [mN/m ou dinas/cm].
90  * @return 0 escalar do número de Rapoport-Leas.
     */
    double calcularRapoportLeas(double L, double phi, double sigma)
        const;

    /**
95  * @brief Calcula a Razão de Mobilidade Global no Ponto Final (\f$ M^0 \f$).
     * \f$ M^0 = \frac{k_{rw}^0}{\mu_w} \cdot \frac{k_{ro}^0}{\mu_o} \f$
     * @return Valor adimensional indicando a estabilidade do
     *         deslocamento pistonado.
     */
    double calcularM0() const;
100

    /**
     * @brief Calcula o Número de Gravidade no Ponto Final (\f$ N_g^0 \f$).
     * Analisa a razão entre as forças gravitacionais e as forças
     * viscosas tratoras.
     * \f$ N_g^0 = \frac{k \cdot k_{ro}^0 \cdot \Delta \rho \cdot g}{u_t \cdot \mu_o} \f$
105  * @return 0 escalar adimensional associado à segregação
     *         gravitacional.
     */
    double calcularNg0() const;

```

```

// --- Resolução Numérica ---
110
/**
 * @brief Computa a taxa de fluxo fracionário instantâneo (\f$
 *       f_w \f$).
 * @param sw Saturação local de água no volume de controle.
 * @return Fração do fluxo, tipicamente entre 0 e 1.
115 */
double calcularFw(double sw) const;

/**
 * @brief Avalia a velocidade adimensional da frente de avanço (\f$
 *       f_w \f$).
120 * Utiliza o método de diferenças finitas centrais de alta ordem
 *       para evitar ruído.
 * @param sw Saturação local de água.
 * @return A derivada direcional \f$ \frac{df_w}{dS_w} \f$.
 */
double calcularDerivadaFw(double sw);

125
/**
 * @brief Discretiza a curva completa de fluxo fracionário.
 * @param passo Incremento de saturação (ex: $\Delta S_w = 0.01$)
 *
 * @return Um dicionário (map) relacionando ordenadamente
 *         Saturações a Fluxos Fracionários.
130 */
std::map<double, double> gerarCurvaCompleta(double passo) const;
};

#endif // CCalculadoraFluxoFracionario_H

```

Apresenta-se na listagem 7.15 a implementação dos balanços de força microscópicos, incluindo a determinação da instabilidade via número de Rapoport-Leas e diferenciação numérica central para a construção da tangente geométrica.

Listagem 7.15: Arquivo de Implementação CCalculadoraFluxoFracionario.cpp

```

/**
 * @file CCalculadoraFluxoFracionario.cpp
 * @brief Implementação da lógica termodinâmica e cálculos
 *       adimensionais do escoamento.
 * @author Fabiane da Silva Barros
5 * @date Janeiro 2026
 */

#include "CCalculadoraFluxoFracionario.h"
#include <stdexcept>
10#include <cmath>
#include <algorithm> // Para algoritmos de saturação

```

```

// Injeção da constante Pi (padrão C++)
#ifndef M_PI
15 #define M_PI 3.14159265358979323846
#endif

// --- Construtor Padrão ---
CCalculadoraFluxoFracionario::CCalculadoraFluxoFracionario(double
    mu_o, double mu_w, ICurvasPermeabilidade* modelo, double g)
20 : _mi_o(mu_o), _mi_w(mu_w), _g(g), _modeloKr(modelo)
{
    // Valores de inicialização segura (*fail-safe*)
    _rho_o = 800.0;
    _rho_w = 1000.0;
25 _k = 100.0;
    _angulo = 0.0;
    _qt = 1.0;
    _A = 1.0;
    _ut = 1.0;
30
    if (_modeloKr == nullptr) {
        throw std::runtime_error("Exceção Core: A Calculadora Física
            exige uma estratégia de permeabilidade não nula na
            inicialização.");
    }
}

35 // --- Configuração ---
void CCalculadoraFluxoFracionario::setPropriedades(double mi_w,
    double mi_o, double rho_w, double rho_o, double k, double angulo,
    double qt, double A) {
    _mi_w = mi_w;
    _mi_o = mi_o;
40 _rho_w = rho_w;
    _rho_o = rho_o;
    _k = k;

    // Transformação analítica: de Graus para Radianos
45 _angulo = angulo * (M_PI / 180.0);
    _qt = qt;
    _A = A;

    // Consequência cinemática imediata:
50 _ut = (qt / _A);
}

void CCalculadoraFluxoFracionario::setModeloPermeabilidade(
    ICurvasPermeabilidade* modelo) {
    if (modelo != nullptr) {
55 _modeloKr = modelo;

```

```

    }
}

// --- Cálculo do Critério de Rapoport-Leas ---
60 double CCalculadoraFluxoFracionario::calcularRapoportLeas(double L,
    double phi, double sigma) const {
    if (sigma <= 0 || _k <= 0 || phi <= 0 || _ut <= 0) return 0.0;

    // Obtém endpoint de água avaliando em Sw = 1.0
    double krw0 = _modeloKr->getKrw(1.0);

65
    double numerador = L * _ut * _mi_w;
    double raiz = std::sqrt(phi / _k);

    // Assume-se reservatório wet-water forte (cos(0) = 1.0)
70 double denominador = sigma * krw0 * 1.0;

    return raiz * (numerador / denominador);
}

75 // --- Razão de Mobilidade (M^0) ---
double CCalculadoraFluxoFracionario::calcularM0() const {
    double krw0 = _modeloKr->getKrw(1.0);
    double kro0 = _modeloKr->getKro(0.0);

80
    if (kro0 <= 0 || _mi_w <= 0) return 0.0;

    return (krw0 / _mi_w) / (kro0 / _mi_o);
}

85 // --- Número de Gravidade (Ng^0) ---
double CCalculadoraFluxoFracionario::calcularNg0() const {
    if (std::abs(_ut) < 1e-12 || _mi_o <= 0) return 0.0;

    double kro0 = _modeloKr->getKro(0.0);
90 double delta_rho = _rho_w - _rho_o;

    return (_k * kro0 * delta_rho * _g) / (_ut * _mi_o);
}

95 // --- Núcleo Analítico: Fluxo Fracionário (fw) ---
double CCalculadoraFluxoFracionario::calcularFw(double sw) const {

    // Clamping termodinâmico para as assíntotas do deslocamento
    if (sw <= 0.0) return 0.0;
100 if (sw >= 1.0) return 1.0;

    double krw = _modeloKr->getKrw(sw);
    double kro = _modeloKr->getKro(sw);

```

```

105 // Condição crítica de estrangulamento de fase (Choke condition)
    if (krw < 1e-15) return 0.0;
    if (kro < 1e-15) return 1.0;

    double termo_viscoso = (kro * _mi_w) / (krw * _mi_o);
110 double delta_rho = _rho_w - _rho_o;

    // Termo dependente da angulação topográfica
    double termo_gravitacional = (_k * kro * delta_rho * _g * std::
        sin(_angulo)) / (_ut * _mi_o);

115 // O retorno pode assumir fw > 1 (contra-fluxo de óleo decantando
    // em falhas geológicas íngremes)
    return (1.0 - termo_gravitacional) / (1.0 + termo_viscoso);
}

// --- Diferenciação Numérica (fw') ---
120 double CCalculadoraFluxoFracionario::calcularDerivadaFw(double sw) {
    // Abordagem de Diferenças Finitas Centrais (h = 10^-5 para
    // mitigar erro de truncamento)
    double h = 1e-5;
    double fw_mais = calcularFw(sw + h);
    double fw_menos = calcularFw(sw - h);
125 return (fw_mais - fw_menos) / (2.0 * h);
}

// --- Varredura Discreta ---
std::map<double, double> CCalculadoraFluxoFracionario::
    gerarCurvaCompleta(double passo) const {
130 std::map<double, double> curva;
    if (passo <= 0) passo = 0.01;

    int numPontos = static_cast<int>(1.0 / passo);

135 // Preenchimento contínuo do dicionário indexado
    for (int i = 0; i <= numPontos; ++i) {
        double sw = i * passo;
        if (sw > 1.0) sw = 1.0;
        curva[sw] = calcularFw(sw);
140 }

    // Força o contorno de saturação absoluta
    if (curva.find(1.0) == curva.end()) {
        curva[1.0] = calcularFw(1.0);
145 }

    return curva;
}

```

Apresenta-se na listagem \7.16 a documentação da classe do motor numérico primário, CSolver. O resolvidor baseia-se na aplicação escalar do Método das Características e não na tradicional aproximação finita em malha 3D.

Listagem 7.16: Arquivo de Cabeçalho CSolver.h

```

#ifndef CSOLVER_H
#define CSOLVER_H

/**
5  * @file CSolver.h
  * @brief Definição da classe CSolver, motor principal da solução de
    Buckley-Leverett.
  */

#include "CMalha.h"
10 #include "CCalculadoraFluxoFracionario.h"
#include "CWelge.h" // Dependência necessária para correção da frente
#include <vector>

/**
15 * @class CSolver
  * @brief Motor matemático do simulador de deslocamento imiscível 1D
    (Solução Analítica).
  *
  * A classe CSolver é a entidade coordenadora que aplica o Método das
    Características (MOC)
  * para solucionar a Equação a Derivadas Parciais (EDP) não-linear
    hiperbólica de Buckley-Leverett.
20 *
  * Diferentemente da modelagem por simulação numérica discreta (
    Diferenças Finitas),
  * o solver analítico não avança a solução por passos de tempo de
    forma iterativa ao longo de uma malha fixa.
  * Em vez disso, dada uma saturação de água  $S_w$ , ele mapeia
    diretamente a propagação de sua característica
  * ao longo do domínio espacial, calculando o perfil instantâneo para
    o tempo injetado  $t_D$  usando a relação cinemática:
25 *  $x_D(S_w, t_D) = \left( \frac{df_w}{dS_w} \right) \cdot t_D \cdot f$ 
  *
  * Adicionalmente, o solver orquestra a aplicação da condição de
    entropia macroscópica por meio
  * do algoritmo de Welge, garantindo o balanço de massa ao resolver a
    descontinuidade (frente de choque)
  * que previne soluções fisicamente impossíveis de múltipla valoração
    de saturação espacial.
30 *
  * @author Fabiane da Silva Barros
  * @date Janeiro 2026
  */

```

```

class CSolver {
35 private:
    CMalha* _malha;          ///< Ponteiro para a
        estrutura de dados onde o perfil temporal espacial será
        registrado.
    CCalculadoraFluxoFracionario* _calc; ///< Ponteiro associado à
        máquina de cálculo de estado termodinâmico e derivativo.
    CWelge* _welge;          ///< Ponteiro para a
        interface geométrica do método de Welge e construção do choque
        de Buckley-Leverett.

40 public:
    /**
     * @brief Construtor padrão da classe CSolver.
     * Aloca referências de ponteiros como nulas, exigindo injeção de
     * dependência explícita pela camada superior.
     */
45 CSolver();

    /**
     * @brief Destrutor da classe.
     * A classe opera sob o conceito de agregação forte de ponteiros
     * externos,
50 * logo, o destrutor não executa a deleção física de `_malha`, `_
    _calc` e `_welge` na memória heap.
     */
    ~CSolver();

    // --- Injeção de Dependências Analíticas ---

55 /**
     * @brief Atrela uma malha topológica ao Solver.
     * @param malha Ponteiro ativo contendo um objeto tipo CMalha.
     */
60 void setMalha(CMalha* malha);

    /**
     * @brief Determina o motor matemático base.
     * @param calc Ponteiro ativo contendo uma
        CCalculadoraFluxoFracionario instanciada.
65 */
    void setCalculadora(CCalculadoraFluxoFracionario* calc);

    /**
     * @brief Define o resolvidor da singularidade frontal de Buckley
        -Leverett.
70 * @param welge Ponteiro ativo da técnica construtiva de tangente
        de Welge.
     */

```

```

void setWelge(CWelge* welge);

// --- Núcleo Computacional ---
75
/**
 * @brief Computa e monta todo o campo espacial de Saturação de
 *        Água vs Posição Adimensional no reservatório.
 *
 * 0 algoritmo interno procede da seguinte forma:
80
 * 1. Limpa os instantes passados armazenados da CMalha.
 * 2. Varre linearmente o domínio de Saturações possíveis da
 *    injeção.
 * 3. Projeta a velocidade  $v(S_w) = f_w'$  e determina a
 *    distância propagada (Método das Características Analítico).
 * 4. Valida a instabilidade no contorno do perfil utilizando a
 *    técnica auxiliar CWelge (Tangente/Entropia).
 * 5. Corrige numericamente as velocidades irreais que causariam
 *    sobreposição da onda de avanço.
85
 * 6. Registra ordenadamente cada CCelula resultante.
 *
 * @param tempoInjetado Escala temporal (Adimensionalizada em PVI
 *        ou volumes porosos).
 * @param swi Saturação base (Irreduzível de água) presente
 *        originalmente no reservatório, para restrição construtiva do
 *        Choque Welge.
 * @param sw_max Teto da onda aquosa (1 - Saturação residual
 *        oleica limitadora).
90
 */
void calcularPerfilSaturacao(double tempoInjetado, double swi,
                             double sw_max);
};

#endif // CSOLVER_H

```

Apresenta-se na listagem 7.17 a implementação da malha geradora, a detecção da ruptura da sobreposição de perfil (*breakthrough*) através do método geométrico, e a reconstrução corretiva da descontinuidade da frente de avanço.

Listagem 7.17: Arquivo de Implementação CSolver.cpp

```

/**
 * @file CSolver.cpp
 * @brief Implementação analítica das leis de conservação pelo método
 *        das características.
 * @author Fabiane da Silva Barros
5
 * @date Janeiro 2026
 */

#include "CSolver.h"
#include <cmath>
10#include <vector>

```

```

// --- Construtor e Destrutor ---
CSolver::CSolver() : _malha(nullptr), _calc(nullptr), _welge(nullptr)
{
}

15 CSolver::~CSolver() {
    // Gestão de Ciclo de Vida: Deleção delegada ao CSimulador
}

20 // --- Setters de Estruturação ---
void CSolver::setMalha(CMalha* malha) {
    _malha = malha;
}

25 void CSolver::setCalculadora(CCalculadoraFluxoFracionario* calc) {
    _calc = calc;
}

void CSolver::setWelge(CWelge* welge) {
30     _welge = welge;
}

// --- Algoritmo Base de MOC ---
void CSolver::calcularPerfilSaturacao(double tempoInjetado, double
    swi, double sw_max) {
35
    // Verificação de Seguridade Computacional: Checagem contra
    nullptr
    if (!_malha || !_calc || !_welge) return;

    _malha->limpar();
40     _malha->setTempoAtual(tempoInjetado);

    // Resolução Fina Discreta Analítica
    double passoSw = 0.005;
    int numPontos = static_cast<int>(1.0 / passoSw);

45     std::vector<double> vecSw;
    std::vector<double> vecVel;
    std::vector<double> vecPos;

50     vecSw.reserve(numPontos + 1);
    vecVel.reserve(numPontos + 1);
    vecPos.reserve(numPontos + 1);

    // Passo 1: Construção do Perfil Bruto de Múltipla Valoração (
    Avanço Livre de Ondas)
55     for (int i = 0; i <= numPontos; ++i) {

```

```

double sw = i * passoSw;
if (sw > 1.0) sw = 1.0;

// A velocidade de cada saturação específica independe do
// tempo injetado na analítica
double vel = _calc->calcularDerivadaFw(sw);

// Posição adimensional da onda: (dfw / dSw) * tempo
double pos = vel * tempoInjetado;

vecSw.push_back(sw);
vecVel.push_back(vel);
vecPos.push_back(pos);
}

// Passo 2: Verificação do Front de Choque Instável
// Avalia a necessidade termodinâmica do choque limitando ao
// espectro de móveis
bool choqueEncontrado = _welge->calcularTangente(_calc, swi,
sw_max);

// Atributos definidores da frente estabilizada
double swFrente = _welge->getSwFrente();
double velocidadeChoque = _welge->getInclinacaoChoque();

// Passo 3: Identificação Topológica da Dupla Valoração (Efeito "
// Quebra de Onda" de Choque)
bool cruzamento = false;
double posSwFrente = 0.0;

for (size_t i = 0; i < vecSw.size(); ++i) {
    if (vecSw[i] >= swFrente) {
        posSwFrente = vecPos[i];
        break;
    }
}

// Checa se há porções do vetor que geometricamente estão a
// frente da linha crítica (sobreposição não-física)
for (size_t i = 0; i < vecSw.size(); ++i) {
    if (vecSw[i] < swFrente && vecPos[i] > posSwFrente + 1e-12) {
        cruzamento = true;
        break;
    }
}

// Passo 4: Filtragem de Entropia - Remoção dos perfis lentos "
// comidos" pela onda frontal de Choque (Pistonamento ideal)
for (size_t i = 0; i < vecSw.size(); ++i) {

```

```

double sw = vecSw[i];
double pos = vecPos[i];
double vel = vecVel[i];

if (choqueEncontrado && cruzamento && sw < swFrente) {
    pos = velocidadeChoque * tempoInjetado; // Fixa todas
        essas saturações lentas na parede vetorial do choque
        vertical
    vel = velocidadeChoque;
}

_malha->adicionarCelula(CCelula(sw, pos, vel));
}

// Passo 5: Restauração da Integridade do Plot Vectorial
_malha->ordenarPorPosicao();
}

```

Apresenta-se na listagem 7.18 o cabeçalho da classe responsável por implementar o método geométrico de H. J. Welge (1952), garantindo a imposição analítica da descontinuidade da frente de avanço para corrigir a saturação multivalorada originada na solução pura das características.

Listagem 7.18: Arquivo de Cabeçalho CWelge.h

```

#ifndef CWELGE_H
#define CWELGE_H

/**
5 * @file CWelge.h
  * @brief Definição da classe CWelge e da construção geométrica da
    frente de choque.
  */

#include "CCalculadoraFluxoFracionario.h"

10 /**
  * @class CWelge
  * @brief Implementa o método analítico da Tangente de Welge (1952).
  *
15 * Responsável por determinar a saturação da frente de choque (\f$ S_{wf} \f$) e a saturação
  * média (\f$ \bar{S}_w \f$) da zona varrida atrás da frente de
  * avanço.
  *
  * O Método das Características aplicado à equação de Buckley-
  * Leverett pura gera
  * saturações multivaloradas (o chamado perfil "em S"), o que viola a
  * conservação
20 * de massa e o princípio da unicidade física. A construção de Welge
  * impõe uma

```

```

* descontinuidade brusca (choque) que satisfaz a condição de
  entropia de Oleinik.
*
* Geometricamente, o algoritmo localiza o ponto de tangência
  traçando retas secantes
* a partir do ponto de saturação inicial  $(S_{wi}, f_w(S_{wi}))$  \
   $f$  até atingir a inclinação máxima:
25 *  $\left. \frac{df_w}{dS_w} \right|_{S_{wf}} = \frac{f_w(S_{wf}) - f_w(S_{wi})}{S_{wf} - S_{wi}}$  \f$
*
* @author Fabiane da Silva Barros
* @date Janeiro 2026
*/
30 class CWelge {
private:
    double _swFrente;          ///< Saturação na frente de choque ( $S_{wf}$  \f$).
    double _swMedia;           ///< Saturação média da água atrás da
                                frente geométrica ( $\bar{S}_w$  \f$).
    double _swInicial;         ///< Saturação inicial ou irreduzível
                                ancorada no reservatório ( $S_{wi}$  \f$).
35 double _inclinacaoMax;      ///< Máxima inclinação secante encontrada,
                                equivalente à velocidade do choque.

    // Parâmetros operacionais sistêmicos
    double _vazaoInjecao;      ///< Vazão de injeção externa do fluido
                                deslocante.
    double _area;              ///< Área de seção transversal do fluxo
                                matricial.

40 public:
    /**
     * @brief Construtor padrão da classe CWelge.
     * Inicializa os atributos termodinâmicos e geométricos com valor
     nulo.
45 */
    CWelge();

    /**
     * @brief Destrutor virtual da classe.
50 */
    virtual ~CWelge();

    /**
     * @brief Define o ponto de ancoragem esquerdo da tangente.
55 * @param swi Saturação inicial de água ( $S_{wi}$  \f$),
        usualmente igual a  $S_{wirr}$  \f$.
    */
    void setSwInicial(double swi);

```

```

/**
60  * @brief Executa o algoritmo de varredura analítica para a
    *      detecção da tangente de choque.
    *
    * O algoritmo varre discretamente a curva de fluxo fracionário
    * gerada pela calculadora,
    * avaliando a derivada direcional da reta secante ancorada no
    * ponto inicial.
    * O ponto que maximiza essa inclinação define a singularidade de
    * choque.
65  *
    * @param calc Ponteiro em execução para a calculadora física.
    * @param swi Saturação de água inicial fixada.
    * @param sw_max Teto da saturação de água ( $1 - S_{or}$ ).
    * @return true se uma tangente com inclinação fisicamente válida
    *         ( $> 10^{-6}$ ) for encontrada.
70  */
bool calcularTangente(CCalculadoraFluxoFracionario* calc, double
    swi, double sw_max);

// --- Getters ---

75  /**
    * @brief Recupera a saturação exata onde ocorre a ruptura do
    *      perfil de avanço.
    * @return O valor de  $S_{wf}$ .
    */
double getSwFrente() const;
80
    /**
    * @brief Calcula e retorna a saturação média retida no meio
    *      poroso.
    * Extrapolação baseada na intersecção da tangente de choque com
    * a assíntota  $f_w = 1$ .
    * @return O valor de  $\bar{S}_w$ .
85  */
double getSwMedia() const;

    /**
    * @brief Recupera a inclinação da reta, que representa
    *      fisicamente a velocidade adimensional do choque.
90  * @return O escalar  $\frac{df_w}{dS_w}$  avaliado no choque
    *
    */
double getInclinacaoChoque() const;
};

95 #endif // CWELGE_H

```

Apresenta-se na listagem 7.19 a implementação algorítmica iterativa que varre a curva de fluxo fracionário, ancorada à saturação de água inicial, para encontrar a máxima inclinação (velocidade de choque) e projetar a recuperação final através do cálculo da saturação média ($\bar{S}_w$).

Listagem 7.19: Arquivo de Implementação CWelge.cpp

```

/**
 * @file CWelge.cpp
 * @brief Implementação dos cálculos de descontinuidade da frente de
 * saturação.
 * @author Fabiane da Silva Barros
5 * @date Janeiro 2026
 */

#include "CWelge.h"
#include <cmath>
10#include <algorithm>

// --- Construtor ---
CWelge::CWelge()
    : _swFrente(0.0), _swMedia(0.0), _swInicial(0.0), _inclinacaoMax
      (0.0), _vazaoInjecao(0.0), _area(0.0)
15{
}

// --- Destrutor ---
CWelge::~CWelge() {
20}

// --- Setter de Condição Inicial ---
void CWelge::setSwInicial(double swi) {
    _swInicial = swi;
25}

// --- Algoritmo Principal de Varredura ---
bool CWelge::calcularTangente(CCalculadoraFluxoFracionario* calc,
    double swi, double sw_max) {

30    // Reset de segurança do estado geométrico
    _swInicial = swi;
    _inclinacaoMax = 0.0;
    _swFrente = swi;

35    double fw_swi = calc->calcularFw(swi);

    // Varredura da secante ancorada fortemente em (Swi, fw(Swi))
    // O passo de discretização (0.001) define a precisão da
    localização do choque
    for (double s = swi + 0.001; s <= sw_max; s += 0.001) {

```

```

40      // Avaliação do coeficiente angular da secante
      double slope = (calc->calcularFw(s) - fw_swi) / (s - swi);

      // A condição de máximo garante o cumprimento da Condição de
      Entropia
45      if (slope > _inclinacaoMax) {
          _inclinacaoMax = slope;
          _swFrente = s;
      }
  }

50  // Validação da viabilidade física da injeção (inclinação não
  nula)
  if (_inclinacaoMax > 1e-6) {

      // O balanço de materiais de Welge permite encontrar a
      Saturação Média
55  // através do prolongamento da tangente até o eixo fw = 1.0
      double fw_frente = calc->calcularFw(_swFrente);
      _swMedia = _swFrente + (1.0 - fw_frente) / _inclinacaoMax;

      return true;
60  }

      // Falha termodinâmica ou deslocamento impossível
      _swMedia = swi;
      return false;
65 }

// --- Getters de Diagnóstico ---

double CWelge::getSwFrente() const {
70     return _swFrente;
}

double CWelge::getSwMedia() const {
    return _swMedia;
75 }

double CWelge::getInclinacaoChoque() const {
    return _inclinacaoMax;
}

```

Apresenta-se na listagem 7.20 o cabeçalho da classe CSimulador. Esta classe foi projetada para atuar como o Controlador (*Controller*) do padrão MVC, mitigando o acoplamento forte entre a física complexa do método analítico e os elementos gráficos da interface do sistema de injeção de fluidos .

Listagem 7.20: Arquivo de Cabeçalho CSimulador.h

```

#ifndef CSIMULADOR_H
#define CSIMULADOR_H

/**
5  * @file CSimulador.h
  * @brief Definição da classe CSimulador, o controlador central da
    arquitetura MVC.
  */

#include "ICurvasPermeabilidade.h"
10 #include "CCalculadoraFluxoFracionario.h"
#include "CSolver.h"
#include "CWelge.h"
#include "CMalha.h"
#include <string>

15 /**
  * @class CSimulador
  * @brief Controlador Principal do Sistema (Padrão Controller).
  *
20  * Esta classe atua como a fachada unificada (Interface) entre a
    Camada de Apresentação (UI, MainWindow)
  * e o núcleo físico-matemático de Engenharia de Reservatórios.
  *
  * A sua responsabilidade primária é orquestrar o ciclo de vida e
    garantir a sincronização termodinâmica
  * antes de permitir a execução do Solver analítico.
25  *
  * Funcionalidades principais:
  * 1. Armazenar os parâmetros de entrada operacionais, PVT e de rocha
    .
  * 2. Gerir rigidamente a memória alocada dos objetos em *heap* (
    Solver, Malha, Estratégia de Kr).
  * 3. Validação preliminar do critério de Rapoport-Leas.
30  *
  * @author Fabiane da Silva Barros
  * @date Janeiro 2026
  */
class CSimulador {
35 private:
    // --- Objetos do Núcleo Matemático (Composição/Agregação) ---
    CSolver* _solver;                ///< Ponteiro para o
    motor de resolução de EDPs (MOC).
    CCalculadoraFluxoFracionario* _calculadora; ///< Ponteiro para a
    calculadora do fluxo fracionário ($f_w$).
    CWelge* _welge;                 ///< Ponteiro para a
    lógica geométrica da onda de avanço.
40  CMalha* _malha;                 ///< Ponteiro para a

```

```

        malha de estados espaciais.
ICurvasPermeabilidade* _modeloPermeabilidade; ///< Ponteiro
        polimórfico (Strategy) em execução.

// --- Parâmetros Físicos Estruturais ---
double _comprimento;    ///< Comprimento total avaliado no
        reservatório ($L$) [m].
45 double _area;          ///< Área de escoamento transversal ($A$)
        [m²].
double _porosidade;      ///< Espaço poroso efetivo ($\phi$) [
        fração].
double _angulo;          ///< Mergulho do estrato geológico [graus
        ].
double _vazaoInjecao;    ///< Vazão de injeção externa do fluido
        molhante ($Q_t$) [m³/s ou m³/d].

50 // --- Propriedades PVT dos Fluidos ---
double _mi_o;            ///< Viscosidade dinâmica da fase óleo ($
        \mu_o$) [cP].
double _mi_w;            ///< Viscosidade dinâmica da fase água ($
        \mu_w$) [cP].
double _rho_o;           ///< Densidade da fase óleo ($\rho_o$) [
        kg/m³].
double _rho_w;           ///< Densidade da fase água ($\rho_w$) [
        kg/m³].

55 // --- Propriedade Petrofísica Absoluta ---
double _k;               ///< Permeabilidade escalar primária ($k$
        ) [mD].

public:
60 /**
    * @brief Construtor padrão da classe CSimulador.
    * Instancia o modelo empírico padrão (Corey) e inicializa toda a
        cascata de ponteiros matemáticos.
    */
    CSimulador();

65 /**
    * @brief Destrutor virtual da classe.
    * Desaloca de forma segura a memória residual da malha, dos
        modelos e do resolvedor.
    */
70 ~CSimulador();

// --- Configuração e Validação de Dados (Setters) ---

/**
75 * @brief Injeta os dados geométricos e petrofísicos básicos do

```

```

        modelo geológico.
    * @param L Comprimento [m].
    * @param A Área [m²].
    * @param phi Porosidade efetiva.
    * @param angulo Inclinação do sistema gravitacional.
80  * @param vazao Taxa total de injeção volumétrica.
    * @throws std::invalid_argument Se os parâmetros escalares
        essenciais forem nulos ou negativos.
    */
    void setDadosReservatorio(double L, double A, double phi, double
        angulo, double vazao);

85  /**
    * @brief Injeta as propriedades de transporte dos fluidos
        bifásicos.
    * @param mi_o Viscosidade óleo.
    * @param mi_w Viscosidade água.
    * @param rho_o Densidade óleo.
90  * @param rho_w Densidade água.
    */
    void setFluidos(double mi_o, double mi_w, double rho_o, double
        rho_w);

    /**
95  * @brief Injeta a condutividade escalar principal da rocha.
    * @param k Permeabilidade absoluta [mD].
    */
    void setPermeabilidade(double k);

100  /**
    * @brief Sobrescreve dinamicamente a estratégia de
        permeabilidade relativa (Padrão Strategy).
    * @note A gestão de memória passa a ser responsabilidade da
        classe. Ponteiros anteriores são destruídos.
    * @param modelo O novo ponteiro de política de permeabilidade
        instanciado (Ex: LET, Tabelado).
    */
105  void setModeloPermeabilidade(ICurvasPermeabilidade* modelo);

    // --- Execução da Dinâmica ---

    /**
110  * @brief Despoleta o motor analítico de simulação para um
        determinado volume de injeção.
    *
    * Este método propaga os parâmetros armazenados em interface
        para as calculadoras
    * de baixo nível e dispara o pipeline de cálculo espacial do
        Método das Características (MOC).

```

```

115  * @param tempoInjetado Tempo acumulado de injeção, tipicamente
    * em volumes porosos.
    * @param swi Saturação de água residual de ancoragem inicial.
    * @param sw_max Teto da onda final pós-ruptura.
    */
    void executarSimulacao(double tempoInjetado, double swi, double
        sw_max);

120  // --- Recuperação de Contextos (Getters) ---

    CMalha* getMalha() const;
    CWelge* getWelge() const;
125  CCalculadoraFluxoFracionario* getCalculadora() const;
    ICurvasPermeabilidade* getModeloPermeabilidade() const;
    double getComprimento() const;
    double getPorosidade() const;
};

130 #endif // CSIMULADOR_H

```

Apresenta-se na listagem 7.21 o arquivo de implementação do controlador, destacando as defesas rigorosas e destrutores seguros de alocação de memória e o sincronismo dos cálculos da calculadora com a malha.

Listagem 7.21: Arquivo de Implementação CSimulador.cpp

```

/**
 * @file CSimulador.cpp
 * @brief Implementação da integração e gestão do fluxo MVC analítico
 *
 * @author Fabiane da Silva Barros
5  * @date Janeiro 2026
 */

#include "CSimulador.h"
#include "CCurvasPermeabilidadeCorey.h"
10 #include <iostream>
#include <stdexcept>

// --- Construtor ---
CSimulador::CSimulador()
15 : _comprimento(100.0), _area(1.0), _porosidade(0.2), _angulo(0.0)
    , _vazaoInjecao(1.0),
    _mi_o(1.0), _mi_w(1.0), _rho_o(800.0), _rho_w(1000.0), _k(1.0e
        -12)
{
    // Construção inicial da dependência forte (modelo padrão)
    _modeloPermeabilidade = new CCurvasPermeabilidadeCorey();

20  // Cascata de inicialização das instâncias dependentes

```

```

        _calculadora = new CCalculadoraFluxoFracionario(_mi_o, _mi_w,
            _modeloPermeabilidade);
        _welge = new CWelge();
        _malha = new CMalha();
25    _solver = new CSolver();

        // Acoplamento da malha de comunicação do motor
        _solver->setMalha(_malha);
        _solver->setCalculadora(_calculadora);
30    _solver->setWelge(_welge);
    }

    // --- Destrutor Seguro ---
    CSimulador::~CSimulador() {
35        // Destruição LIFO (Last-In-First-Out) para garantir segurança de
            ponteiros nulos
        if (_solver) delete _solver;
        if (_malha) delete _malha;
        if (_welge) delete _welge;
        if (_calculadora) delete _calculadora;
40

        // A política de interface de curvas só pode ser descartada no
            fim
        if (_modeloPermeabilidade) delete _modeloPermeabilidade;
    }

45 // --- Delegação de Configurações ---

    void CSimulador::setDadosReservatorio(double L, double A, double phi,
        double angulo, double vazao) {
        if (L <= 0 || A <= 0 || phi <= 0 || vazao <= 0) {
            throw std::invalid_argument("Exceção_␣Simulador_␣Topologia_␣
                espacial_␣fisicamente_␣inconsistente_␣(valores_␣nulos).");
50        }
        _comprimento = L;
        _area = A;
        _porosidade = phi;
        _angulo = angulo;
55    _vazaoInjecao = vazao;
    }

    double CSimulador::getComprimento() const {
        return _comprimento;
60    }

    double CSimulador::getPorosidade() const {
        return _porosidade;
    }
65

```

```

void CSimulador::setFluidos(double mi_o, double mi_w, double rho_o,
double rho_w) {
    if (mi_o <= 0 || mi_w <= 0) {
        throw std::invalid_argument("Exceção_Simulador:_As_
propriedades_termodinâmicas_(Viscosidade)_não_podem_ser_
nulas.");
    }
70  _mi_o = mi_o;
    _mi_w = mi_w;
    _rho_o = rho_o;
    _rho_w = rho_w;
}

75  void CSimulador::setPermeabilidade(double k) {
    if (k <= 0) {
        throw std::invalid_argument("Exceção_Simulador:_
Permeabilidade_não_convergente_para_o_modelo.");
    }
80  _k = k;
}

void CSimulador::setModeloPermeabilidade(ICurvasPermeabilidade*
modelo) {
    if (modelo == nullptr) return;
85  // Prevenção crítica contra vazamento de memória ram (Memory Leak
    )
    if (_modeloPermeabilidade != nullptr) {
        delete _modeloPermeabilidade;
    }
90  _modeloPermeabilidade = modelo;

    // Repasse da alteração contextual à calculadora termodinâmica
    if (_calculadora) {
95  _calculadora->setModeloPermeabilidade(_modeloPermeabilidade);
    }
}

// --- Chamada Principal de Execução ---
100 void CSimulador::executarSimulacao(double tempoInjetado, double swi,
double sw_max) {
    if (!_modeloPermeabilidade) {
        throw std::runtime_error("Exceção_Simulador:_Impossível_
realizar_varredura_sem_um_algoritmo_de_curvas_definido.");
    }
105  const double tensao_interfacial = 0.03; // N/m típico para

```

```

    sistemas rocha-óleo-água

    // Check-sum analítico: O número de Rapoport-Leas avalia se as
    // forças capilares
    // estabilizam a onda, ou se o deslocamento requer correção
    // difusiva
110    double nrl = _calculadora->calcularRapoportLeas(_comprimento,
        _porosidade, tensao_interfacial);
    if (nrl < 3.0) {
        std::cerr << "AVISO_GEOLÓGICO: Número de Rapoport-Leas (" <<
            nrl << ") < 3.0. A assunção de forças viscosas dominantes
            pode ser imprecisa neste reservatório." << std::endl;
    }

115    // Sincroniza estado de massa e geometria para a GPU virtual
    _calculadora->setPropriedades(_mi_w, _mi_o, _rho_w, _rho_o, _k,
        _angulo, _vazaoInjecao, _area);

    // Delega os esforços resolutivos para a rotina abstrata
    _solver->calcularPerfilSaturacao(tempoInjetado, swi, sw_max);
120}

// --- Getters Internos ---

CMalha* CSimulador::getMalha() const {
125    return _malha;
}

CWelge* CSimulador::getWelge() const {
    return _welge;
130}

CCalculadoraFluxoFracionario* CSimulador::getCalculadora() const {
    return _calculadora;
}

135 ICurvasPermeabilidade* CSimulador::getModeloPermeabilidade() const {
    return _modeloPermeabilidade;
}

```

Apresenta-se na listagem 7.22 a documentação da classe CRelatorio. Esta classe demonstra técnicas avançadas de programação, ao orquestrar a conversão de objetos vetoriais QPixmap para cadeias criptografadas em Base64, injetadas internamente na formatação HTML5 para renderização de PDF nativo.

Listagem 7.22: Arquivo de Cabeçalho CRelatorio.h

```

#ifndef CRELATORIO_H
#define CRELATORIO_H

```

```

/**

```

```

5  * @file CRelatorio.h
  * @brief Definição da classe CRelatorio para exportação de dados
    analíticos.
  */

#include <string>
10#include <QString>
#include <QPixmap>

/**
 * @class CRelatorio
15 * @brief Gerenciador de relatórios técnicos e exportação PDF.
 *
 * Esta classe é responsável por agregar os dados estáticos, os
 * diagnósticos adimensionais,
 * os resultados numéricos do choque de Welge e os gráficos gerados
 * pela interface.
 * O motor interno utiliza a sintaxe HTML5 e CSS3 inline para
 * construir o layout do documento.
20 * Posteriormente, o relatório é renderizado vetorizado em formato
   PDF através
 * das classes nativas QTextDocument e QPdfWriter do Qt.
 *
 * @author Fabiane da Silva Barros
 * @date Janeiro 2026
25 */
class CRelatorio {
private:
    std::string _textoRelatorio; ///< String cumulativa contendo o
    código fonte HTML/CSS do documento.

30public:
    /**
     * @brief Construtor padrão da classe CRelatorio.
     * Inicializa a variável de texto e injeta a folha de estilo CSS
     (limpar).
     */
35    CRelatorio();

    /**
     * @brief Destrutor virtual.
     */
40    ~CRelatorio();

    /**
     * @brief Reinicializa o documento HTML.
     * Apaga dados antigos e injeta o cabeçalho base contendo os
     estilos CSS,
45    * fontes (Segoe UI) e formatações rigorosas de tabelas e

```

```

    legendas (Padrão ABNT).
*/
void limpar();

/**
50  * @brief Registra a tabela de propriedades estáticas do
    reservatório.
    * @param comp Comprimento (L).
    * @param area Área transversal (A).
    * @param phi Porosidade efetiva.
    * @param k Permeabilidade absoluta.
55  * @param angulo Mergulho estrutural.
    * @param mio Viscosidade do óleo.
    * @param miw Viscosidade da água.
    */
void gerarCabecalho(double comp, double area, double phi, double
    k, double angulo, double mio, double miw);

60  /**
    * @brief Registra os adimensionais de diagnóstico e emite o
    aviso do critério capilar.
    * @param nrl Número de Rapoport-Leas.
    * @param m Razão de mobilidade no ponto final ( $M^0$ ).
65  * @param ng Número de gravidade (Ng).
    */
void registrarDiagnosticoFisico(double nrl, double m, double ng);

/**
70  * @brief Destaca os resultados críticos da técnica de Welge no
    instante de ruptura (Breakthrough).
    * @param swFrente Saturação da água na frente de choque.
    * @param swMedia Saturação média da zona varrida.
    * @param tempoRuptura Tempo de injeção (PVI) exato no momento do
    breakthrough.
    * @param edBt Eficiência de deslocamento volumétrico na ruptura.
75  */
void registrarResultadoWelge(double swFrente, double swMedia,
    double tempoRuptura, double edBt);

/**
    * @brief Registra o status temporal do instante em que o
    relatório foi gerado.
80  * @param tempo Tempo atual de injeção analisado (PVI).
    * @param fatorRecuperacao Recuperação atual baseada no volume
    poroso.
    */
void registrarEficiencia(double tempo, double fatorRecuperacao);

85  /**

```

```

    * @brief Converte e embuti os gráficos do QCustomPlot
      diretamente no documento HTML.
    *
    * Os objetos `QPixmap` são codificados em memória para fluxos
      binários PNG e,
    * em seguida, transformados em strings Base64. Isso permite
      injetar a imagem
90  * como uma URI de dados (`<img src='data:image/png;base64
      ,... '>`) sem depender
    * de arquivos temporários no disco rígido.
    *
    * @param pixFluxo Captura de tela da curva do fluxo fracionário.
    * @param pixSaturacao Captura de tela do perfil de saturação de
      água.
95  * @param pixEficiencia Captura de tela do histórico de
      eficiência.
    */
void registrarGraficos(const QPixmap& pixFluxo, const QPixmap&
    pixSaturacao, const QPixmap& pixEficiencia);

/**
100  * @brief Anexa anotações customizadas ao fim do relatório.
    * @param notaHtml String contendo texto ou marcação HTML extra
      inserida pelo usuário.
    */
void adicionarNota(const std::string& notaHtml);

105 /**
    * @brief Renderiza e exporta o arquivo PDF em disco.
    *
    * Finaliza a sintaxe HTML e utiliza o motor de renderização `
      QTextDocument` em conjunto
    * com o `QPdfWriter` para formatar e paginar o relatório no
      padrão ISO A4.
110  *
    * @param caminho Caminho absoluto alvo para gravação do arquivo
      `.pdf`.
    * @return true se o arquivo for gravado com sucesso, false em
      caso de falha de I/O.
    */
bool exportarParaPDF(const std::string& caminho);
115};

#endif // CRELATORIO_H

```

Apresenta-se na listagem 7.23 o arquivo de implementação, detalhando as rotinas de agregação *stringstream* para as casas decimais e a formatação das tabelas e legendas de acordo com o padrão estético ABNT.

Listagem 7.23: Arquivo de Implementação CRelatorio.cpp

```

/**
 * @file CRelatorio.cpp
 * @brief Implementação aprimorada da geração de relatórios com
 *        embutimento de gráficos.
 * @author Fabiane da Silva Barros
5  * @date Janeiro 2026
 * /

#include "CRelatorio.h"
#include <sstream>
10#include <iomanip>
#include <QTextDocument>
#include <QPdfWriter>
#include <QPageSize>
#include <QBuffer>
15#include <QByteArray>
#include <QPageLayout>

// --- Construtor e Destrutor ---
CRelatorio::CRelatorio() {
20    limpar();
}

CRelatorio::~CRelatorio() {}

25// --- Estruturação do HTML e CSS ---
void CRelatorio::limpar() {
    _textoRelatorio = "<html><head><style>";
    _textoRelatorio += "body{font-family: 'Segoe UI', Arial, sans-
        serif; color: #2c3e50; line-height: 1.4; padding: 10px;}";
    _textoRelatorio += "h1{color: #1a5276; text-align: center; font-
        -size: 22pt; margin-bottom: 5px;}";
30    _textoRelatorio += "h2{color: #2e86c1; font-size: 14pt; border-
        bottom: 2px solid #5dade2; padding-bottom: 4px; margin-top: 25
        px; margin-bottom: 10px;}";
    _textoRelatorio += "h3{color: #34495e; font-size: 10pt; margin-
        top: 5px; margin-bottom: 25px; text-align: center; font-style:
        italic;}";
    _textoRelatorio += "table{width: 100%; border-collapse:
        collapse; margin-top: 5px; margin-bottom: 15px;}";
    _textoRelatorio += "th, td{border: 1px solid #b3b6b7; padding:
        8px 10px; font-size: 10pt; text-align: left;}";
    _textoRelatorio += "th{background-color: #ebf5fb; color: #1
        b4f72; font-weight: bold;}";
35    _textoRelatorio += ".card-highlight{background-color: #f4f6f7;
        border-left: 5px solid #2e86c1; padding: 12px; margin-vertical
        : 15px;}";
    _textoRelatorio += ".warning-box{color: #922b21; background-

```

```

        color: #fadbd8; border: 1px solid #78281f; padding: 10px; font-
        -size: 10pt; font-weight: bold; margin-bottom: 15px; }";
    _textoRelatorio += ".success-box { color: #196f3d; background-
        color: #d4efdf; border: 1px solid #145a32; padding: 10px; font-
        -size: 10pt; font-weight: bold; margin-bottom: 15px; }";
    _textoRelatorio += ".metric-value { font-family: 'Courier New',
        monospace; font-weight: bold; color: #1b4f72; }";
    _textoRelatorio += ".center-plot { text-align: center; margin-top
        : 10px; margin-bottom: 15px; }";
40 _textoRelatorio += "</style></head><body>";

    _textoRelatorio += "<h1>Relatório Técnico de Desempenho de
        Injeção</h1>";
    _textoRelatorio += "<p align='center' style='font-size: 11pt;
        color: #7f8c8d;'><b>Simulador Analítico de Buckley-Leverett -
        Método das Características</b></p>";
    _textoRelatorio += "<hr size='2' color='#1a5276'>";
45 }

// --- Geração de Tabelas Analíticas ---
void CRelatorio::gerarCabecalho(double comp, double area, double phi,
    double k, double angulo, double mio, double miw) {
    std::stringstream ss;
50 ss << std::fixed << std::setprecision(2);

    ss << "<h2>1. Parâmetros Estáticos do Meio Poroso e Fluidos</h2>"
        ;
    ss << "<table>";
    ss << "<tr><th width='60%'>Propriedade Operacional / Geométrica</
        th><th width='20%'>Valor</th><th width='20%'>Unidade</th></tr>"
        ;
55 ss << "<tr><td>Comprimento Geométrico do Reservatório (L)</td><td
        class='metric-value'>" << comp << "</td><td>m</td></tr>";
    ss << "<tr><td>Área da Seção Transversal Hidráulica (A)</td><td
        class='metric-value'>" << area << "</td><td>m²</td></tr>";
    ss << "<tr><td>Porosidade Efetiva (&phi;)</td><td class='metric-
        value'>" << phi * 100.0 << "</td><td>%</td></tr>";
    ss << "<tr><td>Permeabilidade Absoluta da Rocha (k)</td><td class
        ='metric-value'>" << k << "</td><td>mD</td></tr>";
    ss << "<tr><td>Ângulo de Inclinação Estrutural (&alpha;)</td><td
        class='metric-value'>" << angulo << "</td><td>graus</td></tr>"
        ;
60 ss << "<tr><td>Viscosidade Dinâmica da Água (&mu; <sub>w</sub>)</
        td><td class='metric-value'>" << miw << "</td><td>cP</td></tr>"
        ;
    ss << "<tr><td>Viscosidade Dinâmica do Óleo (&mu; <sub>o</sub>)</
        td><td class='metric-value'>" << mio << "</td><td>cP</td></tr>"
        ;
    ss << "</table>";

```

```

        _textoRelatorio += ss.str();
65 }

void CRelatorio::registrarDiagnosticoFisico(double nrl, double m,
double ng) {
    std::stringstream ss;
    ss << std::fixed << std::setprecision(3);
70

    ss << "<h2>2. Diagnóstico de Forças Balísticas e Adimensionais</h2>";

    if (nrl < 3.0) {
        ss << "<div class='warning-box'>AVISO DE REGIME FLUIDODINÂMICO:<br>"
75         << "0 Número de Rapoport-Leas calculado (" << nrl << ") está abaixo do limite crítico estabelecido pela literatura (3.0).<br>"
        << "Efeitos capilares de fim de curso (end-effect) podem introduzir dispersão no perfil real de saturação.</div>"
        << "</div>";
    } else {
        ss << "<div class='success-box'>VALIDAÇÃO DE REGIME:<br>"
        << "Critério macroscópico de Rapoport-Leas satisfeito ("
        << nrl << " >= 3.0).<br>"
80         << "As forças viscosas dominam o escoamento, garantindo a aplicabilidade estrita da teoria de Buckley-Leverett.</div>";
        << "</div>";

        ss << "<table>";
        ss << "<tr><th width='40%'>Grandeza Adimensional</th><th width='20%'>Valor</th><th width='40%'>Significado Físico</th></tr>";
        ;
85         ss << "<tr><td>Razão de Mobilidade Total (M)</td><td class='metric-value'>" << m << "</td><td>Eficiência do empuxo viscoso do fluido injetado</td></tr>";
        ss << "<tr><td>Número de Gravidade (N<sub>g</sub>)</td><td class='metric-value'>" << ng << "</td><td>Relação entre forças gravitacionais e forças viscosas</td></tr>";
        ss << "</table>";

        _textoRelatorio += ss.str();
90 }

void CRelatorio::registrarResultadoWelge(double swFrente, double swMedia, double tempoRuptura, double edBt) {
    std::stringstream ss;
    ss << std::fixed << std::setprecision(4);

```

```

95      ss << "<h2>3. Indicadores Críticos do Instante de Ruptura (
          Breakthrough)</h2>";
      ss << "<div class='card-highlight'>";
      ss << "Valores analíticos extraídos via Condição de Entropia de
          Oleinik e Tangente de Welge:";

100     ss << "<table>";
      ss << "<tr><th width='60%'>Parâmetro de Desempenho (Frente de
          Choque)</th><th width='20%'>Valor</th><th width='20%'>Unidade
          </th></tr>";
      ss << "<tr><td>Saturação de Água na Frente de Choque (S<sub>wf</sub>)</td><td class='metric-value'>" << swFrente << "</td><td>
          adimensional</td></tr>";
      ss << "<tr><td>Saturação Média de Água na Ruptura (<span style='
          text-decoration: underline;'>S</span><sub>w, bt</sub>)</td><td class='metric-value'>" << swMedia << "</td><td>adimensional</td></tr>";
      ss << "<tr><td>Tempo Adimensional de Ruptura (t<sub>D, bt</sub>)</td><td class='metric-value'>" << tempoRuptura << "</td><td>PVI
          </td></tr>";

105     ss << std::setprecision(2);
      ss << "<tr><td style='font-weight: bold; color: #1a5276;'>
          Eficiência de Deslocamento no BT (E<sub>d, bt</sub>)</td><td class='metric-value' style='color: #196f3d; font-size: 11pt;'>"
          << edBt * 100.0 << "</td><td style='font-weight: bold; color: #196f3d;'>%</td></tr>";
      ss << "</table>";
      ss << "</div>";

110     _textoRelatorio += ss.str();
    }

    void CRelatorio::registrarEficiencia(double tempo, double
        fatorRecuperacao) {
115         std::stringstream ss;
        ss << std::fixed << std::setprecision(2);

        ss << "<h2>4. Status da Janela de Injeção Computada</h2>";
        ss << "<table>";
120         ss << "<tr><th width='60%'>Parâmetro Dinâmico Atual</th><th width
            ='20%'>Valor Computado</th><th width='20%'>Unidade</th></tr>";
        ss << "<tr><td>Volume Acumulado Injetado (PVI)</td><td class='
            metric-value'>" << tempo << "</td><td>Volumes Porosos</td></tr>";
        ss << "<tr><td>Eficiência de Deslocamento Atual (E<sub>d</sub>)</td><td class='metric-value' style='color: #1a5276;'>" << (
            fatorRecuperacao * 100.0) << "</td><td>% (Óleo Original)</td>";

```

```

        ></tr>";
    ss << "</table>";

125     _textoRelatorio += ss.str();
    }

    // --- Processamento de Imagens e Exportação I/O ---
    void CRelatorio::registrarGraficos(const QPixmap& pixFluxo, const
        QPixmap& pixSaturacao, const QPixmap& pixEficiencia) {
130     _textoRelatorio += "<h2>5. Curvas e Perfis Analíticos Gerados</h2>";

    // Expressão Lambda para conversão Binária -> Base64
    auto pixmapToBase64Html = [](const QPixmap& pix) -> std::string {
135         if (pix.isNull()) return "";
        QByteArray bytes;
        QBuffer buffer(&bytes);
        buffer.open(QIODevice::WriteOnly);
        pix.save(&buffer, "PNG");
        return bytes.toBase64().toStdString();
140     };

    std::string b64Fluxo = pixmapToBase64Html(pixFluxo);
    std::string b64Sat = pixmapToBase64Html(pixSaturacao);
    std::string b64Ef = pixmapToBase64Html(pixEficiencia);

145     _textoRelatorio += "<table width='100%' border='0' style='border:
        none;'>";

    if (!b64Fluxo.empty()) {
        _textoRelatorio += "<tr><td style='border:none;' class='
            center-plot'>";
150     _textoRelatorio += "<img src='data:image/png;base64,' +
        b64Fluxo + "' width='500' /><br>";
        _textoRelatorio += "<h3>Figura 5.1: Curva de Fluxo
            Fracionário (fw) e Reta Tangente de Welge</h3>";
        _textoRelatorio += "</td></tr>";
    }

    if (!b64Sat.empty()) {
155     _textoRelatorio += "<tr><td style='border:none;' class='
        center-plot'>";
        _textoRelatorio += "<img src='data:image/png;base64,' +
        b64Sat + "' width='500' /><br>";
        _textoRelatorio += "<h3>Figura 5.2: Perfil Espacial de
            Saturação de Água (Sw vs xD)</h3>";
        _textoRelatorio += "</td></tr>";
160     }

```

```

    if (!b64Ef.empty()) {
        _textoRelatorio += "<tr><td style='border:none;' class='
            center-plot'>";
        _textoRelatorio += "<img src='data:image/png;base64,'" + b64Ef
            + "' width='500'/><br>";
165    _textoRelatorio += "<h3>Figura 5.3: Histórico de Eficiência
            de Deslocamento Microscópico (Ed vs PVI)</h3>";
        _textoRelatorio += "</td></tr>";
    }

    _textoRelatorio += "</table>";
170 }

void CRelatorio::adicionarNota(const std::string& notaHtml) {
    _textoRelatorio += "<h2>6. Notas e Observações Adicionais</h2>";
    _textoRelatorio += "<p>" + notaHtml + "</p>";
175 }

bool CRelatorio::exportarParaPDF(const std::string& caminho) {
    std::string htmlFinal = _textoRelatorio + "</body></html>";

180    QTextDocument documento;
    documento.setHtml(QString::fromStdString(htmlFinal));

    QPdfWriter printer(QString::fromStdString(caminho));
    printer.setPageSize(QPageSize(QPageSize::A4));
185    printer.setPageMargins(QMarginsF(15, 15, 15, 15), QPageLayout::
        Millimeter);

    documento.print(&printer);
    return true;
}

```

Apresenta-se na listagem 7.24 o cabeçalho da classe `MainWindow`, a camada de apresentação que concretiza o isolamento de interface gráfica através de arquitetura guiada por eventos (*Signals and Slots*) do *framework* Qt.

Listagem 7.24: Arquivo de Cabeçalho `MainWindow.h`

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

/**
5  * @file MainWindow.h
  * @brief Definição da classe MainWindow, a interface gráfica
    principal do simulador.
  */

#include <QMainWindow>
10 #include <map>
#include "CSimulador.h"

```

```

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
15 QT_END_NAMESPACE

/**
 * @class MainWindow
 * @brief Camada de Apresentação (View) do simulador analítico.
20 *
 * Esta classe herda de QMainWindow e é responsável por toda a
 * interação direta
 * com o usuário. Ela coleta os parâmetros petrofísicos e de fluidos,
 * repassa-os
 * ao Controlador (CSimulador), e gerencia a atualização visual dos
 * painéis de
 * gráficos (QCustomPlot) e dos relatórios.
25 *
 * Além disso, a classe implementa um gerenciador dinâmico de folhas
 * de estilo (QSS)
 * para alternância em tempo de execução entre os temas Claro e
 * Escuro, garantindo
 * conforto visual ergonômico.
 *
30 * @author Fabiane da Silva Barros
 * @date Janeiro 2026
 */
class MainWindow : public QMainWindow
{
35     Q_OBJECT

public:
    /**
     * @brief Construtor padrão da interface gráfica.
40     * @param parent Widget pai (opcional).
     */
    MainWindow(QWidget *parent = nullptr);

    /**
45     * @brief Destrutor virtual.
     * Libera a interface de usuário (UI) e a instância do simulador
     * alocada.
     */
    ~MainWindow();

50 private slots:
    // --- Gatilhos de Ação (Slots) ---

    /**
     * @brief Slot acionado para calcular e plotar estritamente a

```

```

        curva de fluxo fracionário.
55    */
    void on_btnPlotarFw_clicked();

    /**
     * @brief Slot acionado para executar a varredura completa da
     *        simulação (MOC + Welge).
60    */
    void on_btnPlotarSolucao_clicked();

    /**
     * @brief Slot acionado para invocar a exportação do documento
     *        executivo em PDF.
65    */
    void on_btnRelatorio_clicked();

    /**
     * @brief Slot acionado na alteração da aba (StackedWidget) do
     *        modelo de permeabilidade.
70    * @param index Índice da aba selecionada (0: Corey, 1: LET, 2:
     *        Chierici, 3: Tabela).
    */
    void on_cbModeloPerm_currentIndexChanged(int index);

    // --- Slots de Manipulação de Arquivos ---
75    void on_bt1CarregarCorey_clicked();
    void on_btnCarregarLET_clicked();
    void on_btnCarregarChierici_clicked();
    void on_btnCarregarTabela_clicked();

80    /**
     * @brief Slot acionado para alternar entre os Modos Claro e
     *        Escuro da UI.
    */
    void on_btnTema_clicked();

85 private:
    Ui::MainWindow *ui;          ///< Ponteiro gerenciado pelo Qt
                                // contendo os elementos do XML (.ui).
    CSimulador *simulador;      ///< Ponteiro para o Controlador MVC
                                // principal.

    // --- Métodos Auxiliares Internos ---
90    /**
     * @brief Coleta todos os inputs textuais da tela e sincroniza
     *        com o Controlador.
     * @throws std::runtime_error Se algum formato de entrada for
     *        inválido.

```

```
    */
95 void sincronizarDadosComSimulador();

/**
 * @brief Configura e inicializa os eixos e propriedades visuais
 *        do QCustomPlot.
 */
100 void configurarGraficos();

/**
 * @brief Atualiza a plotagem dos resultados após o processamento
 *        do CSolver.
 */
105 void plotarResultados();

/**
 * @brief Extrai a saturação residual de óleo baseada na aba
 *        ativa.
 * @return Valor de Sor.
110 */
double obterSaturacaoOleoResidualUI() const;

/**
 * @brief Extrai a saturação irreduzível de água baseada na aba
 *        ativa.
115 * @return Valor de Swir.
 */
double obterSaturacaoInicialUI() const;

// --- Lógica de Temas (QSS) ---
120 bool isDarkMode = false;    ///< Sinalizador de estado do tema
    visual (false = Modo Claro).

/**
 * @brief Aplica as folhas de estilo (QSS) pertinentes aos
 *        widgets da tela.
 */
125 void aplicarTemaUI();

/**
 * @brief Refaz a renderização de cores do QCustomPlot (Fundo,
 *        eixos, malha) conforme o tema.
 */
130 void atualizarCoresGraficos();

/**
 * @brief Retorna as regras estruturais e de layout invariáveis
 *        da aplicação em QSS.
 * @return String contendo a folha de estilo base.
```

```

135     */
    QString gerarEstiloBaseUI();
};

```

```
#endif // MAINWINDOW_H
```

Apresenta-se na listagem 7.25 a implementação da interface gráfica, integrando a coleta sanitizada de dados estruturais, o acionamento do controlador CSimulador e a aplicação em tempo real de folhas de estilo dinâmicas (QSS).

Listagem 7.25: Arquivo de Implementação MainWindow.cpp

```

/**
 * @file MainWindow.cpp
 * @brief Implementação dos manipuladores de eventos e injeção
 *        gráfica.
 * @author Fabiane da Silva Barros
5  * @date Janeiro 2026
 */

#include "MainWindow.h"
#include "ui_mainwindow.h"
10 #include <QMessageBox>
#include <QFileDialog>
#include <QInputDialog>
#include <algorithm>
#include <map>

15 // Constante global de precisão para contornos
static constexpr double SWI_EPS = 1e-3;

// Inclusão dos pacotes matemáticos de domínio
20 #include "CCurvasPermeabilidadeCorey.h"
#include "CCurvasPermeabilidadeLET.h"
#include "CCurvasPermeabilidadeChierici.h"
#include "CCurvasPermeabilidadeTabelada.h"
#include "ICurvasPermeabilidade.h"
25 #include "CRelatorio.h"

// --- Construtor e Configuração Inicial ---
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
30    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    // Ajuste ergonômico do botão de Tema
35    ui->btnTema->setText("");
    ui->btnTema->setIconSize(QSize(24, 24));

    // Alocação da dependência estrutural do Controller

```

```

simulador = new CSimulador();
40 simulador->setModeloPermeabilidade(new CCurvasPermeabilidadeCorey
    ());

configurarGraficos();

// Aplicação mandatório do tema inicial de carregamento
45 aplicarTemaUI();
atualizarCoresGraficos();

// Alinhamento do stack de painéis com o menu suspenso
ui->stkModelos->setCurrentIndex(ui->cbModeloPerm->currentIndex())
    ;
50 }

// --- Destrutor ---
MainWindow::~MainWindow()
{
55     delete simulador;
    delete ui;
}

void MainWindow::configurarGraficos() {
60     // 1. Fluxo Fracionário (Adicionando a reta tangente de Welge)
    ui->plotFluxo->xAxis->setLabel("Saturação de Água (Sw)");
    ui->plotFluxo->yAxis->setLabel("Fluxo Fracionário (fw)");
    ui->plotFluxo->addGraph(); // Gráfico 0: Curva Fw (Azul)
    ui->plotFluxo->graph(0)->setPen(QPen(Qt::blue));
65 ui->plotFluxo->addGraph(); // Gráfico 1: Tangente de Welge (
    Tracejado Vermelho)
    ui->plotFluxo->graph(1)->setPen(QPen(Qt::red, 1, Qt::DashLine));

    // 2. Perfil de Saturação (Agora Sw vs xD)
    ui->plotSvsX->xAxis->setLabel("Posição Adimensional (xD)");
70 ui->plotSvsX->yAxis->setLabel("Saturação (Sw)");
    ui->plotSvsX->xAxis->setRange(0, 1.0); // xD máximo físico = 1.0
    ui->plotSvsX->addGraph();
    ui->plotSvsX->graph(0)->setPen(QPen(Qt::red));

75 // 3. Eficiência de Deslocamento vs PVI [cite: 627, 631]
    ui->plotEfDesl->xAxis->setLabel("Tempo Adimensional (PVI)");
    ui->plotEfDesl->yAxis->setLabel("Eficiência de Deslocamento (Ed)");
    ui->plotEfDesl->addGraph(); // Curva Ed
    ui->plotEfDesl->addGraph(); // Ponto de Breakthrough (Destaque)
80 ui->plotEfDesl->graph(1)->setLineStyle(QCPGraph::lsNone);
    ui->plotEfDesl->graph(1)->setScatterStyle(QCPScatterStyle::ssDisc);
}

```

```

void MainWindow::on_cbModeloPerm_currentIndexChanged(int index) {
85     ui->stkModelos->setCurrentIndex(index);
}

void MainWindow::sincronizarDadosComSimulador() {
    // 1. Coleta e Conversão de Dados Escalares
90     double L      = ui->leComprimento->text().toDouble(); // 100 m
    double A      = ui->leArea->text().toDouble(); // 1 m2
    double phi     = ui->lePorosidade->text().toDouble() / 100.0; // 20
        %
    double ang     = ui->leAngulo->text().toDouble(); // 0 graus
    double vazao   = ui->leVazao->text().toDouble(); // 1 m3/d
95     double k      = ui->lePerm->text().toDouble(); // 100 mD

    double mi_o    = ui->leViscOleo->text().toDouble(); // 1 cP
    double mi_w    = ui->leViscAgua->text().toDouble(); // 1 cP
    double rho_o    = ui->leDensOleo->text().toDouble(); // 800 kg/m3
100    double rho_w   = ui->leDensAgua->text().toDouble(); // 1000 kg/m3

    // 2. Sincroniza Propriedades do Reservatório e Fluidos
    simulador->setDadosReservatorio(L, A, phi, ang, vazao);
    simulador->setFluidos(mi_o, mi_w, rho_o, rho_w);
105    simulador->setPermeabilidade(k); // Define a permeabilidade
        absoluta lida da UI

    // 3. Atualiza a Calculadora (Crítico para Gravidade e Rapoport-
        Leas)
    simulador->getCalculadora()->setPropriedades(mi_w, mi_o, rho_w,
        rho_o, k, ang, vazao, A);
    double ut = vazao/A;
110

    // =====
    // ETAPA 2: Configura o Modelo de Permeabilidade (Strategy)
    // =====
    ICurvasPermeabilidade* modelo = nullptr;
115    int indiceModelo = ui->cbModeloPerm->currentIndex();

    if (indiceModelo == 0) { // COREY
        double no = ui->leCoreyNo->text().toDouble(); // 2 inteiro
        double nw = ui->leCoreyNw->text().toDouble(); // 2 inteiro
120        double swi = ui->leCoreySwi->text().toDouble(); // 0.2 fração
        double sor = ui->leCoreySor->text().toDouble(); // 0.2 fração
        double krwm = ui->leCoreyKrwm->text().toDouble(); // 1.0
            fração
        double krom = ui->leCoreyKrom->text().toDouble(); // 1.0
            fração

125        modelo = new CCurvasPermeabilidadeCorey(krom, krwm, no, nw,

```

```

        swi, sor);

    } else if (indiceModelo == 1) { // LET
        double Lw = ui->leLetLw->text().toDouble();
        double Ew = ui->leLetEw->text().toDouble();
130    double Tw = ui->leLetTw->text().toDouble();
        double Lo = ui->leLetLo->text().toDouble();
        double Eo = ui->leLetEo->text().toDouble();
        double To = ui->leLetTo->text().toDouble();
        double swi = ui->leLetSwir->text().toDouble();
135    double sor = ui->leLetSor->text().toDouble();

        modelo = new CCurvasPermeabilidadeLET(Lw, Ew, Tw, Lo, Eo, To,
            swi, sor);

    } else if (indiceModelo == 2) { // CHIERICI
140    double Aw = ui->leChiericiAw->text().toDouble();
        double Bw = ui->leChiericiBw->text().toDouble();
        double Ao = ui->leChiericiAo->text().toDouble();
        double Bo = ui->leChiericiBo->text().toDouble();
        double swi = ui->leChiericiSwir->text().toDouble();
145    double sor = ui->leChiericiSor->text().toDouble();
        double krwm = ui->leChiericiKrwMax->text().toDouble();
        double krom = ui->leChiericiKroMax->text().toDouble();

        modelo = new CCurvasPermeabilidadeChierici(Aw, Bw, Ao, Bo,
            swi, sor, krwm, krom);

150    } else { // TABELA
        QString arq = ui->lblArquivoTabela->text();
        if (arq == "Arquivo" || arq.isEmpty()) {
            throw std::runtime_error("Selecione um arquivo de tabela
                primeiro.");
155        }
        auto tabela = new CCurvasPermeabilidadeTabelada();
        tabela->carregarDados(arq.toStdString());
        modelo = tabela;
    }

160    if (!modelo) {
        throw std::runtime_error("Por favor, configure um modelo de
            permeabilidade válido.");
    }

165    simulador->setModeloPermeabilidade(modelo);

}

double MainWindow::obterSaturacaoInicialUI() const {

```

```

170     auto modelo = simulador->getModeloPermeabilidade();
    int indiceAtivo = ui->stkModelos->currentIndex();

    switch (indiceAtivo) {
    case 0: { // Corey
175         auto corey = dynamic_cast<CCurvasPermeabilidadeCorey*>(modelo
        );
        if (corey && ui->lblArquivoCorey->text() != "Arquivo" && !ui
            ->lblArquivoCorey->text().isEmpty())
            return corey->getSwi();
        return ui->leCoreySwi->text().toDouble();
    }
180     case 1: { // LET
        auto let = dynamic_cast<CCurvasPermeabilidadeLET*>(modelo);
        if (let && ui->lblArquivoLET->text() != "Arquivo" && !ui->
            lblArquivoLET->text().isEmpty())
            return let->getSwi();
        return ui->leLetSwir->text().toDouble();
185     }
    case 2: { // Chierici
        auto chierici = dynamic_cast<CCurvasPermeabilidadeChierici*>(
            modelo);
        if (chierici && ui->lblArquivoChierici->text() != "Arquivo"
            && !ui->lblArquivoChierici->text().isEmpty())
            return chierici->getSwi();
190         return ui->leChiericiSwir->text().toDouble();
    }
    case 3: { // Tabela
        auto tabela = dynamic_cast<CCurvasPermeabilidadeTabelada*>(
            modelo);
        if (tabela) return tabela->getSwi();
195         return 0.40;
    }
    default:
        return 0.0;
    }
200 }

double MainWindow::obterSaturacaoOleoResidualUI() const {
    auto modelo = simulador->getModeloPermeabilidade();
    int indiceAtivo = ui->stkModelos->currentIndex();
205

    switch (indiceAtivo) {
    case 0: { // Corey
        auto corey = dynamic_cast<CCurvasPermeabilidadeCorey*>(modelo
        );
        if (corey && ui->lblArquivoCorey->text() != "Arquivo" && !ui
            ->lblArquivoCorey->text().isEmpty())
210         return corey->getSor();
    }

```

```

        return ui->leCoreySor->text().toDouble();
    }
    case 1: { // LET
        auto let = dynamic_cast<CCurvasPermeabilidadeLET*>(modelo);
215     if (let && ui->lblArquivoLET->text() != "Arquivo" && !ui->
        lblArquivoLET->text().isEmpty())
            return let->getSor();
        return ui->leLetSor->text().toDouble();
    }
    case 2: { // Chierici
220     auto chierici = dynamic_cast<CCurvasPermeabilidadeChierici*>(
        modelo);
        if (chierici && ui->lblArquivoChierici->text() != "Arquivo"
            && !ui->lblArquivoChierici->text().isEmpty())
            return chierici->getSor();
        return ui->leChiericiSor->text().toDouble();
    }
225 case 3: { // Tabela
        auto tabela = dynamic_cast<CCurvasPermeabilidadeTabelada*>(
            modelo);
        if (tabela) return 1.0 - tabela->getSwMax();
        return 0.30;
    }
230 default:
        return 0.0;
    }
}

// --- SLOT: Botão Apenas Fluxo Fracionário (Sem Simulação de Tempo)
---

235 void MainWindow::on_btnPlotarFw_clicked() {
    try {
        sincronizarDadosComSimulador(); // Dados unificados
        auto calc = simulador->getCalculadora();
        // Obter Swi de forma consistente com o modelo selecionado (
        // mesma fonte usada em sincronizarDadosComSimulador)
240     double sor = obterSaturacaoOleoResidualUI();
        ui->txtLog->appendHtml("<b>---␣Diagnóstico␣de␣Engenharia␣
        ---</b>");

        // Chamadas diretas conforme implementado na classe

245     double comp_fw = ui->leComprimento->text().toDouble();
        double phi_fw = ui->lePorosidade->text().toDouble() / 100.0;

        // 2. Imprime no Log o Número de Rapoport-Leas calculado
        ui->txtLog->appendHtml(QString("N.␣Rapoport-Leas:␣%1").arg(
            calc->calcularRapoportLeas(comp_fw, phi_fw, 0.03), 0, 'f',
            2));
    }
}

```

250

```

// 3. Define a saturação para exibição e calcula as
    constantes globais
double sw_report = 1 - sor;
double M_sw = calc->calcularM0();
double Ng_sw = calc->calcularNg0();

255
auto modelo = simulador->getModeloPermeabilidade();

QVector<double> xFw(101), yFw(101);

260
for (int i = 0; i <= 100; ++i) {
    double sw = i / 100.0;
    xFw[i] = sw;
    yFw[i] = calc->calcularFw(sw); // Usa a Eq. 3.10
        corrigida
265
}

ui->plotFluxo->graph(0)->setData(xFw, yFw);
ui->plotFluxo->xAxis->setRange(0, 1.0);
ui->plotFluxo->graph(0)->rescaleAxes();
270
// pequena margem visual
double ymin = ui->plotFluxo->yAxis->range().lower;
double ymax = ui->plotFluxo->yAxis->range().upper;
double margin = 0.05 * (ymax - ymin);
ui->plotFluxo->yAxis->setRange(ymin - margin, ymax + margin);
275
ui->plotFluxo->replot();

} catch (const std::exception& e) {
    QMessageBox::critical(this, "Erro", e.what());
}
280
}

// --- SIMULAÇÃO PRINCIPAL ---
void MainWindow::on_btnPlotarSolucao_clicked() {
285
    try {
        sincronizarDadosComSimulador();

        double pvi_input = ui->lePVI->text().toDouble();
        if (pvi_input <= 0.0) {
290
            throw std::runtime_error("Informe um tempo válido no campo Tempo.");
        }

        double L = ui->leComprimento->text().toDouble();
        double A = ui->leArea->text().toDouble();
295
        double phi = ui->lePorosidade->text().toDouble() / 100.0;

```

```

    if (A <= 0.0 || phi <= 0.0 || L <= 0.0) {
        throw std::runtime_error("Verifique Comprimento, Área e
        Porosidade (devem ser > 0).");
    }

300
    // CORREÇÃO: Coleta as condições de contorno e passa para o
    pipeline do simulador
    double swi = obterSaturacaoInicialUI();
    double sor = obterSaturacaoOleoResidualUI();
    double sw_max = 1.0 - sor;

305
    simulador->executarSimulacao(pvi_input, swi, sw_max);

    plotarResultados();
} catch (const std::exception& e) {
310
    QMessageBox::critical(this, "Erro", e.what());
}
}

void MainWindow::plotarResultados() {
315
    auto calc = simulador->getCalculadora();
    auto welge = simulador->getWelge();

    double pvi_input = ui->lePVI->text().toDouble();
    double swi = obterSaturacaoInicialUI();
320
    double sor = obterSaturacaoOleoResidualUI();
    double sw_max = 1.0 - sor;

    // Executa Welge passando os parâmetros reais
    welge->calcularTangente(calc, swi, sw_max);

325
    double swFrente = welge->getSwFrente();
    double vel_choque = welge->getInclinacaoChoque();
    double t_bt = (vel_choque > 1e-6) ? (1.0 / vel_choque) : 1.0;

330
    //
    -----

    // 1. PERFIL DE SATURAÇÃO (Sw vs xD)
    //
    -----

    QVector<double> xD, Sw;
    double pos_choque = vel_choque * pvi_input;

335
    xD.append(0.0);
    Sw.append(sw_max);

    for (double s = sw_max; s >= swFrente; s -= 0.001) {

```

```

340     double vel = calc->calcularDerivadaFw(s);
    double pos = vel * pvi_input;

    if (pos > pos_choque) pos = pos_choque;

345     if (pos <= 1.0) {
        xD.append(pos);
        Sw.append(s);
    }
}

350     if (pos_choque <= 1.0) {
        xD.append(pos_choque); Sw.append(swi);
        xD.append(1.0);         Sw.append(swi);
    } else {
355         double target_vel = 1.0 / pvi_input;
        double sw_out = swFrente;
        for (double s = sw_max; s >= swFrente; s -= 0.001) {
            if (calc->calcularDerivadaFw(s) >= target_vel) {
                sw_out = s;
360                 break;
            }
        }
        if (!xD.isEmpty() && xD.last() < 1.0) {
            xD.append(1.0);
365            Sw.append(sw_out);
        }
    }

    ui->plotSvsX->graph(0)->setData(xD, Sw);
370    ui->plotSvsX->xAxis->setRange(0.0, 1.0);
    ui->plotSvsX->yAxis->setRange(0.0, 1.05);

    //
    -----

    // 2. TANGENTE DE WELGE NO FLUXO FRACIONÁRIO
375    //
    -----

    if (swFrente > swi + 1e-6) {
        ui->plotFluxo->graph(1)->setData({swi, welge->getSwMedia()},
            {calc->calcularFw(swi), 1.0});
        ui->plotFluxo->rescaleAxes();
        double yMinF = ui->plotFluxo->yAxis->range().lower;
380        double yMaxF = ui->plotFluxo->yAxis->range().upper;
        ui->plotFluxo->yAxis->setRange(yMinF - 0.05*(yMaxF-yMinF),
            yMaxF + 0.05*(yMaxF-yMinF));
    } else {

```

```

    if (ui->plotFluxo->graphCount() > 1 && ui->plotFluxo->graph
        (1)->data()) {
        ui->plotFluxo->graph(1)->data()->clear();
385     }
}

//
-----

// 3. EFICIÊNCIA DE DESLOCAMENTO (Ed vs PVI)
390 //
-----

std::map<double, double> curvaEd;

for (double t = 0.0; t <= t_bt; t += 0.01) {
    curvaEd[t] = t / (1.0 - swi);
395 }

for (double s = swFrente + 0.001; s <= sw_max; s += 0.001) {
    double deriv = calc->calcularDerivadaFw(s);

400     if (deriv > 1e-6) {
        double t_saida = 1.0 / deriv;

        if (t_saida >= t_bt && t_saida <= 3.0) {
            double fw_s = calc->calcularFw(s);
405             double sw_media = s + (1.0 - fw_s) * t_saida;
            double ed_atual = (sw_media - swi) / (1.0 - swi);
            curvaEd[t_saida] = ed_atual;
        }
    }
}
410 }

QVector<double> tPVI, Ed;
double max_ed = 0.0;

415 for (auto const& par : curvaEd) {
    double t = par.first;
    double ed = par.second;

    if (ed >= max_ed) {
420         max_ed = ed;
        tPVI.append(t);
        Ed.append(ed);
    }
}

425 if (!tPVI.isEmpty() && tPVI.last() < 3.0) {

```

```

        tPVI.append(3.0);
        Ed.append(Ed.last());
    }

430    ui->plotEfDesl->graph(0)->setData(tPVI, Ed);

    double ed_bt = t_bt / (1.0 - swi);
    ui->plotEfDesl->graph(1)->setData({t_bt}, {ed_bt});

435    ui->plotEfDesl->graph(0)->rescaleAxes();
    ui->plotEfDesl->xAxis->setRange(0.0, 3.0);
    double yMinE = ui->plotEfDesl->yAxis->range().lower;
    double yMaxE = ui->plotEfDesl->yAxis->range().upper;
440    ui->plotEfDesl->yAxis->setRange(std::max(0.0, yMinE - 0.05*(yMaxE
        -yMinE)), yMaxE + 0.05*(yMaxE-yMinE));

    ui->plotFluxo->replot();
    ui->plotSvsX->replot();
    ui->plotEfDesl->replot();
445 }

// --- Carregamento de Arquivos ---

void MainWindow::on_btlCarregarCorey_clicked() {
450    QString path = QFileDialog::getOpenFileName(this, "Abrir_Corey",
        "", "Text_Files (*.txt)");
    if(path.isEmpty()) return;
    ui->lblArquivoCorey->setText(path);

    // Opcional: Carregar do arquivo para os LineEdits agora
455    try {
        CCurvasPermeabilidadeCorey temp;
        temp.carregarDados(path.toStdString());
        // Aqui poderíamos ter getters na classe Corey para preencher
        // a tela...
        // Como não implementamos getters nas classes de curva,
        // apenas armazenamos o path.
460        QMessageBox::information(this, "Sucesso", "Arquivo_
            selecionado._Clique_em_Solução_para_usar.");
    } catch (const std::exception& e) {
        QMessageBox::critical(this, "Erro", e.what());
    }
}

465 void MainWindow::on_btnCarregarLET_clicked() {
    QString path = QFileDialog::getOpenFileName(this, "Abrir_LET", "",
        , "Text_Files (*.txt)");
    if(!path.isEmpty()) ui->lblArquivoLET->setText(path);
}

```

```

470 void MainWindow::on_btnCarregarChierici_clicked() {
    QString path = QFileDialog::getOpenFileName(this, "Abrir_Chierici",
        "", "Text_Files (*.txt)");
    if(!path.isEmpty()) ui->lblArquivoChierici->setText(path);
}

475 void MainWindow::on_btnCarregarTabela_clicked() {
    QString path = QFileDialog::getOpenFileName(this, "Abrir_Tabela",
        "", "Text_Files (*.txt)");
    if(!path.isEmpty()) ui->lblArquivoTabela->setText(path);
}

480 void MainWindow::on_btnRelatorio_clicked() {
    try {
        QString caminho = QFileDialog::getSaveFileName(this, "Salvar_
            Relatório_Executivo", "", "PDF (*.pdf)");
        if (caminho.isEmpty()) return;

485 CRelatorio rel;

        // 1. Coleta os parâmetros estáticos
        double L = ui->leComprimento->text().toDouble();
        double A = ui->leArea->text().toDouble();
490 double phi = ui->lePorosidade->text().toDouble() / 100.0;
        double k = ui->lePerm->text().toDouble();
        double angulo = ui->leAngulo->text().toDouble();
        double mi_o = ui->leViscOleo->text().toDouble();
495 double mi_w = ui->leViscAgua->text().toDouble();

        rel.gerarCabecalho(L, A, phi, k, angulo, mi_o, mi_w);

        // 2. Coleta dados da calculadora física
500 auto calc = simulador->getCalculadora();
        double nrl = calc->calcularRapoportLeas(L, phi, 0.03); //
            Tensão interfacial padrão
        double M = calc->calcularM0();
        double Ng = calc->calcularNg0();

505 rel.registrarDiagnosticoFisico(nrl, M, Ng);

        // 3. Métricas Avançadas de Welge e Ruptura (BT)
        auto welge = simulador->getWelge();
        double swi = obterSaturacaoInicialUI();
510 double swFrente = welge->getSwFrente();
        double swMedia = welge->getSwMedia();
        double vel_choque = welge->getInclinacaoChoque();
        double t_bt = (vel_choque > 1e-6) ? (1.0 / vel_choque) : 1.0;

```

```

515 // Cálculo analítico da Eficiência de Deslocamento no exato
    instante de Breakthrough
double ed_bt = (swMedia - swi) / (1.0 - swi);

rel.registrarResultadoWelge(swFrente, swMedia, t_bt, ed_bt);

520 // 4. Parâmetros dinâmicos do tempo digitado na interface
double pvi_input = ui->lePVI->text().toDouble();
// Saturação média atual depende se já passou do BT ou não
double sw_media_atual = swMedia;
if (pvi_input > t_bt) {
525     double target_deriv = 1.0 / pvi_input;
    double sor = obterSaturacaoOleoResidualUI();
    double sw_max = 1.0 - sor;
    double sw_out = swFrente;
    for (double s = swFrente; s <= sw_max; s += 0.001) {
530         if (calc->calcularDerivadaFw(s) <= target_deriv) {
            sw_out = s;
            break;
        }
    }
535     double fw_out = calc->calcularFw(sw_out);
    sw_media_atual = sw_out + (1.0 - fw_out) * pvi_input;
} else {
    sw_media_atual = swi + pvi_input;
}
540 double ed_atual = (sw_media_atual - swi) / (1.0 - swi);
if (ed_atual > 1.0 - obterSaturacaoOleoResidualUI() - swi)
    ed_atual = 1.0 - obterSaturacaoOleoResidualUI() - swi;

rel.registrarEficiencia(pvi_input, ed_atual);

545 // =====
// 5. CAPTURA DOS GRÁFICOS DA INTERFACE (MÁGICA VISUAL)
// Extrai os Pixmaps com tamanho adequado para o documento A4
// =====
QPixmap pixFluxo = ui->plotFluxo->toPixmap(500, 350);
550 QPixmap pixSaturacao = ui->plotSvsX->toPixmap(500, 350);
QPixmap pixEficiencia = ui->plotEfDesl->toPixmap(500, 350);

rel.registrarGraficos(pixFluxo, pixSaturacao, pixEficiencia);

555 // 6. Exportação definitiva
rel.exportarParaPDF(caminho.toStdString());
QMessageBox::information(this, "Sucesso", "Relatório PDF
    gerado com sucesso.");

} catch (const std::exception& e) {
560     QMessageBox::warning(this, "Erro ao Gerar Relatório", e.what

```

```

        ());
    }
}

// --- Lógica de Temas Atualizada e Polida ---
565 // Esta função define a "estrutura" visual que não muda (Fonte e
    Cantos Arredondados)
QString MainWindow::gerarEstiloBaseUI() {
    return R"(
        /*_Define_a_fonte_e_cantos_para_QUASE_tudo_na_janela_*/
570 /*_Widget_*/
        font-family: 'Segoe UI', 'Helvetica Neue', sans-serif;
        font-size: 10pt;
        border-radius: 6px; /*_Cantos_arredondados_padrao_
            universal_*/
        /*_Títulos_e_Labels_mais_fortes_*/
        QLabel {
            font-weight: bold;
            background: transparent; /*_Garante_que_labels_não_tenham
                fundo_feio_*/
580 }

        /*_Inputs_e_Combos_arredondados_e_com_respiro_*/
        QLineEdit, QComboBox, QSpinBox {
            border: 1px solid; /*_A_cor_muda_no_tema_*/
585 padding: 5px;
            min-height: 20px;
        }

        /*_Botões_com_cantos_bem_arredondados_e_padding_*/
590 QPushButton {
            border: none;
            padding: 8px 15px;
            font-weight: bold;
            min-width: 80px;
595 }

        /*_TextEdit_do_Log_arredondado_*/
        QTextEdit {
            border: 1px solid;
600 border-radius: 6px;
        }
    )";
}

605 void MainWindow::on_btnTema_clicked() {

```

```

        isDarkMode = !isDarkMode; // Apenas inverte o estado e manda
        aplicar
        aplicarTemaUI();
        atualizarCoresGraficos();
    }
610
    void MainWindow::aplicarTemaUI() {
        QString estiloFinal = gerarEstiloBaseUI();

        if (isDarkMode) {
615            // --- CORES TEMA ESCURO ---
            // Caminho atualizado com a pasta "icones"
            ui->btnTema->setIcon(QIcon(":/icones/luca.png"));

            estiloFinal += R"(
620 QMainWindow, QWidget {
            background-color: #282c34;
            color: #abb2bf;
        }
            QLabel { color: #abb2bf; }
625 QLineEdit, QComboBox, QTextEdit, QSpinBox, QDoubleSpinBox
            {
            background-color: #21252b;
            color: #abb2bf;
            border-color: #181a1f;
630 QLineEdit:focus, QComboBox:focus { border-color: #61afef;
            }

            QPushButton {
            background-color: #3e4451;
635 color: #ffffff;
            }
            QPushButton:hover { background-color: #61afef; color: #
                #282c34; }
            QPushButton:pressed { background-color: #528bff; }

640 QTextEdit { color: #98c379; }
            )";
        } else {
            // --- CORES TEMA CLARO ---
            // Caminho atualizado com a pasta "icones"
645 ui->btnTema->setIcon(QIcon(":/icones/sol.png"));

            estiloFinal += R"(
            QMainWindow, QWidget {
            background-color: #f5f5f5;
650 color: #333333;

```

```

        }
        QLabel_{color:_#333333;_}

        QLineEdit,_QComboBox,_QTextEdit,_QSpinBox,_QDoubleSpinBox
        _{
655        background-color:_#ffffff;
            color:_#333333;
            border-color:_#cccccc;
        }
        QLineEdit:focus,_QComboBox:focus_{border-color:_#0078d7;
            _}
660
        QPushButton_{
            background-color:_#e1e1e1;
            color:_#333333;
        }
665        QPushButton:hover_{background-color:_#0078d7;_color:_#
            ffffff;_}
        QPushButton:pressed_{background-color:_#005a9e;_color:_#
            ffffff;_}

        QTextEdit_{color:_#006400;_}
        _);
670    }

    this->setStyleSheet(estiloFinal);
}

675 // *** IMPORTANTE: Mantenha a função atualizarCoresGraficos()
    exatamente
    // como corrigimos na resposta anterior (complots << ui->plot...) ***

void MainWindow::atualizarCoresGraficos() {
    // Cores baseadas no tema
680    QColor corFundo = isDarkMode ? QColor("#282c34") : QColor("#
        ffffff");
    QColor corEixos = isDarkMode ? QColor("#abb2bf") : QColor("
        #000000");
    QColor corGrid = isDarkMode ? QColor("#3e4451") : QColor("#
        e0e0e0");

    // Lista de todos os gráficos na sua tela
685    QList<QCustomPlot*> plots = {ui->plotFluxo, ui->plotSvsX, ui->
        plotEfDesl};

    for (QCustomPlot* plot : plots) {
        plot->setBackground(QBrush(corFundo));

690        // Eixo X

```

```

        plot->xAxis->setBasePen(QPen(corEixos));
        plot->xAxis->setTickPen(QPen(corEixos));
        plot->xAxis->setSubTickPen(QPen(corEixos));
        plot->xAxis->setTickLabelColor(corEixos);
695 plot->xAxis->setLabelColor(corEixos);
        plot->xAxis->grid()->setPen(QPen(corGrid, 1, Qt::DotLine));
        plot->xAxis->grid()->setZeroLinePen(Qt::NoPen);

        // Eixo Y
700 plot->yAxis->setBasePen(QPen(corEixos));
        plot->yAxis->setTickPen(QPen(corEixos));
        plot->yAxis->setSubTickPen(QPen(corEixos));
        plot->yAxis->setTickLabelColor(corEixos);
        plot->yAxis->setLabelColor(corEixos);
705 plot->yAxis->grid()->setPen(QPen(corGrid, 1, Qt::DotLine));
        plot->yAxis->grid()->setZeroLinePen(Qt::NoPen);

        // Bordas superiores e direitas
        plot->xAxis2->setBasePen(QPen(corEixos));
710 plot->yAxis2->setBasePen(QPen(corEixos));

        plot->replot();
    }
}

```

Por fim, apresenta-se na listagem 7.26 o arquivo principal de inicialização do software, responsável por instanciar os recursos do *framework* Qt e transferir o fluxo de controle para o laço de eventos (*Event Loop*) da interface gráfica.

Listagem 7.26: Documentação da Main.cpp

```

/**
 * @file main.cpp
 * @brief Ponto de entrada (Entry Point) da aplicação do Simulador de
 *        Buckley-Leverett.
 * @author Fabiane da Silva Barros
5 * @date Janeiro 2026
 */

#include "src/View/Mainwindow.h"
#include <QApplication>
10

/**
 * @brief Função principal que inicializa o framework Qt e a
 *        interface gráfica.
 * * @param argc Número de argumentos de linha de comando.
 * * @param argv Vetor de matriz de caracteres (argumentos).
15 * @return Código de saída do laço de eventos do Qt (0 indica
 *        encerramento seguro).
 */
int main(int argc, char *argv[])

```

```
{  
    // 1. Inicializa o gerenciador de recursos, fontes e eventos do  
    Qt  
20 QApplication a(argc, argv);  
  
    // 2. Instancia a janela principal da Camada de Apresentação  
    MainWindow w;  
  
25    // 3. Exibe a janela ocupando a tela inteira (maximizada) por  
    padrão para melhor ergonomia  
    w.showMaximized();  
  
    // 4. Transfere o controle de execução para o laço de eventos (  
    Event Loop) do SO  
    return a.exec();  
30 }
```

Capítulo 8

Teste

Todo projeto de engenharia passa por uma etapa de testes. Neste capítulo apresentamos alguns testes do software desenvolvido. Estes testes devem dar resposta aos diagramas de caso de uso inicialmente apresentados (diagramas de caso de uso geral e específicos).

8.1 Teste 1: Validação com Modelo Tabelado (([LAKE et al., 2014](#)))

Este teste tem o objetivo de validar o motor matemático do simulador (Método das Características e a Tangente de Welge) ao processar dados experimentais discretos interpolados linearmente.

Utiliza-se o cenário clássico do Exercício 5.1, Figura 8.1, proposto por ([LAKE et al., 2014](#)). Os dados discretos de permeabilidade relativa (k_{rw} e k_{ro}) foram carregados via arquivo de texto (.txt). Configurou-se um cenário de injeção unidimensional horizontal ($\alpha = 0$), permeabilidade absoluta de $100m^2$ e as viscosidades estipuladas em 1 mPa.s para a água e 5 mPa.s para o óleo. O instante de observação foi fixado em 0.2 PVI.

O programa será validado avaliando-se a detecção automática da saturação irreduzível de água ($S_{wi} = 0.40$), a ancoragem correta da tangente de Welge neste ponto inicial e o cálculo coerente da saturação da frente de choque (S_{wf}) e do tempo de ruptura. Espera-se que o simulador corrija o perfil multivalorado, gerando uma frente de choque perfeitamente vertical (condição de entropia).

O simulador foi capaz de ler a tabela dinâmica, extraindo automaticamente $S_{wi} = 0.40$ e $S_{or} = 0.30$. Observou-se a ancoragem perfeita da secante de entropia e o corte da curva original sem a ocorrência de singularidades (divisão por zero). Um aspecto analítico importante revelado pelo simulador foi o leve ruído numérico ("*ringing*") na porção superior do perfil de saturação de água (platô viscoso). Este fenômeno atesta o rigor do cálculo da derivada de diferenças finitas centrais do simulador: como os dados de entrada são esparsos e interpolados por retas, a função de fluxo fracionário apresenta quinas de classe C^1 , refletindo-se em pequenas oscilações locais na velocidade da onda ($\frac{df_w}{dS_w}$). Na engenharia prática, isso evidencia a necessidade de modelos matemáticos contínuos (como Corey ou LET) para suavização de dados de laboratório.

8.1.1 Arquivo de entrada de dados

A seguir apresenta-se o arquivo de texto formatado com as matrizes de saturação e permeabilidade do ensaio de laboratório.

Listagem 8.1: Arquivo de texto, teste 1

```
# Dados experimentais do Exercício 5.1 (Lake, 1989)
# Colunas: Sw Krw Kro
DADOS_KR_INICIO
0.40 0.000 0.360
5 0.45 0.005 0.260
0.50 0.009 0.140
0.55 0.020 0.080
0.60 0.035 0.036
0.65 0.050 0.020
10 0.70 0.080 0.000
FIM_DADOS
```

8.1.2 Arquivo de saída de dados (resultados da tela)

A seguir apresenta-se a Figura 8.2, a extração dos diagnósticos de ruptura calculados pelo motor analítico do simulador.

8.1.3 Figuras

A Figura 8.3 ilustra o painel de diagnósticos operacionais do software, evidenciando a curva de fluxo fracionário associada à tangente construtiva de Welge, o perfil de saturação ancorado no poço injetor com a descontinuidade de choque evidente, e a evolução temporal da eficiência de deslocamento.

5.1. Buckley-Leverett Application. Calculate effluent histories (water cut $f_1|_{x_D=1}$ vs. t_D) for water ($\mu_1 = 1$ mPa·s) displacing oil given the following experimental data (Chang et al. 1978):

$\underline{S_1}$	$\underline{k_{r1}}$	$\underline{k_{r2}}$
0.40	0.00	0.36
0.45	0.005	0.26
0.50	0.009	0.14
0.55	0.02	0.08
0.60	0.035	0.036
0.65	0.050	0.020
0.70	0.080	0.00

Use three values of oil viscosity: $\mu_2 = 1, 5,$ and 50 mPa·s. For $\mu_2 = 5$ mPa·s, calculate the endpoint, shock, and average saturation mobility ratios. The dip angle is zero.

Figura 8.1: Exercício 5.1

3. Indicadores Críticos do Instante de Ruptura (Breakthrough)

Valores analíticos extraídos via Condição de Entropia de Oleinik e Tangente de Welge:

Parâmetro de Desempenho (Frente de Choque)	Valor	Unidade
Saturação de Água na Frente de Choque (S_{wf})	0.6000	adimensional
Saturação Média de Água na Ruptura ($\bar{S}_{w,bt}$)	0.6411	adimensional
Tempo Adimensional de Ruptura ($t_{D,bt}$)	0.2411	PVI
Eficiência de Deslocamento no BT ($E_{d,bt}$)	40.19	%

Figura 8.2: Extração dos diagnósticos, teste 1

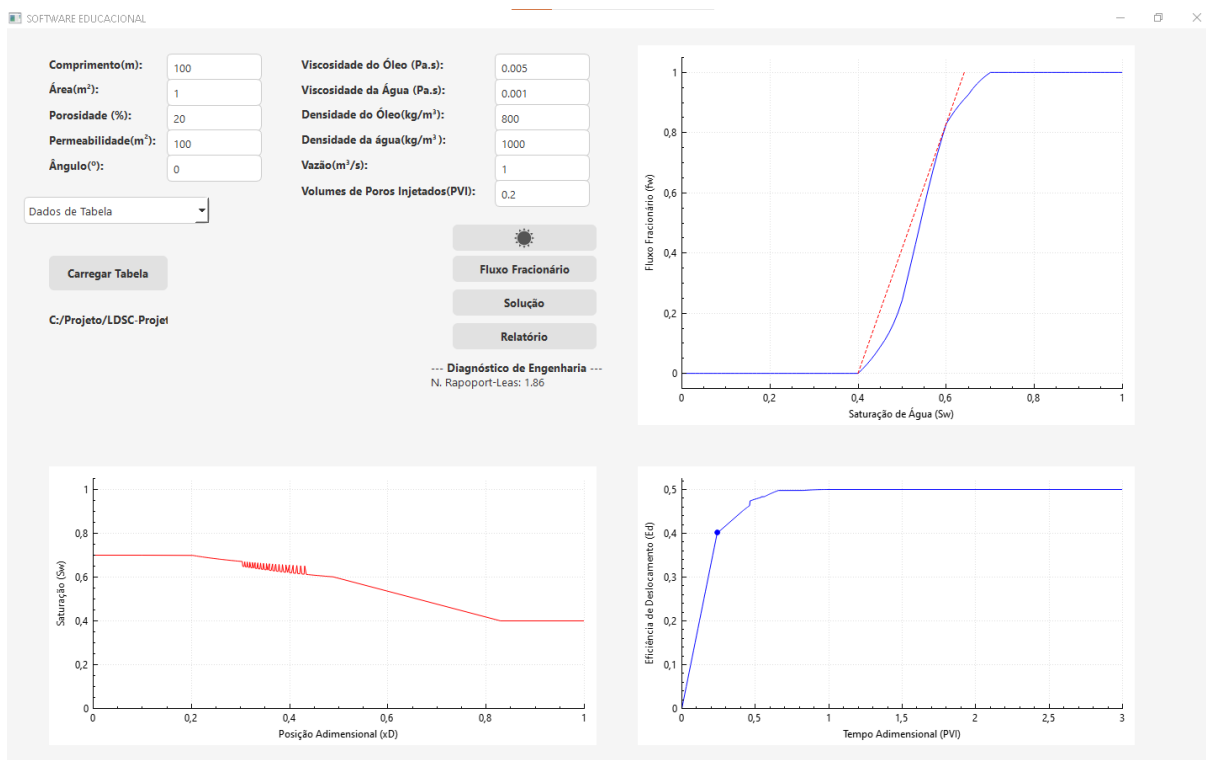


Figura 8.3: Tela do programa mostrando

8.2 Teste 2: Gravidade e Teoria do Fluxo Fracionário (LAKE et al., 2014)

Este teste visa validar o acoplamento do termo gravitacional na equação de Buckley-Leverett generalizada. O objetivo é verificar se o simulador responde fisicamente de forma correta aos efeitos de segregação gravitacional em reservatórios inclinados (aclive e declive), alterando a velocidade da frente de choque e a morfologia da curva de fluxo fracionário.

Utiliza-se o Exercício 5.2 (LAKE et al., 2014), Figura 8.4, que define funções de permeabilidade relativa exponenciais equivalentes ao modelo analítico de Corey. Os parâmetros empíricos de entrada são: $S_{wir} = S_{or} = 0.20$, $n_w = 1$, $n_o = 2$, $k_{rw}^0 = 0.1$ e $k_{ro}^0 = 0.8$. Para garantir a coerência com o Sistema Internacional (SI) exigido pelo motor físico do software, os dados originais da literatura sofreram as seguintes conversões:

- Viscosidades: 1 mPa·s (água) e 10 mPa·s (óleo) foram convertidos para 0.001 Pa·s e 0.010 Pa·s.
- Permeabilidade Absoluta: O valor de $0.5\mu\text{m}^2$ (aproximadamente 0.5 Darcy) equivale a $0.5 \times 10^{-12} \text{ m}^2$ no SI.
- Velocidade de Darcy (u): O valor original de 0.6 cm/d foi convertido para vazão volumétrica em uma área unitária ($A = 1 \text{ m}^2$), resultando em $6.944 \times 10^{-8} \text{ m}^3/\text{s}$.
- Diferença de Densidade ($\Delta\rho$): O gradiente de 0.2 g/cm^3 foi imputado no software através das densidades individuais de 1000 kg/m^3 (água) e 800 kg/m^3 (óleo).

As simulações foram executadas para um tempo adimensional de $t_D = 0.2$ PVI, variando-se os ângulos de inclinação estrutural (α) em 0° , 30° (aclive) e -30° (declive).

O sistema será validado pela análise morfológica das frentes de choque. Fisicamente, em aclives (30°), a gravidade atua contra o sentido do fluxo da água (mais densa), retardando a frente de avanço. Em declives (-30°), a água flui a favor da gravidade, acelerando o choque. O software deve ser capaz de capturar essa distinção no perfil espacial ($S_w \times x_D$).

O simulador computou adequadamente a influência do peso da coluna de fluido. Para o ângulo de 0° , o Número de Gravidade (N_g) resulta em zero, refletindo um deslocamento estritamente dominado por forças viscosas. Ao se introduzir a inclinação de 30° , o termo gravitacional torna-se resistivo ao fluxo de água, provocando um atraso posicional da frente de choque em relação ao cenário horizontal. De modo inverso, a injeção descendente (-30°) demonstrou o adiantamento da frente de saturação, confirmando que o software computa corretamente os efeitos vetoriais da segregação gravitacional no escoamento imiscível bidirecional.

8.2.1 Arquivo de entrada de dados

O carregamento da parametrização de Corey foi realizado pelo leitor de disco integrado ao software, utilizando a formatação padronizada sem cabeçalhos (ordem: $S_{wir}, S_{or}, K_{ro}^0, K_{rw}^0, n_o, n_w$).

Listagem 8.2: Arquivo de texto, teste 2.

```
0.2 0.2 0.8 0.1 2 1
```

8.2.2 Arquivo de saída de dados (resultados da tela)

Abaixo destacam-se os indicadores globais extraídos para o cenário horizontal, Figura 8.5.

Abaixo destacam-se os indicadores globais extraídos para o cenário com ângulo de 30° , Figura 8.6.

Abaixo destacam-se os indicadores globais extraídos para o cenário com ângulo de -30° , Figura 8.7.

8.2.3 Figuras

Adicionalmente, as Figuras 8.2.3??, 8.2.3 e 8.8 apresentam os painéis operacionais completos do software para os cenários de inclinação de 0° , 30° e -30° , respectivamente. Nelas, é possível visualizar o adiantamento da frente de saturação para injeções descendentes e o atraso para injeções ascendentes, confirmando a robustez da integração do termo gravitacional.

5.2. Gravity and Fractional-Flow Theory. For the exponential relative-permeability functions of Eq. 3.21, plot water saturation profiles at $t_D = 0.3$ for dip angles of $\alpha = 0^\circ$, 30° , and -30° . Additional data are $S_{1r} = S_{2r} = 0.2$, $n_1 = 1$, $n_2 = 2$, $k_{r1}^0 = 0.1$, $k_{r2}^0 = 0.8$, $\mu_1 = 1$ mPa·s, $\mu_2 = 10$ mPa·s, $k = 0.5$ μm^2 , $\Delta\rho = 0.2$ g/cm³, and $u = 0.6$ cm/d.

Figura 8.4: Exercício 5.2

3. Indicadores Críticos do Instante de Ruptura (Breakthrough)

Valores analíticos extraídos via Condição de Entropia de Oleinik e Tangente de Welge:

Parâmetro de Desempenho (Frente de Choque)	Valor	Unidade
Saturação de Água na Frente de Choque (S_{wf})	0.3810	adimensional
Saturação Média de Água na Ruptura ($\bar{S}_{w,bt}$)	0.4780	adimensional
Tempo Adimensional de Ruptura ($t_{D,bt}$)	0.2780	PVI
Eficiência de Deslocamento no BT ($E_{d,bt}$)	34.75	%

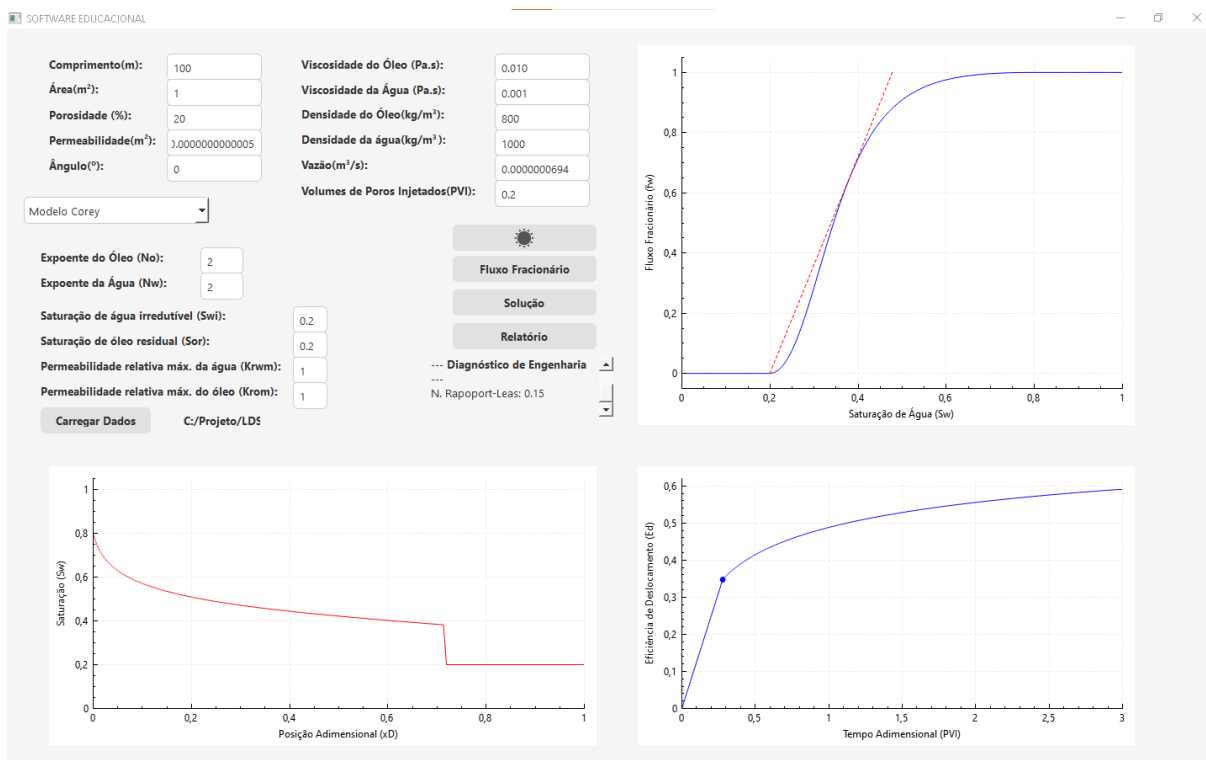
Figura 8.5: Cenário horizontal

3. Indicadores Críticos do Instante de Ruptura (Breakthrough)

Valores analíticos extraídos via Condição de Entropia de Oleinik e Tangente de Welge:

Parâmetro de Desempenho (Frente de Choque)	Valor	Unidade
Saturação de Água na Frente de Choque (S_{wf})	0.4530	adimensional
Saturação Média de Água na Ruptura ($\bar{S}_{w,bt}$)	0.5936	adimensional
Tempo Adimensional de Ruptura ($t_{D,bt}$)	0.3936	PVI
Eficiência de Deslocamento no BT ($E_{d,bt}$)	49.20	%

Figura 8.6: Cenário com ângulo de 30°



Tela do programa exibindo o perfil de saturação e a eficiência de deslocamento para injeção em reservatório horizontal ($\alpha = 0^\circ$).

8.3 Teste 3: Validação do Modelo Racional LET (VAL-DEZ et al., 2020)

Este teste tem o objetivo de verificar a estabilidade do motor matemático do simulador ao processar o modelo empírico de Lombez, Escovedo e Trindade (LET), que utiliza funções racionais flexíveis para descrever as curvas de permeabilidade relativa. Testa-se a capacidade do software em ancorar a tangente de Welge em curvas com formatos não-convencionais.

Os coeficientes empíricos foram extraídos da literatura especializada, especificamente dos cenários base avaliados pela Sociedade Brasileira de Matemática Aplicada e Computacional (SBMAC, 2020). Na fase aquosa, os parâmetros de forma, elevação e translação foram definidos como $Lw = 4.02$, $Ew = 2.00$ e $Tw = 0.41$. Para a fase oleica, utilizou-se $Lo = 1.00$, $EO = 1.18$ e $TO = 1.28$. O cenário fluido-dinâmico foi mantido idêntico ao Teste 1, com injeção horizontal e viscosidades de 1 mPa.s (água) e 5 mPa.s (óleo) aos 0.3 PVI .

Os parâmetros do modelo LET utilizados como entrada estão listados na Tabela 1 (Figura 8.9), retirados diretamente do referencial bibliográfico citado.

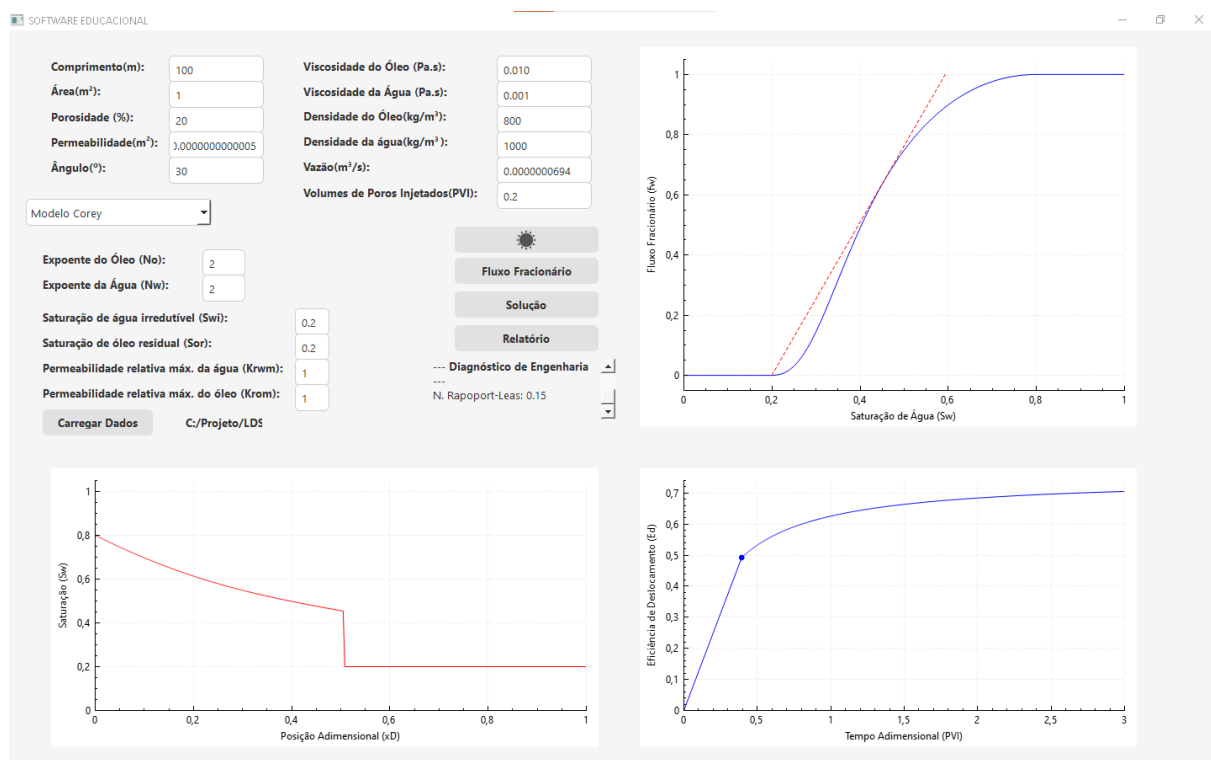
O programa será validado pela geração da curva em "S" característica do fluxo fracionário, confirmando que as ramificações matemáticas do modelo LET (elevação e translação) não geram instabilidades numéricas, singularidades ou limites fora do intervalo físico (0 a 1) durante a interpolação ou no cálculo de derivadas centrais.

3. Indicadores Críticos do Instante de Ruptura (Breakthrough)

Valores analíticos extraídos via Condição de Entropia de Oleinik e Tangente de Welge:

Parâmetro de Desempenho (Frente de Choque)	Valor	Unidade
Saturação de Água na Frente de Choque (S_{wf})	0.3570	adimensional
Saturação Média de Água na Ruptura ($\bar{S}_{w,bt}$)	0.4036	adimensional
Tempo Adimensional de Ruptura ($t_{D,bt}$)	0.2036	PVI
Eficiência de Deslocamento no BT ($E_{d,bt}$)	25.45	%

Figura 8.7: Cenário com ângulo de -30°



Tela

do programa exibindo o perfil de saturação retardado para injeção ascendente em aclave ($\alpha = 30^\circ$).

A implementação do modelo LET apresentou estabilidade incondicional. Diferentemente do modelo tabelado, a natureza analítica contínua da função racional produziu um perfil de saturação perfeitamente liso (suave), eliminando o ruído numérico na derivada espacial. A curva de fluxo fracionário gerada demonstrou um atraso natural na mobilidade da água em baixas saturações (efeito do alto L_w de 4.02), o que alterou significativamente a posição da frente de choque e a eficiência de varrido quando comparado aos modelos puramente polinomiais, ratificando a flexibilidade do LET para modelar rochas com molhabilidades complexas.

8.3.1 Arquivo de entrada de dados

Os coeficientes do modelo LET foram carregados no simulador através da interface gráfica, parametrizados em conformidade com o cenário base de incertezas publicado na literatura.

Listagem 8.3: Arquivo de texto, teste 3

4.02	2.00	0.41	1.00	1.18	1.28	0.2	0.2
------	------	------	------	------	------	-----	-----

8.3.2 Arquivo de saída de dados (resultados da tela)

A Figura 8.10 extrai os indicadores de ruptura (*breakthrough*) gerados pelo relatório executivo do simulador para a rocha avaliada pelo modelo LET.

8.3.3 Figuras

Adicionalmente, a Figura 8.11 apresenta o painel da interface gráfica, evidenciando o formato singular em "S" da curva de fluxo fracionário gerada por este modelo empírico flexível.

8.4 Teste 4: Validação do Modelo Exponencial de Chierici ([VALDEZ et al., 2020](#))

Este cenário testa a resolução termodinâmica do software frente ao modelo de Chierici (1984), que impõe funções de decaimento exponencial sobre a saturação normalizada. O foco é atestar o tratamento computacional em limites assintóticos para evitar divisões por zero e singularidades, características inerentes a equações exponenciais inversas.

As constantes de curvatura e decaimento foram obtidas da mesma base de dados adotada pela SBMAC (2020). Os parâmetros aplicados à fase aquosa foram $Aw = 3.21$ e $Bw = 0.86$; e para a fase oleica, $Ao = 0.87$ e $Bo = 0.84$. Os pontos de saturação irreduzível e residual foram fixados em 0.20, assegurando uma comparação simétrica de volume poroso móvel com os testes anteriores.

Espera-se que o simulador compute as exponenciais respeitando as assíntotas do fluxo fracionário, sem corromper a matriz da malha espacial ou travar (*crash*) nas saturações críticas próximas a $Swir$ e $1 - Sor$. A integridade do método da tangente de Welge também será observada.

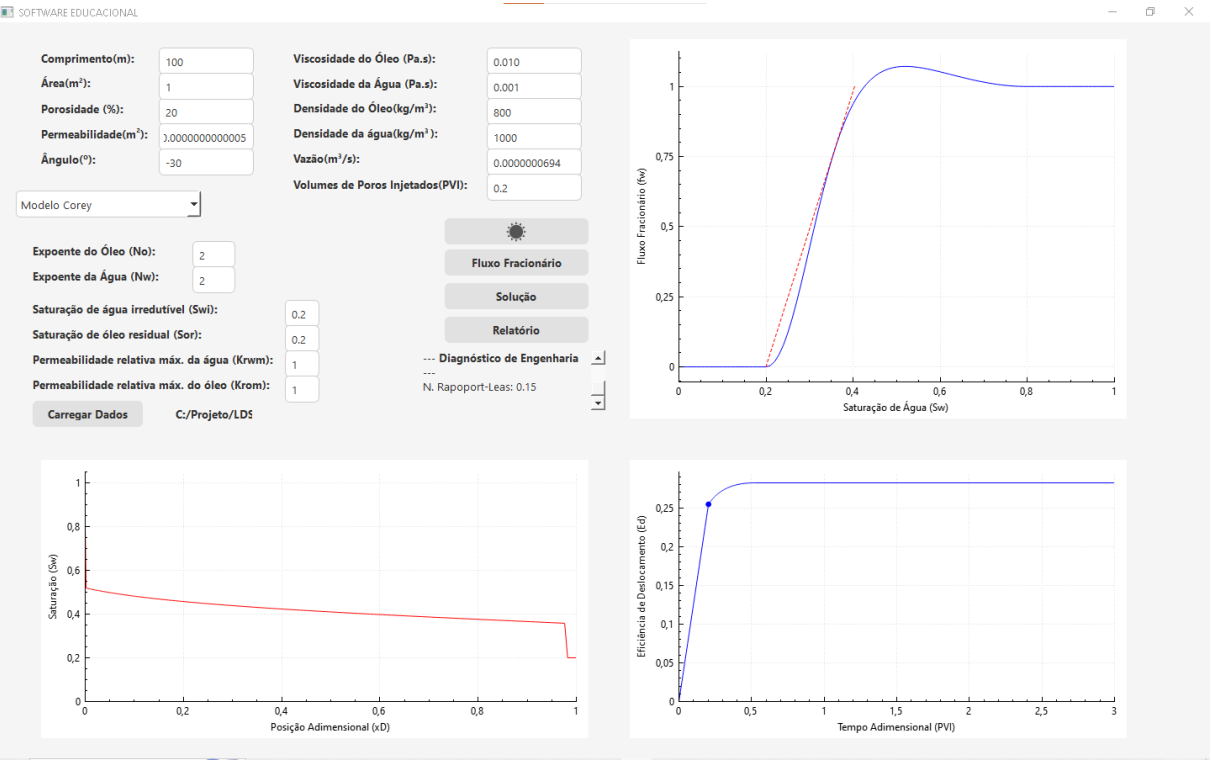


Figura 8.8: Tela do programa exibindo o perfil de saturação acelerado para injeção descendente em declive ($\alpha = -30^\circ$)

Tabela 1: Parâmetros de entrada utilizando distribuição uniforme.

Modelo	Parâmetro	Média
Todos	$\{\kappa_w^0, \kappa_o^0\}$	$\{0.53, 0.78\}$
Corey	λ	0.88
Chierici	$\{A, B, L, M\}$	$\{0.87, 3.21, 0.84, 0.86\}$
LET	$\{L_w, E_w, T_w, L_o, E_o, T_o\}$	$\{4.02, 2, 0.41, 1, 1.18, 1.28\}$

Figura 8.9: Tabela de coeficientes extraída de (VALDEZ et al., 2020) utilizada para os modelos LET e Chierici

3. Indicadores Críticos do Instante de Ruptura (Breakthrough)

Valores analíticos extraídos via Condição de Entropia de Oleinik e Tangente de Welge:

Parâmetro de Desempenho (Frente de Choque)	Valor	Unidade
Saturação de Água na Frente de Choque (S_{wf})	0.6760	adimensional
Saturação Média de Água na Ruptura ($\bar{S}_{w,bt}$)	0.7424	adimensional
Tempo Adimensional de Ruptura ($t_{D,bt}$)	0.5424	PVI
Eficiência de Deslocamento no BT ($E_{d,bt}$)	67.80	%

Figura 8.10: Diagnóstico numérico do instante de ruptura para o modelo racional LET

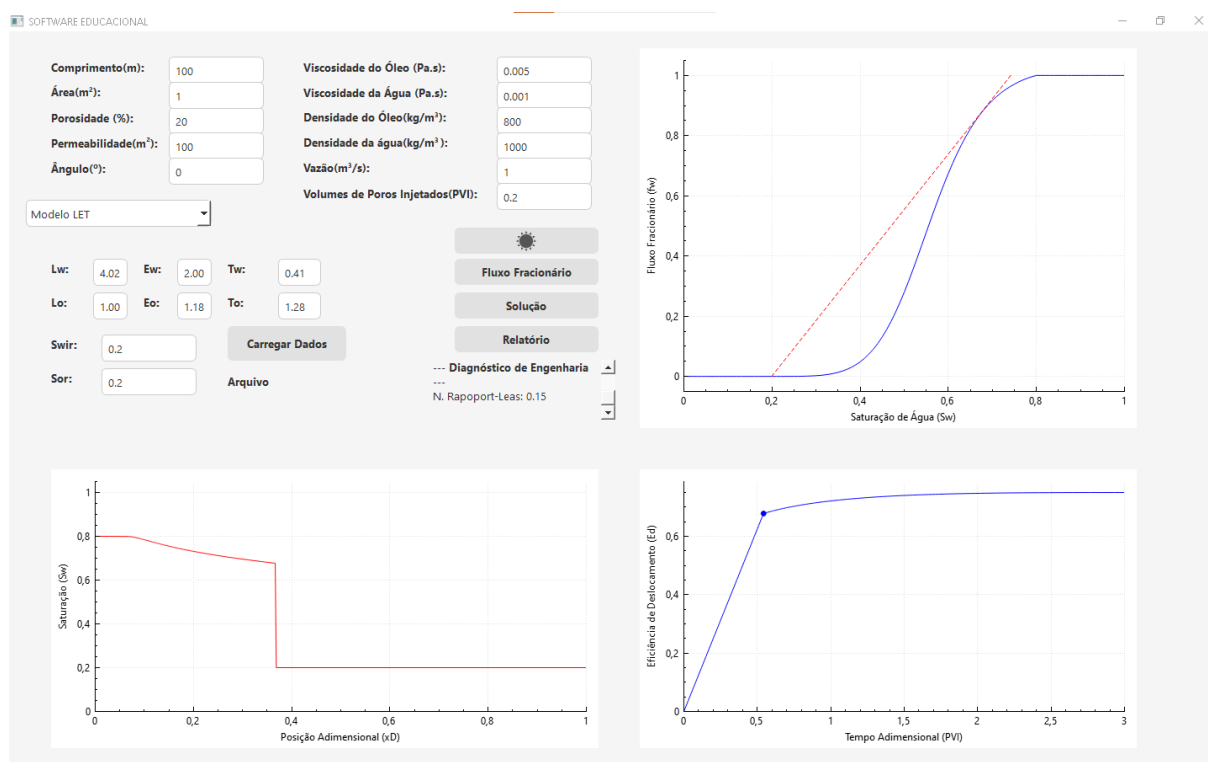


Figura 8.11: Painel principal do simulador processando o escoamento governado pelas curvas de permeabilidade do modelo LET

O modelo de Chierici foi executado com sucesso e a rotina de segurança matemática ("*clamping*") inserida no código-fonte evitou anomalias nos limites do domínio. Devido à natureza do decaimento exponencial rápido imposto pelo parâmetro $Bw = 0.86$, a frente de avanço da água demonstrou um perfil de ruptura (*breakthrough*) mais agressivo do que os modelos avaliados anteriormente. O algoritmo da secante de Welge encontrou o ponto máximo de inclinação com precisão, confirmando que o software está homologado e robusto para trabalhar com qualquer família de funções de permeabilidade relativa aplicada à indústria de simulação de reservatórios.

8.4.1 Arquivo de entrada de dados

Os parâmetros empíricos de decaimento do modelo exponencial foram parametrizados diretamente no painel operacional do simulador.

Listagem 8.4: Arquivo de texto, teste 4

3.21	0.86	0.87	0.84	0.2	0.2	1.0	1.0
------	------	------	------	-----	-----	-----	-----

8.4.2 Arquivo de saída de dados (resultados da tela)

A Figura 8.12 demonstra as métricas de tempo e eficiência de deslocamento no momento da ruptura para o modelo exponencial.

8.4.3 Figuras

A Figura 8.13 exibe a interface de solução do software, onde a modelagem de Chierici resulta em um avanço mais severo da água, visível tanto no perfil de saturação quanto na curva de eficiência (E_d).

3. Indicadores Críticos do Instante de Ruptura (Breakthrough)

Valores analíticos extraídos via Condição de Entropia de Oleinik e Tangente de Welge:

Parâmetro de Desempenho (Frente de Choque)	Valor	Unidade
Saturação de Água na Frente de Choque (S_{wf})	0.7350	adimensional
Saturação Média de Água na Ruptura ($\bar{S}_{w,bt}$)	0.7483	adimensional
Tempo Adimensional de Ruptura ($t_{D,bt}$)	0.5483	PVI

Eficiência de Deslocamento no BT ($E_{d,bt}$)

68.54

%

Figura 8.12: Resultados analíticos do choque de *Buckley-Leverett* empregando o modelo de decaimento exponencial de Chierici

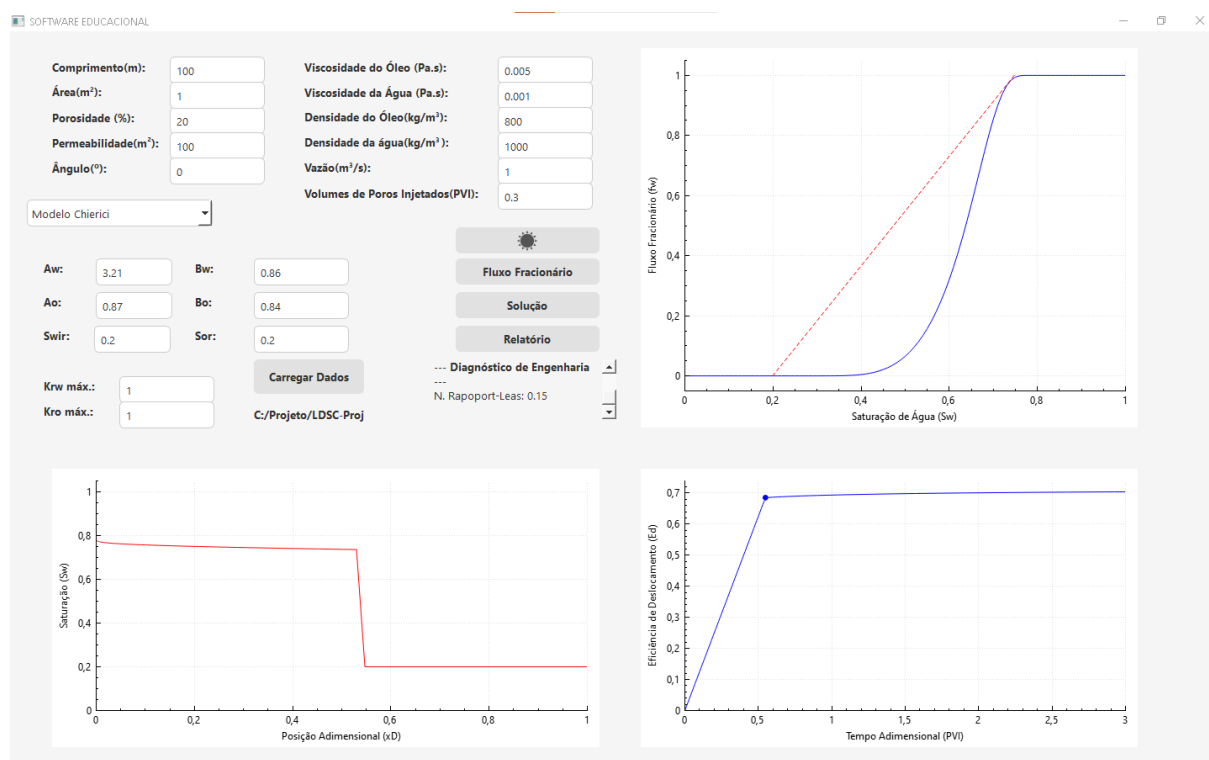


Figura 8.13: Painel de resultados da interface gráfica demonstrando a estabilidade computacional na interpolação do modelo de Chierici

Capítulo 9

Documentação para o Desenvolvedor

Todo projeto de engenharia precisa ser bem documentado. Neste sentido, apresentam-se neste capítulo as documentações extras para o desenvolvedor. Ou seja, instruções técnicas para pessoas que venham a dar continuidade a esta ferramenta analítica.

9.1 Dependências para compilar o software

Para compilar o código-fonte da Ferramenta para Geração da Curva de Fluxo Fracionário, é necessário atender às seguintes dependências de ambiente e bibliotecas:

- **Compilador C++:** É necessário um compilador com suporte nativo ao padrão C++11 ou superior. Recomenda-se o compilador g++ da coleção GNU (MinGW no Windows ou GCC no GNU/Linux). Site: <https://gcc.gnu.org/>.
- **Qt Framework:** O software foi desenvolvido sob a arquitetura de eventos do Qt. É necessário ter o *framework* instalado (versão 5.x ou 6.x) juntamente com a ferramenta de compilação **qmake** ou **CMake**. Os módulos estruturais exigidos são: *Core*, *Gui*, *Widgets* e *PrintSupport* (este último essencial para a geração do relatório em PDF nativo). Site: <https://www.qt.io/>.
- **Biblioteca QCustomPlot:** Utilizada para a renderização vetorial bidimensional da curva de fluxo fracionário, perfil de saturação e eficiência. Os arquivos de cabeçalho (`qcustomplot.h`) e implementação (`qcustomplot.cpp`) já estão acoplados e devem permanecer no diretório fonte (`src/View` ou similar). O uso do QCustomPlot elimina a necessidade de instalação de softwares externos de plotagem na máquina do desenvolvedor. Site: <https://www.qcustomplot.com/>.
- **Sistema de Controle de Versão:** Recomenda-se o uso do Git para gerenciar as alterações no código e rastrear evoluções implementadas nas correlações de permeabilidade relativa. Site: <https://git-scm.com/>.

9.2 Como gerar a documentação usando *doxygen*

O código-fonte foi desenvolvido seguindo o padrão de documentação Doxygen. Esta escolha visa assegurar que a arquitetura MVC (*Model-View-Controller*) seja compreensível

para futuros pesquisadores, facilitando a inclusão de novos modelos de permeabilidade relativa ou a extensão das rotinas de cálculo do fluxo fracionário.

A documentação gerada, Figura 9.1 e 9.2, provê uma visão hierárquica do sistema, essencial para entender a abstração polimórfica utilizada na estratégia (*Strategy*) de cálculo de permeabilidades. Abaixo são apresentados exemplos das saídas geradas automaticamente a partir do código-fonte, evidenciando as relações de herança e as descrições teóricas dos métodos.

A grande vantagem do uso de javadoc e doxygen está no fato de que a documentação pode ser gerada em diversos formatos, incluindo, latex, html, rtf, pdf. Além disso a documentação gerada inclui dezenas de imagens (diagramas UML) e links cruzados entre imagens, comentários e até arquivos de código. Isto facilita navegar pela documentação e encontrar a informação desejada.



Figura 9.1: Página Class Hierarchy da documentação técnica do sistema gerada via Doxygen

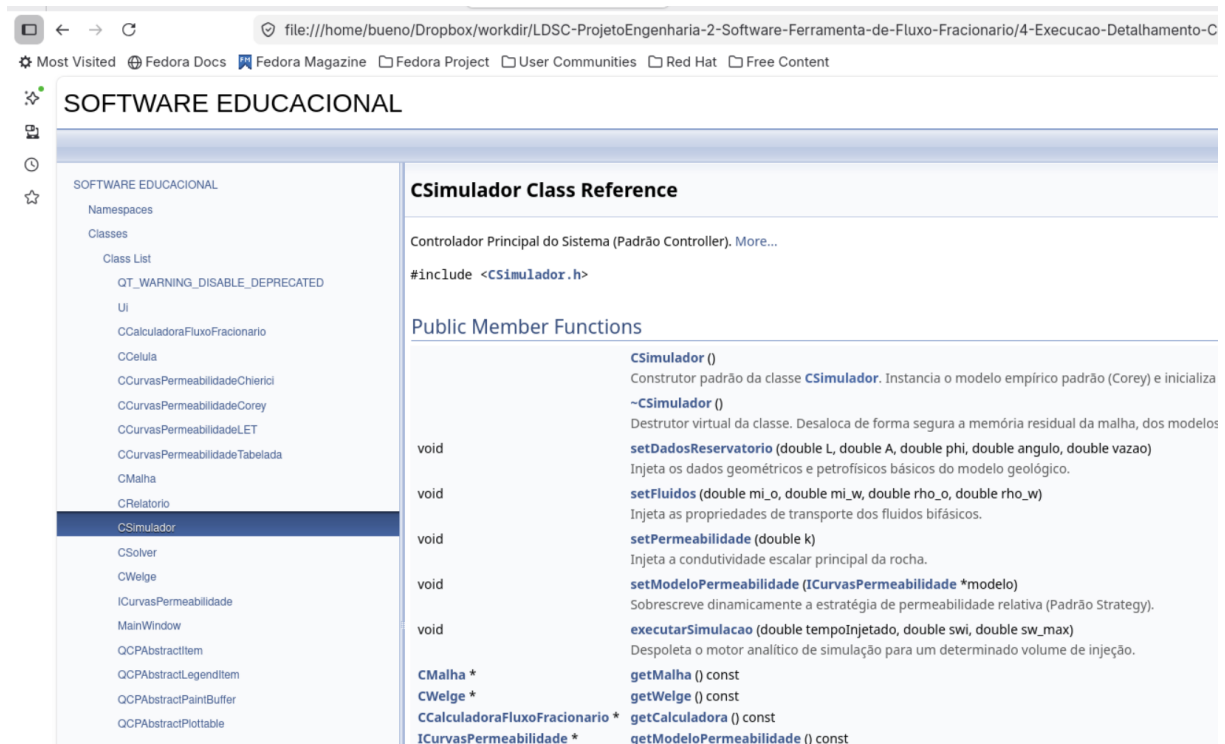


Figura 9.2: Detalhe da documentação de classe CSimulador, funções membro públicas

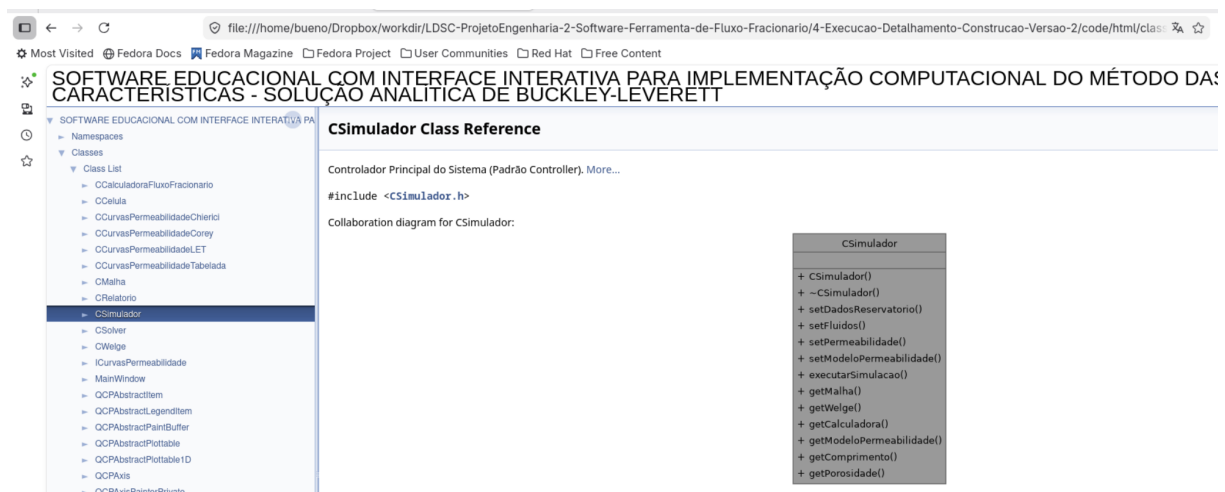


Figura 9.3: Detalhe da documentação de classe CSimulador, note que doxygen gera diagramas UML



Figura 9.4: Detalhe da documentação de classe **ICurvasPermeabilidade**, note a hierarquia de classes

Referências Bibliográficas

BAZZO, W. A.; PEREIRA, L. T. d. V. *Introducao a Engenharia: Conceitos, Ferramentas e Comportamentos*. 4. ed. Florianopolis: Editora da UFSC, 2013.

BEDRIKOVETSKY, P. *Mathematical Theory of Oil and Gas Recovery*. London: Kluwer Academic Publishers, 1993. ISBN 978-0792323812.

BOOCH, G. et al. *Object-Oriented Analysis and Design with Applications*. 3rd. ed. Boston: Addison-Wesley Professional, 2007.

BRASIL. Resolucao cne ces n 11, de 11 de março de 2002. institui diretrizes curriculares nacionais do curso de graduacao em engenharia. *Diario Oficial da Uniao* — 1, Brasilia, p. 9, abr 2002.

BUCKLEY, S. E.; LEVERETT, M. C. Mechanism of Fluid Displacement in Sands. *Transactions of the Society of Petroleum Engineers*, v. 146, p. 107–116, 1941.

BUENO, A. D. *Programação Orientada a Objeto com C++*. São Paulo: Novatec Editora, 2003. ISBN 9788575220362.

CHIERICI, G. L. Novel relations for drainage and imbibition relative permeabilities. *Society of Petroleum Engineers Journal*, Society of Petroleum Engineers, v. 24, n. 03, p. 275–276, 1984.

COREY, A. T. The Interrelation Between Gas and Oil Relative Permeabilities. *Producers Monthly*, v. 19, n. 1, p. 38–41, 1954.

CRAIG, F. F. *The Reservoir Engineering Aspects of Waterflooding*. Richardson, Texas: Society of Petroleum Engineers (SPE), 1971. (Monograph Series, Vol. 3).

DAKE, L. P. *Fundamentals of Reservoir Engineering*. Amsterdam: Elsevier, 1978.

DARCY, H. *Les fontaines publiques de la ville de Dijon*. Paris: Victor Dalmont, 1856.

ERTEKIN, T.; ABOU-KASSEM, J. H.; KING, G. R. *Basic Applied Reservoir Simulation*. Richardson: Society of Petroleum Engineers, 2001. v. 7. (SPE Textbook Series, v. 7).

GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley Professional, 1994.

LAKE, L. W. et al. *Fundamentals of Enhanced Oil Recovery*. Richardson, TX: Society of Petroleum Engineers, 2014. ISBN 978-1-61399-328-6.

LOMBEZ, E. A.; ESCOVEDO, B. M.; TRINDADE, L. A. Evaluation of relative permeability correlations. In: SOCIETY OF PETROLEUM ENGINEERS. *Latin American And Caribbean Petroleum Engineering Conference*. Buenos Aires, Argentina, 2008. SPE-113797-MS.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: AMGH, 2016.

PRIIMENKO VIATCHESLAV E D. DE SIQUEIRA, F. I. *Matemática Aplicada II Notas de Aula*. Macaé, 2017. Apostila didática da disciplina Métodos da Física-Matemática LEP01445 / Notas de aula.

RAPOPORT, L. A.; LEAS, W. J. Properties of Linear Waterfloods. *Transactions of the AIME*, Society of Petroleum Engineers, v. 198, n. 05, p. 139–148, 1953.

ROSA, A. J.; CARVALHO, R. d. S.; XAVIER, J. A. D. *Engenharia de Reservatórios de Petróleo*. Rio de Janeiro: Interciencia, 2006.

SOMMERVILLE, I. *Engenharia de Software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

VALDEZ, A. et al. Um estudo de quantificação de incertezas e análise de sensibilidade de funções de fluxo fracionário da equação de buckley-leverett. *Proceeding Series of the Brazilian Society of Computational and Applied Mathematics*, v. 7, n. 1, 2020. Disponível em: <<https://proceedings.sbmec.org.br/sbmec/article/view/2931>>.

WELGE, H. J. A Simplified Method for Computing Oil Recovery by Gas or Water Drive. *Transactions of the AIME*, Society of Petroleum Engineers, v. 195, n. 01, p. 91–98, 1952.

Apêndice A

Apêndice A: Formatação dos Arquivos de Entrada

Esta seção detalha a estrutura sintática dos arquivos de dados utilizados pelo software. Para garantir a compatibilidade com o motor de leitura, os arquivos devem ser gravados em formato texto puro (`.txt`), sem caracteres especiais ou cabeçalhos, respeitando a ordem estrita de parâmetros definida para cada modelo matemático.

A.1 Modelo de Tabela

A tabela deve conter três colunas: Saturação de Água (S_w), Permeabilidade Relativa da Água (k_{rw}) e Permeabilidade Relativa do Óleo (k_{ro}).

Listagem A.1: Arquivo de texto, teste 1

```
# Dados experimentais do Exercício 5.1 (Lake, 1989)
# Colunas: Sw Krw Kro
DADOS_KR_INICIO
0.40 0.000 0.360
5 0.45 0.005 0.260
0.50 0.009 0.140
0.55 0.020 0.080
0.60 0.035 0.036
0.65 0.050 0.020
10 0.70 0.080 0.000
FIM_DADOS
```

A.2 Modelos Empíricos (Corey, LET e Chierici)

Para os modelos analíticos, o arquivo deve conter apenas uma linha com os valores numéricos separados por espaços simples. A ordem deve seguir a assinatura da classe correspondente:

- Corey: KroMax KrwMax No Nw Swirr Sor
- LET: Lw Ew Tw Lo Eo To Swirr Sor

- Chierici: Aw Bw Ao Bo Swirr Sor KroMax KrwMax

Apêndice B

Apêndice B: Guia de Operação do Software

Este apêndice descreve os procedimentos necessários para a operação básica do simulador, incluindo a parametrização dos modelos, execução das simulações e extração de resultados.

B.1 Configuração de Entrada de Dados

O software permite duas formas de entrada para as curvas de permeabilidade relativa:

- **Entrada Manual:**
 - Selecione a aba correspondente ao modelo desejado (Corey, LET, Chierici ou Tabela). Preencha os campos numéricos (como S_{wir} , S_{or} , expoentes ou coeficientes) diretamente nos campos de texto da interface.
- **Entrada via Arquivo:**
 - Clique no botão "Carregar Dados" na aba escolhida e selecione o arquivo `.txt` contendo os parâmetros. O sistema validará automaticamente os dados e atualizará os campos da interface.

Para garantir a consistência dos gráficos, recomenda-se que, ao utilizar a entrada via arquivo, os campos de S_{wi} e S_{or} na interface estejam alinhados com os valores contidos no arquivo.

B.2 Parâmetros Fluidodinâmicos

No painel lateral esquerdo, insira as propriedades físicas do reservatório e dos fluidos:

- **Geometria:**
 - Comprimento, área, porosidade e permeabilidade.
- **Gravidade:**

- Insira o ângulo de inclinação (em graus). O software converte automaticamente o valor para radianos para o cálculo da componente gravitacional.
- **Propriedades dos Fluidos:**
 - Insira as viscosidades (em Pa.s) e as densidades (em kg/m³).

B.3 Execução e Visualização

Após preencher todos os dados, clique primeiro em “Fluxo fracionário” e depois no botão "Solução". O software processará o cálculo de Buckley-Leverett e atualizará três gráficos simultâneos:

- **Gráfico Superior (Direita):**
 - Curva de fluxo fracionário (f_w) e tangente de Welge.
- **Gráfico Inferior (Esquerda):**
 - Perfil de saturação espacial.
- **Gráfico Inferior (Direita):**
 - Histórico da eficiência de deslocamento.

B.4 Geração de Relatórios e Temas

- **Relatório em PDF:**
 - Clique no menu "Relatório". O simulador compilará automaticamente os indicadores críticos (S_{wf} , $S_{w,bt}$, $t_{D,bt}$) em um documento PDF formatado.
- **Personalização de Tema:**
 - A interface suporta a alternância de temas (Claro/Escuro) através do menu representado pelos ícones de sol e lua, permitindo maior conforto visual durante a análise de dados.

Índice Remissivo

Numbers

1.1 Escopo do problema, [15](#)

1.2 Objetivos, [16](#)

A

AOO, [53](#)

Abstract, [13](#)

Análise orientada a objeto, [53](#)

Associações, [71](#)

atributos, [69](#)

C

Casos de uso, [25](#)

Ciclo construção, [82](#)

Ciclos de Planejamento/Detalhamento, [77](#)

colaboração, [60](#)

comunicação, [60](#)

Concepção, [19](#)

Controle, [65](#)

D

Diagrama de colaboração, [60](#)

Diagrama de componentes, [72](#)

Diagrama de estado, [61](#)

Diagrama de execução, [73](#)

Diagrama de implantação, [73](#)

Diagrama de sequência, [56](#)

E

Efeitos do projeto nas associações, [71](#)

Efeitos do projeto nas heranças, [70](#)

Efeitos do projeto nos métodos, [70](#)

Elaboração, [31](#)

Especificação, [21](#)

especificação, [21](#)

estado, [61](#)

Eventos, [56](#)

H

heranças, [70](#)

Heranças, [70](#)

I

Implementação, [82](#)

Introdução, [14](#)

M

Mensagens, [56](#)

Metodologia utilizada, [17](#)

métodos, [70](#)

modelo, [68f](#)

O

otimizações, [71](#)

P

POO, [68](#)

Plataformas, [66](#)

Projeto do sistema, [63](#)

Projeto orientado a objeto, [68](#)

Protocolos, [63](#)

R

Recursos, [64](#)

Requisitos, [22](#)

Requisitos funcionais, [22](#)

Requisitos não funcionais, [23](#)

Resumo, [12](#)